



CS2569/09

รายงานโครงการคอมพิวเตอร์

ระบบแจ้งซ่อมภายในหอพัก

Dormitory Repair Request System

โดย

673380056-7	นายรพีภัทร	แก้วคุณ
673380068-0	นางสาวสุชาดา	ศรีจักรโคตร
673380286-0	นางสาวพิไลพร	คำเวียง
673380297-5	นางสาวหทัยพัช	วิสุทธิธรรม
673380299-1	นางสาวเจตนิพัฐ	แก้วทรัพย์

อาจารย์ที่ปรึกษา รศ.ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353002 หลักการออกแบบพัฒนาซอฟต์แวร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

(เดือน ตุลาคม พ.ศ. 2569)

คำนำ

รายงานฉบับนี้จัดทำขึ้นเพื่อประกอบการพัฒนาระบบแจ้งซ่อมภายในหอพัก (Dormitory Repair Request System) มีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับช่วยอำนวยความสะดวกในการแจ้งปัญหาและติดตามการซ่อมแซมภายในหอพัก โดยระบบถูกออกแบบมาเพื่อแก้ไขปัญหาการแจ้งซ่อมในรูปแบบเดิมที่อาจเกิดความล่าช้า การจัดเก็บข้อมูลไม่เป็นระบบ และการติดตามสถานะของงานซ่อมทำได้ยาก

ระบบที่พัฒนาขึ้นรองรับการทำงานของผู้ใช้งานหลายบทบาท ได้แก่ ผู้แจ้ง (Reporter) ผู้ดูแลระบบ (Admin) และช่าง (Technician) โดยผู้แจ้งซึ่งเป็นผู้พักอาศัย สามารถส่งคำร้องแจ้งซ่อม ระบุรายละเอียดและช่วงเวลาที่จะสะดวกในการเข้าซ่อม รวมถึงติดตามสถานะของคำร้องได้ ผู้ดูแลระบบสามารถตรวจสอบ อนุมัติหรือปฏิเสธคำร้อง และมอบหมายงานให้ช่าง ส่วนช่างสามารถตรวจสอบงานที่ได้รับมอบหมายและปรับปรุงสถานะการดำเนินงาน ทั้งนี้ระบบยังรองรับการยืนยันผลการซ่อมจากผู้แจ้ง เพื่อให้กระบวนการแจ้งซ่อมมีความชัดเจนและสามารถตรวจสอบย้อนหลังได้

ในการพัฒนาระบบได้ประยุกต์ใช้แนวความคิดทางวิศวกรรมซอฟต์แวร์และหลักการออกแบบซอฟต์แวร์ เช่น SOLID Principles, Layered Architecture, MVC, Repository Pattern และ Service Layer รวมถึง Design Pattern ตามรูปแบบของ GoF ได้แก่ State Pattern, Command Pattern และ Observer Pattern เพื่อช่วยจัดการสถานะของคำร้อง การดำเนินการตามคำสั่ง และการแจ้งเตือนเมื่อเกิดการเปลี่ยนแปลงของสถานะ นอกจากนี้ยังมีการออกแบบฐานข้อมูลเชิงสัมพันธ์และใช้ PostgreSQL ในการจัดเก็บข้อมูล โดยระบบสามารถทำงานร่วมกับ Supabase สำหรับการจัดการฐานข้อมูลและการยืนยันตัวตน และรองรับการนำระบบไปใช้งานบนระบบ Cloud

คณะผู้จัดทำหวังเป็นอย่างยิ่งว่ารายงานฉบับนี้จะช่วยแสดงให้เห็นถึงกระบวนการวิเคราะห์ ออกแบบ พัฒนา และทดสอบระบบจัดการแจ้งซ่อมหอพัก ตลอดจนการประยุกต์ใช้หลักการและรูปแบบการออกแบบซอฟต์แวร์ในการพัฒนาระบบให้มีโครงสร้างที่เป็นระเบียบ สามารถดูแลและพัฒนาต่อยอดได้ในอนาคต หากรายงานฉบับนี้มีข้อผิดพลาดหรือข้อบกพร่องประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำและข้อเสนอแนะเพื่อนำไปปรับปรุงให้มีความสมบูรณ์ยิ่งขึ้น

คณะผู้จัดทำ

ตุลาคม 2569

สารบัญ

คำนำ.....	ก
สารบัญ.....	ข
สารบัญภาพ.....	ง
สารบัญตาราง.....	ฉ
บทที่ 1.....	7
1.1 ที่มาและความสำคัญ.....	7
1.2 จุดประสงค์ของการศึกษา.....	7
1.3 ขอบเขตของการศึกษา.....	8
1.4 ประโยชน์ที่คาดว่าจะได้รับ.....	10
1.5 แผนการดำเนินงาน.....	11
บทที่ 2.....	12
2.1 ทฤษฎีที่เกี่ยวข้อง.....	12
บทที่ 3.....	16
3.1 ศึกษาความต้องการของระบบ.....	16
3.2 วิเคราะห์และออกแบบกระบวนการทำงานของระบบ.....	17
3.3 พัฒนาระบบ.....	91
3.4 พัฒนารฐานข้อมูล.....	92
3.5 ทดสอบระบบ.....	92
บทที่ 4.....	95
4.1 ผลการพัฒนาเว็บไซต์แอปพลิเคชัน.....	95
4.2 การทดสอบระบบแจ้งซ่อมในหอพัก.....	103

สารบัญ (ต่อ)

บทที่ 5.....	107
5.1 สรุปผลการดำเนินงาน.....	107
5.2 ปัญหาที่พบในการทำงาน.....	108
5.3 แนวทางในการพัฒนาต่อ.....	108

สารบัญภาพ

ภาพที่ 1 Use Case Diagram ส่วนจัดการบัญชี	17
ภาพที่ 2 Use Case Diagram ส่วนจัดการคำร้องแจ้งซ่อม	18
ภาพที่ 3 Use Case Diagram ส่วนมอบหมายงานช่าง	20
ภาพที่ 4 Use Case Diagram ส่วนจัดการบัญชีและผู้พักอาศัย	21
ภาพที่ 5 Domain Model ของระบบแจ้งซ่อมในหอพัก	23
ภาพที่ 6 Class Diagram ของระบบ	26
ภาพที่ 7 Class Diagram ส่วน State Pattern	30
ภาพที่ 8 Class Diagram ส่วน Command Pattern	31
ภาพที่ 9 Class Diagram ส่วน Observer Pattern	33
ภาพที่ 10 Sequence Diagram การสมัครสมาชิกและเข้าสู่ระบบ	44
ภาพที่ 11 Sequence Diagram การส่งคำร้องแจ้งซ่อม	46
ภาพที่ 12 Sequence Diagram การดูและจัดการคำร้องของผู้แจ้ง	48
ภาพที่ 13 Sequence Diagram การตรวจสอบคำร้องโดยผู้ดูแลระบบ	51
ภาพที่ 14 Sequence Diagram การมอบหมายงานให้ช่าง	53
ภาพที่ 15 Sequence Diagram การดำเนินงานซ่อม	55
ภาพที่ 16 Sequence Diagram การยืนยันผลการซ่อม	57
ภาพที่ 17 Sequence Diagram การจัดการข้อมูลบัญชีและผู้พักอาศัย	59
ภาพที่ 18 Activity Diagram – Register	61
ภาพที่ 19 Activity Diagram: Login	63
ภาพที่ 20 Activity Diagram: Submit Repair Request	64
ภาพที่ 21 Activity Diagram: View Repair History	65
ภาพที่ 22 Activity Diagram: Reporter Delete Repair Request	66
ภาพที่ 23 Activity Diagram: Track Repair Status	68
ภาพที่ 24 Activity Diagram: View All Repair Requests	69
ภาพที่ 25 Activity Diagram: View Request Details	70
ภาพที่ 26 Activity Diagram: Review Repair Request	72
ภาพที่ 27 Activity Diagram: Assign Job to Technician	74

สารบัญภาพ (ต่อ)

ภาพที่ 29 Activity Diagram: Confirm Repair Completion	78
ภาพที่ 30 Activity Diagram: View Daily Repair Jobs	80
ภาพที่ 31 Activity Diagram: Update Repair Job Status	82
ภาพที่ 32 State Diagram	83
ภาพที่ 33 Component Diagram	86
ภาพที่ 34 Deployment Diagram	89
ภาพที่ 35 หน้าเข้าสู่ระบบ	95
ภาพที่ 36 หน้าสมัครสมาชิกของผู้แจ้ง	96
ภาพที่ 37 หน้าแจ้งซ่อมใหม่	96
ภาพที่ 38 หน้ารายการคำร้องของผู้แจ้ง	97
ภาพที่ 39 หน้ารายละเอียดและสถานะคำร้อง	97
ภาพที่ 40 หน้าลบคำร้องแจ้งซ่อม	98
ภาพที่ 41 หน้ารายการคำร้องทั้งหมด	98
ภาพที่ 42 หน้ารายละเอียดคำร้องและการพิจารณา	99
ภาพที่ 43 หน้ามอบหมายงานให้ช่าง	99
ภาพที่ 44 หน้าตรวจสอบและยืนยันผลการซ่อม	100
ภาพที่ 45 หน้าจัดการข้อมูลช่าง	100
ภาพที่ 46 หน้าจัดการข้อมูลผู้พักอาศัย	101
ภาพที่ 47 หน้างานซ่อมประจำวัน	101
ภาพที่ 48 หน้าการแจ้งเตือนของผู้แจ้ง	102
ภาพที่ 49 หน้าการแจ้งเตือนของผู้ดูแลระบบ	102
ภาพที่ 50 หน้าการแจ้งเตือนของช่าง	103

สารบัญตาราง

ตารางที่ 1	แผนการดำเนินงาน	11
ตารางที่ 2	คำอธิบายเอนทิตี.....	35
ตารางที่ 3	ความสัมพันธ์ระหว่างเอนทิตี.....	37
ตารางที่ 4	ผู้แจ้ง (reporter).....	38
ตารางที่ 5	ผู้ดูแลระบบ (admin).....	39
ตารางที่ 6	ช่าง (technician)	39
ตารางที่ 7	ผู้พักอาศัย (resident).....	39
ตารางที่ 8	ตึก (building).....	40
ตารางที่ 9	ห้อง (room)	40
ตารางที่ 10	คำร้องแจ้งซ่อม (repair_request).....	40
ตารางที่ 11	ประวัติการเปลี่ยนแปลงสถานะคำร้อง (repair_request_status_history)	41
ตารางที่ 12	รายการมอบหมายงาน (repair_assignment).....	42
ตารางที่ 13	ประวัติการเปลี่ยนแปลงสถานะรายการมอบหมายงาน (repair_assignment_status_history)	42
ตารางที่ 14	การแจ้งเตือน (notification).....	43
ตารางที่ 15	สรุประดับการทดสอบ เครื่องมือ และจำนวนกรณีทดสอบ	104
ตารางที่ 16	ชุดทดสอบจำแนกตามบทบาทผู้ใช้งาน	104

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

หอพักมีบทบาทสำคัญอย่างยิ่งต่อการดำเนินชีวิตของผู้คน ไม่ว่าจะเป็นนักศึกษา หรือวัยทำงาน การบริหารจัดการหอพักอย่างมีประสิทธิภาพจึงเป็นหัวใจสำคัญในการสร้างความพึงพอใจและรักษามาตรฐานของหอพัก โดยหนึ่งในกระบวนการทำงานหลักที่เกิดขึ้นอย่างสม่ำเสมอคือการแจ้งซ่อมอุปกรณ์และสาธารณูปโภคที่ชำรุด

ระบบการรับแจ้งซ่อมในหอพักมีหลายรูปแบบ เช่น รับแจ้งผ่านแอปพลิเคชันไลน์ รับแจ้งปากเปล่า รับแจ้งรูปแบบกระดาษ ซึ่งเกิดปัญหาด้านข้อมูลตกหล่นหรือสูญหาย และระบบแจ้งซ่อมออนไลน์ เช่น ระบบแจ้งซ่อมภายในหอในของมหาวิทยาลัยขอนแก่น <https://dorm.kku.ac.th/dorm-repair/> ระบบดังกล่าวยังมีข้อจำกัดสำคัญด้านการจำกัดสิทธิ์ของข้อมูล โดยระบบเปิดให้ผู้ใช้งานทุกคนสามารถเข้าถึงและมองเห็นรายการแจ้งซ่อมทั้งหมด

จากปัญหาข้างต้น นำมาสู่การพัฒนาระบบแจ้งซ่อมภายในหอพัก เพื่อเป็นสื่อกลางในการเชื่อมโยงระหว่างผู้พักอาศัย เจ้าหน้าที่หอพัก และช่างซ่อมของหอพักเข้าด้วยกัน

1.2 จุดประสงค์ของการศึกษา

เพื่อออกแบบและพัฒนาระบบแจ้งซ่อมภายในหอพักในรูปแบบเว็บแอปพลิเคชันด้วย Spring Boot โดยประยุกต์ใช้ SOLID Principles และ Design Patterns

1.3 ขอบเขตของการศึกษา

การศึกษานี้มุ่งเน้นการออกแบบและพัฒนาระบบแจ้งซ่อมในรูปแบบเว็บแอปพลิเคชัน (Web Application) เพื่อสนับสนุนการแจ้งซ่อม การจัดการคำร้อง และการติดตามสถานะดำเนินงาน โดยพัฒนาด้วยภาษา Java และใช้ Spring Boot เป็นเฟรมเวิร์กหลักในการพัฒนาระบบ ร่วมกับฐานข้อมูล PostgreSQL สำหรับจัดเก็บข้อมูลที่เกี่ยวข้องกับผู้ใช้งาน ห้องพัก คำร้องแจ้งซ่อม และประวัติการเปลี่ยนแปลงสถานะ ทั้งนี้ ระบบแบ่งขอบเขตการศึกษาออกเป็นด้านต่าง ๆ ดังนี้

1.3.1 ด้านผู้ใช้งาน

ระบบกำหนดผู้ใช้งานออกเป็น 3 บทบาท ได้แก่ ผู้พักอาศัย (Reporter) ผู้ดูแลระบบ (Admin) และช่าง (Technician) โดยแต่ละบทบาทมีขอบเขตการใช้งานแตกต่างกันตามหน้าที่และสิทธิ์ของผู้ใช้งาน ดังนี้

- (1) ผู้แจ้ง (Reporter) ซึ่งเป็นผู้พักอาศัย สามารถเข้าสู่ระบบและจัดการข้อมูลส่วนบุคคลที่ระบบอนุญาตให้แก้ไข สามารถส่งคำร้องแจ้งซ่อมโดยระบุประเภทงาน รายละเอียดปัญหา วันที่และช่วงเวลาที่สะดวกให้เข้าดำเนินการ รวมถึงหมายเหตุเพิ่มเติม สามารถตรวจสอบประวัติคำร้อง ดูรายละเอียดคำร้อง ติดตามสถานะ และลบคำร้องได้ในกรณีที่คำร้องยังอยู่ในสถานะที่ระบบกำหนดให้ยกเลิกได้
- (2) ผู้ดูแลระบบ (Admin) ซึ่งเป็นเจ้าหน้าที่หอพัก มีหน้าที่จัดการคำร้องแจ้งซ่อม ตรวจสอบข้อมูลคำร้อง อนุมัติหรือปฏิเสธคำร้อง และจัดการข้อมูลที่เกี่ยวข้องกับช่างซ่อมบำรุง รวมถึงการมอบหมายงานให้แก่ช่างตามขอบเขตการทำงานของระบบ
- (3) ช่าง (Technician) สามารถตรวจสอบรายการงานซ่อมที่ได้รับมอบหมาย ดูรายละเอียดของงาน และเปลี่ยนสถานะการดำเนินงานของงานที่ได้รับมอบหมายได้ โดยการเปลี่ยนสถานะของงานซ่อมและสถานะของคำร้องถูกแยกออกจากกันตามกระบวนการทำงานของระบบ

1.3.2 ด้านการจัดการคำร้องแจ้งซ่อม

ระบบรองรับการสร้างและจัดการคำร้องแจ้งซ่อม โดยข้อมูลคำร้องประกอบด้วยผู้แจ้ง ห้องพัก ประเภทงาน รายละเอียดปัญหา หมายเหตุของผู้แจ้ง วันที่และเวลาที่สะดวกให้เข้าดำเนินการ รวมถึงข้อมูลวันที่และเวลาที่เกี่ยวข้องกับการดำเนินงาน ระบบกำหนดสถานะของคำร้องด้วย Enumeration ได้แก่ รอดำเนินการ (PENDING), อนุมัติคำร้อง (APPROVED), ปฏิเสธคำร้อง (REJECTED), กำลังดำเนินการ (IN_PROGRESS) และ ดำเนินการสำเร็จ (COMPLETED) เพื่อควบคุมลำดับการดำเนินงานของคำร้อง

ประเภทงานซ่อมถูกจัดเก็บในรูปแบบ Enumeration เช่น งานไฟฟ้า(ELECTICAL), งานประปา (PLUMBING), งานไม้ (CARPENTRY) และงานประเภทอื่น ๆ (OTHER)

1.3.3 ด้านการจัดการสิทธิ์และการเข้าสู่ระบบ

ระบบมีการแบ่งสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้งาน ได้แก่ ADMIN, REPORTER และ TECHNICIAN เพื่อให้ผู้ใช้งานสามารถเข้าถึงเฉพาะส่วนของระบบที่เกี่ยวข้องกับหน้าที่ของตนเอง การเข้าสู่ระบบใช้ระบบยืนยันตัวตนของ Supabase Auth และระบบจัดเก็บ User ID ที่ได้รับจากระบบยืนยันตัวตนไว้ใน Session เพื่อใช้ระบุผู้ใช้งานในระหว่างการทำงานของระบบ

1.3.4 ด้านการจัดการข้อมูล

ระบบจัดเก็บข้อมูลหลัก ได้แก่ ข้อมูลผู้ใช้งาน ข้อมูลผู้พักอาศัย ข้อมูลห้องพัก ข้อมูลอาคาร ข้อมูลคำร้องแจ้งซ่อม ข้อมูลการมอบหมายงาน และประวัติการเปลี่ยนแปลงสถานะคำร้อง โดยมีการกำหนดความสัมพันธ์ระหว่างข้อมูล เช่น ผู้พักอาศัยมีความสัมพันธ์กับห้องพัก ผู้แจ้งมีความสัมพันธ์กับผู้ใช้งานและผู้พักอาศัย และคำร้องแจ้งซ่อมมีความสัมพันธ์กับผู้แจ้ง ห้องพัก และเจ้าหน้าที่ที่เกี่ยวข้อง

1.3.5 ด้านการติดตามสถานะและประวัติคำร้อง

ระบบสามารถแสดงรายการคำร้องของผู้พักอาศัยโดยเรียงตามช่วงเวลาที่กำหนด สามารถแสดงประเภทงาน สถานะ วันที่แจ้ง และวันที่หรือช่วงเวลาที่ผู้พักอาศัยสะดวกให้ดำเนินการ รวมถึงสามารถดูรายละเอียดของคำร้องแต่ละรายการได้ นอกจากนี้ ระบบมีการจัดเก็บประวัติการเปลี่ยนแปลงสถานะของคำร้องเพื่อใช้ในการตรวจสอบลำดับการดำเนินงาน

1.3.6 ด้านเทคโนโลยีที่ใช้ในการพัฒนา (Tech Stack)

ระบบพัฒนาในรูปแบบเว็บแอปพลิเคชัน โดยใช้เทคโนโลยีและเครื่องมือหลัก ดังนี้

- (1) ภาษา Java ใช้เป็นภาษาหลักในการพัฒนาส่วนประมวลผลของระบบ
- (2) Spring Boot ใช้เป็นเฟรมเวิร์กหลักในการพัฒนาเว็บแอปพลิเคชัน รวมถึงการจัดการ Controller, Service และ Repository ตามโครงสร้างของระบบ
- (3) Spring Data JPA / Hibernate ใช้สำหรับการจัดการการเชื่อมต่อและการทำงานกับฐานข้อมูลเชิงสัมพันธ์ รวมถึงการกำหนดความสัมพันธ์ระหว่าง Entity ต่าง ๆ
- (4) Thymeleaf ใช้เป็น Template Engine สำหรับสร้างหน้าเว็บและแสดงข้อมูลจากฝั่ง Server โดยเชื่อมโยงข้อมูลจาก Controller และ Entity/DTO เข้ากับหน้า HTML
- (5) HTML และ CSS ใช้สำหรับพัฒนาโครงสร้างและรูปแบบส่วนติดต่อผู้ใช้งานของระบบ
- (6) PostgreSQL ใช้เป็นระบบจัดการฐานข้อมูลสำหรับจัดเก็บข้อมูลของระบบ โดยฐานข้อมูลถูกใช้งานผ่าน Supabase
- (7) Supabase Auth ใช้สำหรับการยืนยันตัวตนของผู้ใช้งานและจัดการข้อมูลบัญชีผู้ใช้

- (8) Maven ใช้สำหรับจัดการ Dependencies และกระบวนการ Build ของโครงการ
- (9) Git และ GitHub ใช้สำหรับจัดการและควบคุมเวอร์ชันของซอร์สโค้ด รวมถึงการทำงานร่วมกันของสมาชิกภายในทีมพัฒนา
- (10) Railway ใช้สำหรับนำเว็บแอปพลิเคชันไปติดตั้งและให้บริการในสภาพแวดล้อมออนไลน์ โดยเชื่อมต่อกับฐานข้อมูล PostgreSQL ที่ใช้งานผ่าน Supabase

1.3.7 ด้านขอบเขตของการพัฒนาระบบ

การศึกษาค้นคว้าครั้งนี้มุ่งเน้นการพัฒนาระบบสำหรับกระบวนการแจ้งซ่อมภายในหอพัก ตั้งแต่การเข้าสู่ระบบ การสร้างคำร้อง การตรวจสอบและจัดการคำร้อง การมอบหมายงานให้ช่าง การดำเนินงานซ่อม และการติดตามสถานะของคำร้อง โดยเน้นการจัดการข้อมูลผ่านเว็บแอปพลิเคชันและการควบคุมกระบวนการทำงานตามกฎหมายและสถานะที่กำหนดไว้ในระบบ ทั้งนี้ ขอบเขตของการศึกษาไม่ได้ครอบคลุมการควบคุมคลังอุปกรณ์ซ่อมบำรุงจริง หรือการวิเคราะห์ปัญหาด้วยปัญญาประดิษฐ์

1.4 ประโยชน์ที่คาดว่าจะได้รับ

- 1.4.1 ได้ระบบแจ้งซ่อมภายในหอพักที่เชื่อมโยงผู้ดูแลระบบ ผู้พักอาศัย และช่างเข้าด้วยกัน
- 1.4.2 ช่วยอำนวยความสะดวกให้ผู้พักอาศัยสามารถแจ้งปัญหาและรายการซ่อมแซมภายในหอพักได้อย่างเป็นระบบ
- 1.4.3 ช่วยจัดเก็บข้อมูลและประวัติการแจ้งซ่อม ทำให้สามารถตรวจสอบข้อมูลย้อนหลังได้สะดวก

1.5 แผนการดำเนินงาน

ตารางที่ 1 แผนการดำเนินงาน

การดำเนินงาน	กันยายน				ตุลาคม	
	1	2	3	4	1	2
1. คิดหัวข้อโครงงาน						
2. ศึกษาค้นคว้าข้อมูลที่เกี่ยวข้อง และเก็บ Requirement						
3. ออกแบบ Use Case Diagram						
4. ออกแบบ Domain Model						
5. ออกแบบ Class Diagram และ Design Pattern						
6. ออกแบบฐานข้อมูล						
7. ออกแบบ Sequence Diagram และ Activity Diagram						
8. ออกแบบ UI						
9. พัฒนาโปรแกรม						
10. ทดสอบระบบ						
11. Deploy						
12. จัดทำรูปเล่มรายงาน						

บทที่ 2

ทฤษฎีและงานวิจัยที่เกี่ยวข้อง

2.1 ทฤษฎีที่เกี่ยวข้อง

การพัฒนาระบบแจ้งซ่อมภายในหอพักเป็นการประยุกต์ใช้ความรู้ทางด้านการพัฒนาเว็บแอปพลิเคชัน การเขียนโปรแกรมเชิงวัตถุ การจัดการฐานข้อมูล และการรักษาความปลอดภัยของระบบ เพื่อให้ระบบสามารถรองรับกระบวนการแจ้งซ่อม การจัดการคำร้อง การมอบหมายงาน การติดตามสถานะ และการจัดเก็บข้อมูลอย่างเป็นระบบ โดยทฤษฎีและแนวคิดที่เกี่ยวข้องกับการพัฒนาระบบมีดังต่อไปนี้

2.1.1 เว็บแอปพลิเคชัน (Web Application)

เว็บแอปพลิเคชัน คือ ซอฟต์แวร์ที่ทำงานผ่านเว็บเบราว์เซอร์ โดยผู้ใช้งานสามารถเข้าถึงระบบผ่านเครือข่ายโดยไม่จำเป็นต้องติดตั้งโปรแกรมเฉพาะบนเครื่อง ระบบแจ้งซ่อมภายในหอพักได้รับการพัฒนาในรูปแบบเว็บแอปพลิเคชัน เพื่อให้ผู้พักอาศัย เจ้าหน้าที่ และช่างสามารถเข้าถึงระบบผ่านเว็บเบราว์เซอร์สำหรับดำเนินการตามหน้าที่ของแต่ละบทบาท

ระบบประกอบด้วยส่วนติดต่อผู้ใช้สำหรับการเข้าสู่ระบบ การแจ้งซ่อม การตรวจสอบข้อมูลคำร้อง การติดตามสถานะ และการจัดการงานซ่อม โดยข้อมูลที่ผู้ใช้งานส่งผ่านหน้าเว็บจะถูกส่งไปยังส่วนประมวลผลของระบบเพื่อดำเนินการและจัดเก็บลงในฐานข้อมูล

2.1.2 รูปแบบสถาปัตยกรรม MVC (Model-View-Controller)

Model-View-Controller หรือ MVC เป็นรูปแบบการออกแบบซอฟต์แวร์ที่แบ่งระบบออกเป็น 3 ส่วน ได้แก่ Model, View และ Controller เพื่อแยกหน้าที่ของแต่ละส่วนออกจากกัน

- Model ทำหน้าที่จัดการข้อมูลและโครงสร้างข้อมูลของระบบ เช่น RepairRequest, Resident, Room และ User
- View ทำหน้าที่นำเสนอข้อมูลแก่ผู้ใช้งาน โดยระบบใช้ Thymeleaf ร่วมกับ HTML และ CSS ในการสร้างหน้าเว็บ เช่น หน้าสำหรับแจ้งซ่อมและหน้าสำหรับแสดงรายการคำร้อง
- Controller ทำหน้าที่รับ Request จากผู้ใช้งาน ประสานงานกับส่วนประมวลผล และส่งผลลัพธ์กลับไปยัง View เช่น ReporterController ที่จัดการการทำงานของผู้แจ้งซ่อม

การแบ่งระบบตามแนวคิด MVC ช่วยแยกส่วนการแสดงผลออกจากส่วนจัดการข้อมูลและการควบคุมการทำงาน ทำให้โครงสร้างของระบบมีความชัดเจนและง่ายต่อการบำรุงรักษา

2.1.3 สถาปัตยกรรมแบบแบ่งชั้น (Layered Architecture)

Layered Architecture เป็นแนวคิดในการออกแบบระบบโดยแบ่งการทำงานออกเป็นชั้นต่างๆ ซึ่งแต่ละชั้นมีหน้าที่รับผิดชอบแตกต่างกัน ระบบแจ้งซ่อมประยุกต์ใช้แนวคิดดังกล่าวโดยแบ่งการทำงานหลักออกเป็น Controller, Service และ Repository

Controller รับผิดชอบการติดต่อกับผู้ใช้งานและรับคำขอจากเว็บ จากนั้นส่งข้อมูลไปยัง Service เพื่อดำเนินการตามตรรกะทางธุรกิจ ส่วน Service รับผิดชอบกระบวนการของระบบ เช่น การสร้างคำร้อง การตรวจสอบข้อมูล และการเปลี่ยนสถานะ ขณะที่ Repository รับผิดชอบการติดต่อกับฐานข้อมูล

การแบ่งหน้าที่ดังกล่าวช่วยให้แต่ละส่วนมีความรับผิดชอบที่ชัดเจน และลดการนำตรรกะทางธุรกิจไปปะปนกับส่วนติดต่อผู้ใช้งานหรือคำสั่งในการเข้าถึงฐานข้อมูล

2.1.4 การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming)

Object-Oriented Programming หรือ OOP เป็นแนวคิดในการพัฒนาโปรแกรมโดยจัดโครงสร้างซอฟต์แวร์ให้อยู่ในรูปของคลาสและวัตถุ ซึ่งวัตถุแต่ละชนิดจะประกอบด้วยข้อมูลและพฤติกรรมที่เกี่ยวข้องกัน

ระบบแจ้งซ่อมประยุกต์ใช้ OOP ในการสร้างคลาสสำหรับองค์ประกอบต่าง ๆ ของระบบ เช่น User, Reporter, Resident, Room, RepairRequest และ RepairRequestStatus โดยแต่ละคลาสมีหน้าที่และข้อมูลที่สัมพันธ์กับองค์ประกอบของระบบจริง

แนวคิด OOP ช่วยให้สามารถจำลององค์ประกอบของระบบให้อยู่ในรูปแบบที่เข้าใจง่าย และสนับสนุนการนำแนวคิดอื่นมาใช้ในการออกแบบ เช่น Encapsulation, Abstraction และ Polymorphism

2.1.5 หลักการ SOLID

SOLID เป็นหลักการสำหรับการออกแบบซอฟต์แวร์เชิงวัตถุ โดยมีวัตถุประสงค์เพื่อให้ซอฟต์แวร์มีโครงสร้างที่สามารถดูแลรักษา แก้ไข และขยายความสามารถได้ง่าย

ในการพัฒนาระบบแจ้งซ่อม หลักการ SOLID ถูกนำมาใช้โดยเฉพาะการแยกความรับผิดชอบของแต่ละส่วน เช่น Controller รับผิดชอบการจัดการ Request และ Response, Service รับผิดชอบตรรกะทางธุรกิจ และ Repository รับผิดชอบการเข้าถึงฐานข้อมูล แนวทางดังกล่าวสอดคล้องกับหลัก Single Responsibility Principle ซึ่งแต่ละส่วนควรมีหน้าที่หลักที่ชัดเจน

นอกจากนี้ การแยกส่วนการทำงานยังช่วยลดการพึ่งพาระหว่างคลาส และทำให้สามารถปรับเปลี่ยนส่วนใดส่วนหนึ่งของระบบได้โดยส่งผลกระทบต่อส่วนอื่นน้อยลง

2.1.6 ฐานข้อมูลเชิงสัมพันธ์ (Relational Database)

ฐานข้อมูลเชิงสัมพันธ์เป็นรูปแบบการจัดเก็บข้อมูลโดยใช้ตารางเป็นโครงสร้างหลัก และสามารถกำหนดความสัมพันธ์ระหว่างตารางผ่าน Primary Key และ Foreign Key

ระบบแจ้งซ่อมใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูล โดยจัดเก็บข้อมูลที่เกี่ยวข้องกับระบบ เช่น ข้อมูลผู้ใช้งาน ข้อมูลผู้พักอาศัย ข้อมูลห้องพัก ข้อมูลคำร้องแจ้งซ่อม และประวัติการเปลี่ยนแปลงสถานะ

ตัวอย่างความสัมพันธ์ของข้อมูล ได้แก่ ผู้พักอาศัยมีความสัมพันธ์กับห้องพัก และคำร้องแจ้งซ่อมมีความสัมพันธ์กับผู้แจ้ง ที่พัก และผู้ดูแลคำร้อง การกำหนดความสัมพันธ์ระหว่างข้อมูลช่วยให้สามารถจัดเก็บข้อมูลอย่างเป็นระบบและลดความซ้ำซ้อนของข้อมูล

2.1.7 Object-Relational Mapping (ORM)

Object-Relational Mapping หรือ ORM เป็นแนวคิดที่ใช้เชื่อมโยงระหว่างวัตถุในโปรแกรมกับข้อมูลที่จัดเก็บอยู่ในฐานข้อมูลเชิงสัมพันธ์ ในระบบนี้ใช้ JPA และ Hibernate เป็นเครื่องมือสำหรับการ ORM

ตัวอย่างเช่น คลาส RepairRequest สามารถเชื่อมโยงกับตาราง repair_request และกำหนดความสัมพันธ์กับ Entity อื่น ๆ ผ่าน Annotation ของ JPA เช่น @OneToOne, @ManyToOne และ @OneToMany

การใช้ ORM ช่วยให้นักพัฒนาสามารถจัดการข้อมูลผ่าน Entity และ Repository ของระบบ แทนการเขียนคำสั่ง SQL สำหรับการดำเนินการพื้นฐานทั้งหมดด้วยตนเอง และช่วยให้โครงสร้างของโปรแกรมสอดคล้องกับโครงสร้างข้อมูลในฐานข้อมูล

2.1.8 การยืนยันตัวตนและการกำหนดสิทธิ์ (Authentication and Authorization)

Authentication คือกระบวนการตรวจสอบตัวตนของผู้ใช้งานก่อนเข้าสู่ระบบ ส่วน Authorization คือกระบวนการกำหนดว่าผู้ใช้งานที่ผ่านการยืนยันตัวตนแล้วสามารถเข้าถึงข้อมูลหรือดำเนินการใดได้บ้าง

ระบบแจ้งซ่อมแบ่งผู้ใช้งานออกเป็น 3 บทบาท ได้แก่ Reporter, Admin และ Technician โดยแต่ละบทบาทมีขอบเขตการทำงานแตกต่างกัน เช่น Reporter สามารถแจ้งซ่อมและติดตามคำร้องของตนเอง Admin สามารถจัดการคำร้องและดำเนินการที่เกี่ยวข้องกับการบริหารระบบ และ Technician สามารถดำเนินการกับงานซ่อมที่ได้รับมอบหมาย

ระบบใช้ Supabase Authentication สำหรับการยืนยันตัวตน และจัดเก็บ User ID ที่ได้รับจากระบบ Authentication ไว้ใน Session เพื่อใช้ระบุผู้ใช้งานในระหว่างการทำงานของระบบ

2.1.9 Transaction

Transaction เป็นแนวคิดในการจัดการการเปลี่ยนแปลงข้อมูลหลายรายการให้ถือเป็นกระบวนการเดียวกัน โดยการดำเนินงานภายใน Transaction จะต้องรักษาความถูกต้องและความสอดคล้องของข้อมูล หากเกิดข้อผิดพลาดในกระบวนการ ระบบสามารถยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction ได้

ในระบบแจ้งซ่อม Transaction มีความสำคัญในกระบวนการที่เกี่ยวข้องกับข้อมูลหลายส่วน เช่น การเปลี่ยนสถานะของคำร้องและการบันทึกประวัติการเปลี่ยนสถานะ เพื่อป้องกันกรณีที่ข้อมูลบางส่วนถูกบันทึกสำเร็จ ในขณะที่ข้อมูลอีกส่วนหนึ่งไม่สามารถบันทึกได้

การจัดการ Transaction ในระดับ Service ยังช่วยให้ตรรกะทางธุรกิจและการจัดการความถูกต้องของข้อมูลอยู่ในตำแหน่งที่เหมาะสม โดยไม่จำเป็นต้องให้ Controller จัดการรายละเอียดของ Transaction โดยตรง

2.1.10 Observer Pattern

Observer Pattern เป็นรูปแบบการออกแบบซอฟต์แวร์ที่ใช้เมื่อวัตถุหนึ่งมีการเปลี่ยนแปลงสถานะและต้องแจ้งให้วัตถุอื่นที่เกี่ยวข้องรับทราบ โดยวัตถุที่ติดตามจะสามารถตอบสนองต่อการเปลี่ยนแปลงดังกล่าวได้

สำหรับระบบแจ้งซ่อม แนวคิด Observer สามารถนำมาใช้กับกระบวนการเปลี่ยนแปลงสถานะของคำร้อง เช่น เมื่อ RepairRequest มีการเปลี่ยนแปลงสถานะ ระบบสามารถส่งข้อมูลไปยังส่วนที่เกี่ยวข้องกับการแจ้งเตือนผ่าน NotificationService เพื่อให้ผู้ที่เกี่ยวข้องได้รับข้อมูลการเปลี่ยนแปลง

การใช้ Observer Pattern ช่วยลดการเชื่อมโยงโดยตรงระหว่างส่วนจัดการคำร้องกับส่วนที่ตอบสนองต่อการเปลี่ยนแปลง ทำให้สามารถเพิ่มหรือแก้ไขรูปแบบการแจ้งเตือนได้โดยไม่ต้องแก้ไขตรรกะหลักของคำร้องทั้งหมด

บทที่ 3

ขั้นตอนการดำเนินโครงการ

การดำเนินโครงการแบ่งออกเป็นขั้นตอนสำคัญ ได้แก่ การศึกษาความต้องการของระบบ การวิเคราะห์และออกแบบระบบ การออกแบบฐานข้อมูล การออกแบบกระบวนการทำงาน การพัฒนาระบบ การทดสอบระบบ และการนำระบบไปใช้งานบน Cloud โดยมีรายละเอียดดังต่อไปนี้

3.1 ศึกษาความต้องการของระบบ

จากการศึกษาความต้องการ พบว่ากระบวนการแจ้งซ่อมประกอบด้วยหลายขั้นตอน ตั้งแต่ผู้พักอาศัยส่งคำร้องแจ้งซ่อม เจ้าหน้าที่ตรวจสอบคำร้อง อนุมัติหรือปฏิเสธคำร้อง การมอบหมายงานให้ช่าง การดำเนินงานซ่อม และการยืนยันผลการซ่อมจากผู้แจ้ง ดังนั้นจึงมีความจำเป็นต้องพัฒนาระบบที่สามารถจัดการกระบวนการดังกล่าวอย่างเป็นระบบ และสามารถติดตามสถานะของคำร้องได้ตลอดกระบวนการ

จากความต้องการดังกล่าว จึงกำหนดผู้ใช้งานในระบบออกเป็น 3 บทบาท ได้แก่

- (1) ผู้แจ้ง (Reporter) ซึ่งเป็นผู้พักอาศัยภายในหอพักและเป็นผู้ส่งคำร้องแจ้งซ่อม สามารถติดตาม สถานะและดูประวัติคำร้องของตนเองได้
- (2) ผู้ดูแลระบบ (Admin) ซึ่งเป็นเจ้าหน้าที่หอพักที่รับผิดชอบในการจัดการคำร้อง ได้แก่ การอนุมัติหรือปฏิเสธคำร้อง การมอบหมายงาน การตรวจงาน และการจัดการข้อมูลในระบบ
- (3) ช่าง (Technician) ซึ่งเป็นช่างประจำหอพักที่รับผิดชอบการดำเนินงานซ่อมตามงานที่ได้รับมอบหมาย และอัปเดตสถานะการซ่อม

นอกจากนี้ ระบบกำหนดความต้องการด้านการทำงานหลัก ได้แก่ ระบบจัดการบัญชี ระบบจัดการคำร้องแจ้งซ่อม ระบบมอบหมายงานช่าง และระบบจัดการบัญชีและข้อมูลผู้พักอาศัย เพื่อใช้เป็นขอบเขตในการวิเคราะห์และออกแบบระบบในขั้นตอนถัดไป

3.2 วิเคราะห์และออกแบบกระบวนการทำงานของระบบ

นำข้อมูลที่ได้มาวิเคราะห์และออกแบบกระบวนการทำงานของระบบ โดยใช้แผนภาพ UML เพื่ออธิบายความต้องการ โครงสร้าง และลำดับการทำงานของระบบให้มีความชัดเจนก่อนเริ่มพัฒนาโปรแกรม

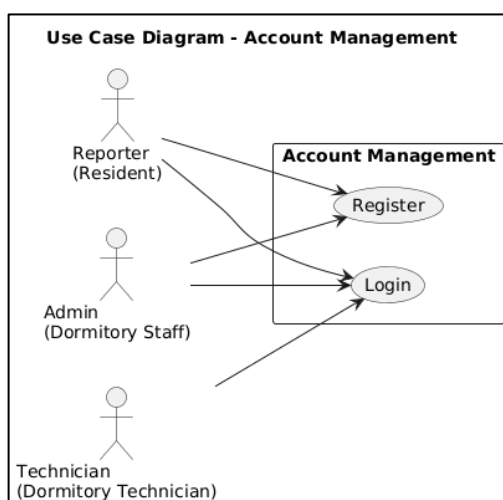
การออกแบบประกอบด้วย Use Case Diagram, Domain Model, Class Diagram, Sequence Diagram, Activity Diagram, State Diagram, Component Diagram และ Deployment Diagram โดยแต่ละแผนภาพมีหน้าที่แตกต่างกัน ดังนี้

3.2.1. Use Case Diagram

Use Case Diagram ใช้สำหรับแสดงฟังก์ชันหลักของระบบและความสัมพันธ์ระหว่างผู้ใช้งานกับระบบ โดยระบบกำหนดผู้ใช้งานออกเป็น 3 บทบาท ได้แก่ ผู้แจ้ง (Reporter), ผู้ดูแลระบบ (Admin) และ ช่าง (Technician)

ระบบแจ้งซ่อมภายในหอพักประกอบด้วยระบบย่อยทั้งหมด 4 ระบบ ดังนี้

(1) ระบบจัดการบัญชี



ภาพที่ 1 Use Case Diagram ส่วนจัดการบัญชี

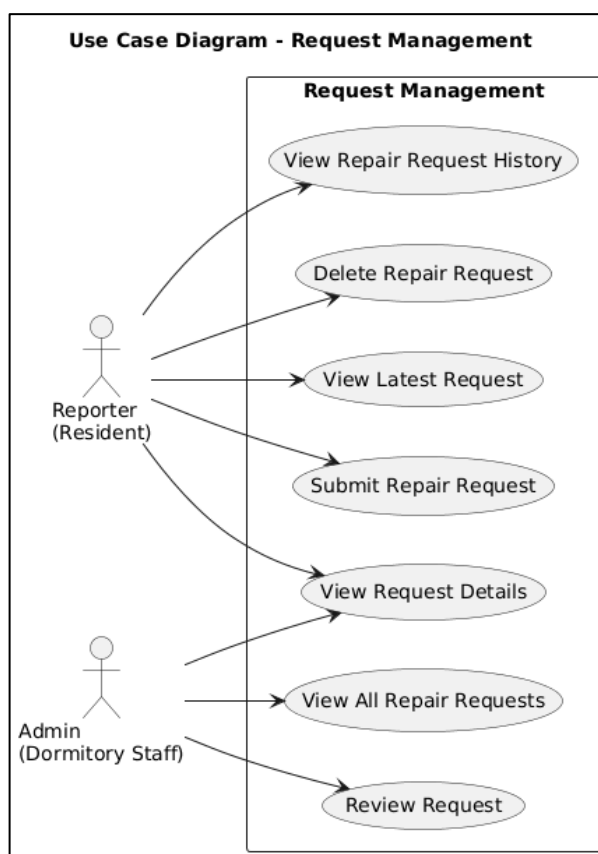
Actor

- Reporter - ผู้แจ้งซึ่งเป็นผู้พักอาศัยที่ใช้ระบบสำหรับสมัครบัญชีและเข้าใช้งานระบบ
- Admin - ผู้ดูแลระบบซึ่งเป็นเจ้าหน้าที่หอพักที่สามารถสร้างบัญชีผู้ใช้สำหรับช่างและเข้าใช้งานระบบ
- Technician - ช่างประจำหอพักที่ใช้บัญชีสำหรับเข้าใช้งานระบบ

Use Cases

- Register : ผู้แจ้งสามารถสมัครบัญชีผู้ใช้ใหม่เพื่อใช้งานระบบแจ้งซ่อมได้ ในขณะที่ผู้ดูแลระบบสามารถสร้างบัญชีผู้ใช้ใหม่สำหรับช่างประจำหอพักได้
- Login : ผู้ใช้งานทั้ง 3 บทบาท ได้แก่ ผู้แจ้ง ผู้ดูแลระบบ และช่าง สามารถลงชื่อเข้าใช้ระบบแจ้งซ่อมด้วยบัญชีของตนเอง เพื่อเข้าถึงฟังก์ชันต่าง ๆ ตามสิทธิ์ของแต่ละบทบาท

(2) ระบบจัดการคำร้องแจ้งซ่อม



ภาพที่ 2 Use Case Diagram ส่วนจัดการคำร้องแจ้งซ่อม

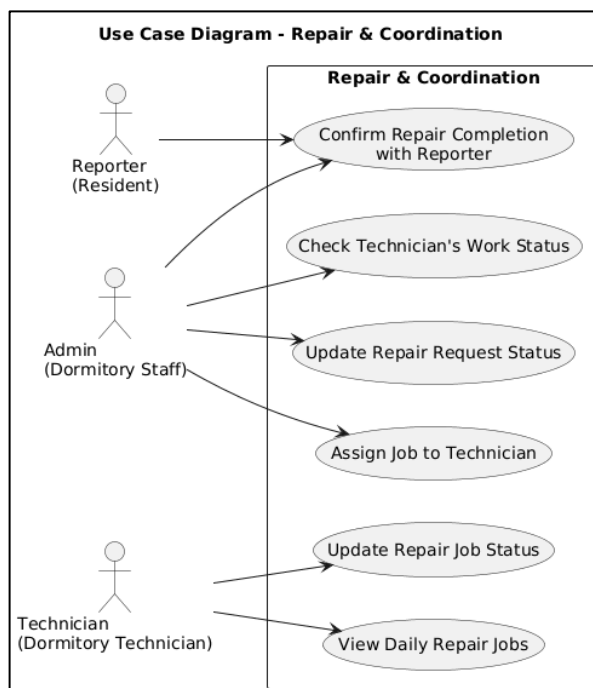
Actor

- Reporter - ผู้แจ้งซึ่งเป็นผู้พักอาศัยที่สามารถสร้าง ดู และจัดการคำร้องแจ้งซ่อมของตนเอง
- Admin - ผู้ดูแลระบบซึ่งเป็นเจ้าหน้าที่หอพักที่สามารถตรวจสอบและจัดการคำร้องแจ้งซ่อมทั้งหมดภายในหอพัก

Use Cases

- View Repair Request History : ผู้แจ้งสามารถดูประวัติคำร้องแจ้งซ่อมทั้งหมดของตนเองได้
- Submit Repair Request : ผู้แจ้งสามารถส่งคำร้องแจ้งซ่อม โดยระบุรายละเอียดของปัญหาและข้อมูลที่เกี่ยวข้องกับการแจ้งซ่อมได้
- Delete Repair Request : ผู้แจ้งสามารถลบคำร้องแจ้งซ่อมของตนเองได้ เฉพาะกรณีที่คำร้องยังไม่ได้รับการอนุมัติจากผู้ดูแลระบบ
- View Latest Request : ผู้แจ้งสามารถดูคำร้องแจ้งซ่อมล่าสุดของตนเองได้
- View Request Details : ผู้แจ้งและผู้ดูแลระบบสามารถดูรายละเอียดของคำร้องแจ้งซ่อมได้
- View All Repair Requests : ผู้ดูแลระบบสามารถดูรายการคำร้องแจ้งซ่อมทั้งหมดที่มีอยู่ในระบบได้
- Review Request : ผู้ดูแลระบบสามารถตรวจสอบและจัดการสถานะของคำร้องแจ้งซ่อมได้ โดยสถานะของคำร้องประกอบด้วย
 - PENDING - รอดำเนินการ
 - APPROVED - อนุมัติ
 - REJECTED - ปฏิเสธ
 - IN_PROGRESS - กำลังดำเนินงาน
 - COMPLETED - ดำเนินการสำเร็จ
 - IN_COMPLETED - ดำเนินการไม่สำเร็จ

(3) ระบบมอบหมายงานช่าง



ภาพที่ 3 Use Case Diagram ส่วนมอบหมายงานช่าง

Actor

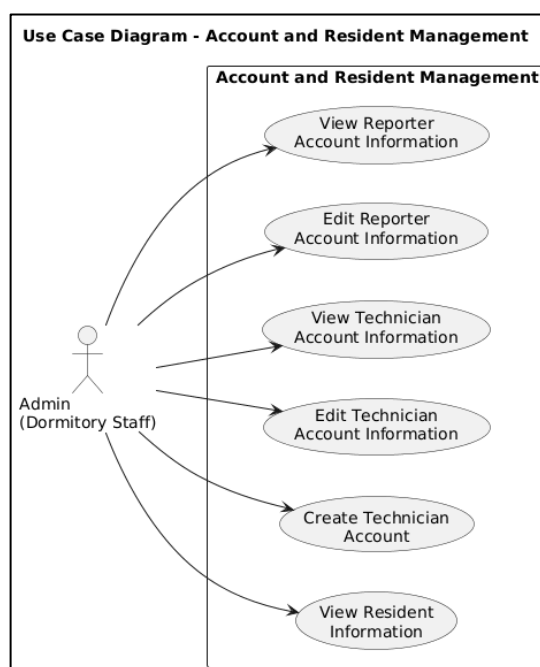
- Admin - ผู้ดูแลระบบซึ่งเป็นเจ้าหน้าที่หอพักที่รับผิดชอบการมอบหมายงาน ตรวจสอบสถานะ และติดตามผลการดำเนินงานซ่อม
- Technician - ช่างประจำหอพักที่รับผิดชอบการดำเนินงานซ่อมและอัปเดตสถานะของงานที่ได้รับมอบหมาย
- Reporter - ผู้แจ้งซึ่งเป็นผู้พักอาศัย ทำหน้าที่ยืนยันผลการซ่อมแก่ผู้ดูแลระบบ

Use Cases

- Assign Job to Technician : ผู้ดูแลระบบสามารถมอบหมายงานซ่อมให้แก่ช่างที่รับผิดชอบ โดยพิจารณาจากรายละเอียดของงานและช่างที่เหมาะสมกับงาน
- View Daily Repair Jobs : ช่างสามารถดูรายการงานซ่อมที่ได้รับมอบหมายและต้องดำเนินการในแต่ละวัน เพื่อใช้ในการวางแผนและดำเนินงานซ่อม
- Update Repair Job Status : ช่างสามารถอัปเดตสถานะของงานซ่อมที่ได้รับมอบหมายตามผลการดำเนินงาน โดยสถานะประกอบด้วย
 - COMPLETED - ดำเนินการสำเร็จ
 - IN_COMPLETED - ดำเนินการไม่สำเร็จ

- Check Technician's Work Status : ผู้ดูแลระบบสามารถตรวจสอบสถานะการดำเนินงานของช่างแต่ละราย เพื่อใช้ติดตามความคืบหน้าและตรวจสอบว่างานที่ได้รับมอบหมายอยู่ในขั้นตอนใด
- Confirm Repair Completion with Reporter : ผู้ดูแลระบบสามารถติดต่อผู้แจ้งเพื่อยืนยันผลการดำเนินงานซ่อมว่าได้ดำเนินการเสร็จสิ้นและตรงตามความต้องการของผู้แจ้งหรือไม่
- Update Repair Request Status : ผู้ดูแลระบบสามารถอัปเดตสถานะของคำร้องแจ้งซ่อมตามผลการดำเนินงานของช่างและผลการยืนยันจากผู้แจ้ง เพื่อให้สถานะของคำร้องสอดคล้องกับการดำเนินงานจริง

(4) ระบบจัดการบัญชีและผู้พักอาศัย



ภาพที่ 4 Use Case Diagram ส่วนจัดการบัญชีและผู้พักอาศัย

Actor

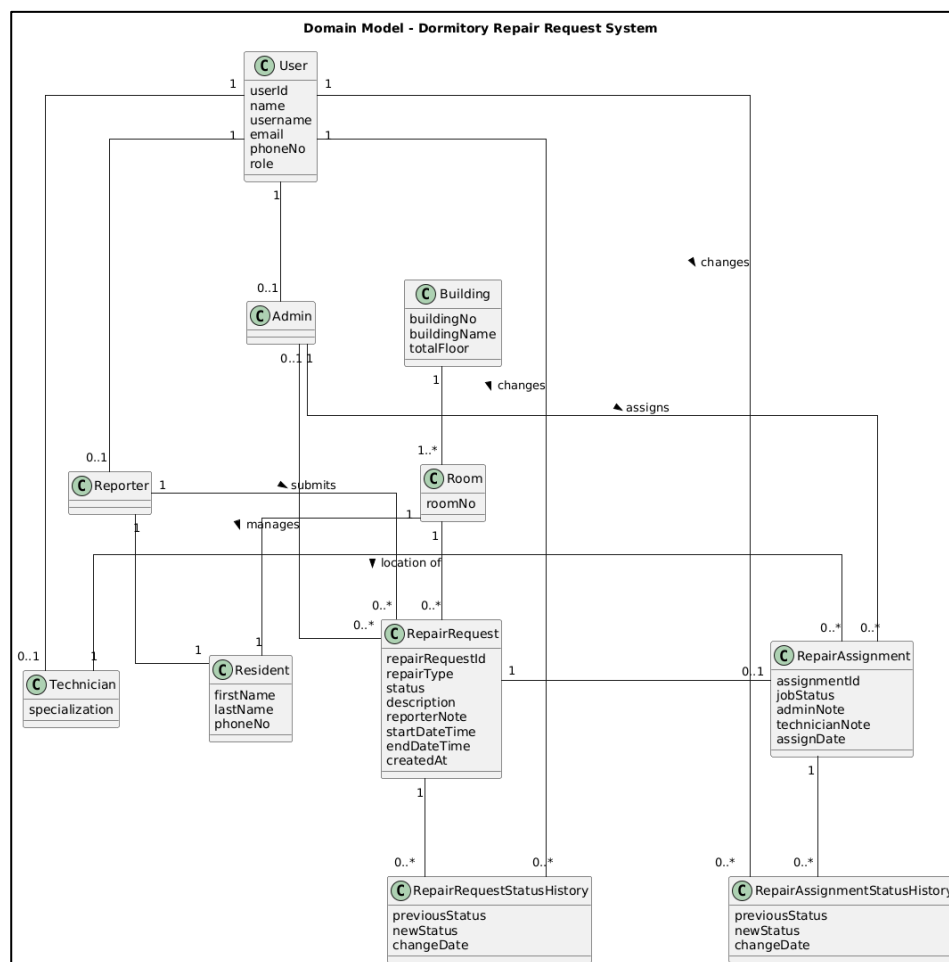
- Admin - ผู้ดูแลระบบซึ่งเป็นเจ้าหน้าที่หอพักที่รับผิดชอบการจัดการข้อมูลบัญชีผู้ใช้งานและข้อมูลผู้พักอาศัยภายในหอพัก

Use Cases

- View Reporter Account Information : ผู้ดูแลระบบสามารถดูข้อมูลบัญชีของผู้แจ้งซ่อมเพื่อใช้ในการตรวจสอบและจัดการข้อมูลผู้ใช้งานในระบบ
- Edit Reporter Account Information : ผู้ดูแลระบบสามารถแก้ไขข้อมูลบัญชีของผู้แจ้งซ่อม เพื่อให้ข้อมูลบัญชีในระบบมีความถูกต้องและเป็นปัจจุบัน
- Create Technician Account : ผู้ดูแลระบบสามารถสร้างบัญชีผู้ใช้ใหม่สำหรับช่างเพื่อให้ช่างสามารถเข้าสู่ระบบและใช้งานฟังก์ชันตามสิทธิ์ของตนเองได้
- View Technician Account Information : ผู้ดูแลระบบสามารถดูข้อมูลบัญชีของช่าง เพื่อใช้ในการตรวจสอบข้อมูลและจัดการบัญชีผู้ใช้งานของช่าง
- Edit Technician Account Information : ผู้ดูแลระบบสามารถแก้ไขข้อมูลบัญชีของช่าง เพื่อให้ข้อมูลบัญชีของช่างมีความถูกต้องและเป็นปัจจุบัน
- View Resident Information : ผู้ดูแลระบบสามารถดูข้อมูลผู้พักอาศัยในหอพักเพื่อใช้ในการตรวจสอบข้อมูลผู้พักอาศัยและสนับสนุนการจัดการข้อมูลที่เกี่ยวข้องกับการแจ้งซ่อม

3.2.2. Domain Model

Domain Model ใช้สำหรับแสดงแนวคิดหลักของระบบและความสัมพันธ์ระหว่างแนวคิดเหล่านั้น โดยแบ่งแนวคิดหลักออกเป็น 5 ส่วน ได้แก่ การจัดการผู้ใช้งาน โครงสร้างหอพัก การแจ้งซ่อม การมอบหมายงานซ่อม และการติดตามประวัติการเปลี่ยนแปลงสถานะ



ภาพที่ 5 Domain Model ของระบบแจ้งซ่อมในหอพัก

(1) การจัดการผู้ใช้งาน

- User แทนบัญชีผู้ใช้งานของระบบ โดยผู้ใช้งานสามารถมีบทบาทเป็น ผู้แจ้ง (Reporter), ผู้ดูแลระบบ (Admin) หรือ ช่าง (Technician)
- Admin หมายถึงเจ้าหน้าที่หอพักที่มีหน้าที่จัดการคำร้องแจ้งซ่อมและมอบหมายงานให้ช่าง Reporter หมายถึงผู้แจ้งซ่อม ซึ่งมีความสัมพันธ์กับ Resident เพื่อเชื่อมโยงบัญชีผู้ใช้งานกับข้อมูลผู้พักอาศัย

- Technician หมายถึงช่างซ่อมที่รับผิดชอบงานซ่อมที่ได้รับมอบหมาย โดยมีข้อมูลด้านความเชี่ยวชาญของช่าง

ความสัมพันธ์ระหว่าง User กับ Admin, Reporter และ Technician เป็นความสัมพันธ์แบบหนึ่งต่อหนึ่ง กล่าวคือ บัญชีผู้ใช้งานหนึ่งบัญชีมีข้อมูลของบทบาทที่เกี่ยวข้องได้หนึ่งรายการ ในขณะที่ข้อมูลของบทบาทแต่ละประเภทต้องอ้างอิงกลับมายังบัญชีผู้ใช้งาน

(2) โครงสร้างของหอพัก

- Building แทนอาคารภายในหอพัก โดยอาคารหนึ่งสามารถประกอบด้วยห้องพักได้หลายห้อง ดังนั้นจึงมีความสัมพันธ์แบบหนึ่งต่อหลายกับ Room
- Room แทนห้องพักภายในอาคาร โดยห้องหนึ่งสามารถมีผู้อยู่อาศัย (Resident) ได้หนึ่งคน และผู้อยู่อาศัยแต่ละคนจะมีห้องพักที่เกี่ยวข้องหนึ่งห้อง นอกจากนี้ Room ยังมีความสัมพันธ์กับ RepairRequest เนื่องจากคำร้องแจ้งซ่อมต้องระบุสถานที่หรือห้องพักที่เกิดปัญหา

(3) การแจ้งซ่อม

- RepairRequest แทนของคำร้องแจ้งซ่อมที่ผู้แจ้งซึ่งเป็นผู้พักอาศัยส่งเข้าสู่ระบบ โดยคำร้องประกอบด้วยข้อมูลสำคัญ เช่น ประเภทงานซ่อม รายละเอียดปัญหาสถานะคำร้อง หมายเหตุของผู้แจ้ง วันที่และเวลาที่สะดวก และวันที่สร้างคำร้อง
- Reporter สามารถสร้างคำร้องแจ้งซ่อมได้หลายรายการ ขณะที่คำร้องแต่ละรายการมีผู้แจ้งซ่อมหนึ่งราย นอกจากนี้คำร้องแต่ละรายการยังเกี่ยวข้องกับ Room ที่เกิดปัญหาเพื่อให้ระบบสามารถระบุสถานที่ของงานซ่อมได้
- Admin มีความสัมพันธ์กับ RepairRequest ในฐานะผู้ดูแลระบบที่เกี่ยวข้องกับคำร้อง เช่น การตรวจสอบและจัดการคำร้องที่ส่งเข้ามา

(4) การมอบหมายงานซ่อม

- RepairAssignment แทนงานซ่อมที่ถูกมอบหมายให้ช่างดำเนินการ โดยมีข้อมูลเกี่ยวกับสถานะของงาน หมายเหตุจากผู้ดูแลหรือช่าง และวันที่มอบหมายงาน
- RepairRequest ทนของคำร้องแจ้งซ่อมที่ผู้แจ้งซึ่งเป็นผู้พักอาศัยส่งเข้าสู่ระบบ RepairRequest หนึ่งรายการสามารถยังไม่มี การมอบหมายงาน หรือมี RepairAssignment หนึ่งรายการ เมื่อมีการมอบหมายงาน ระบบจะเชื่อมโยงงานดังกล่าวกับ Technician ซึ่งเป็นช่างผู้รับผิดชอบดำเนินงาน และ Admin ซึ่งเป็นผู้ทำการมอบหมายงาน

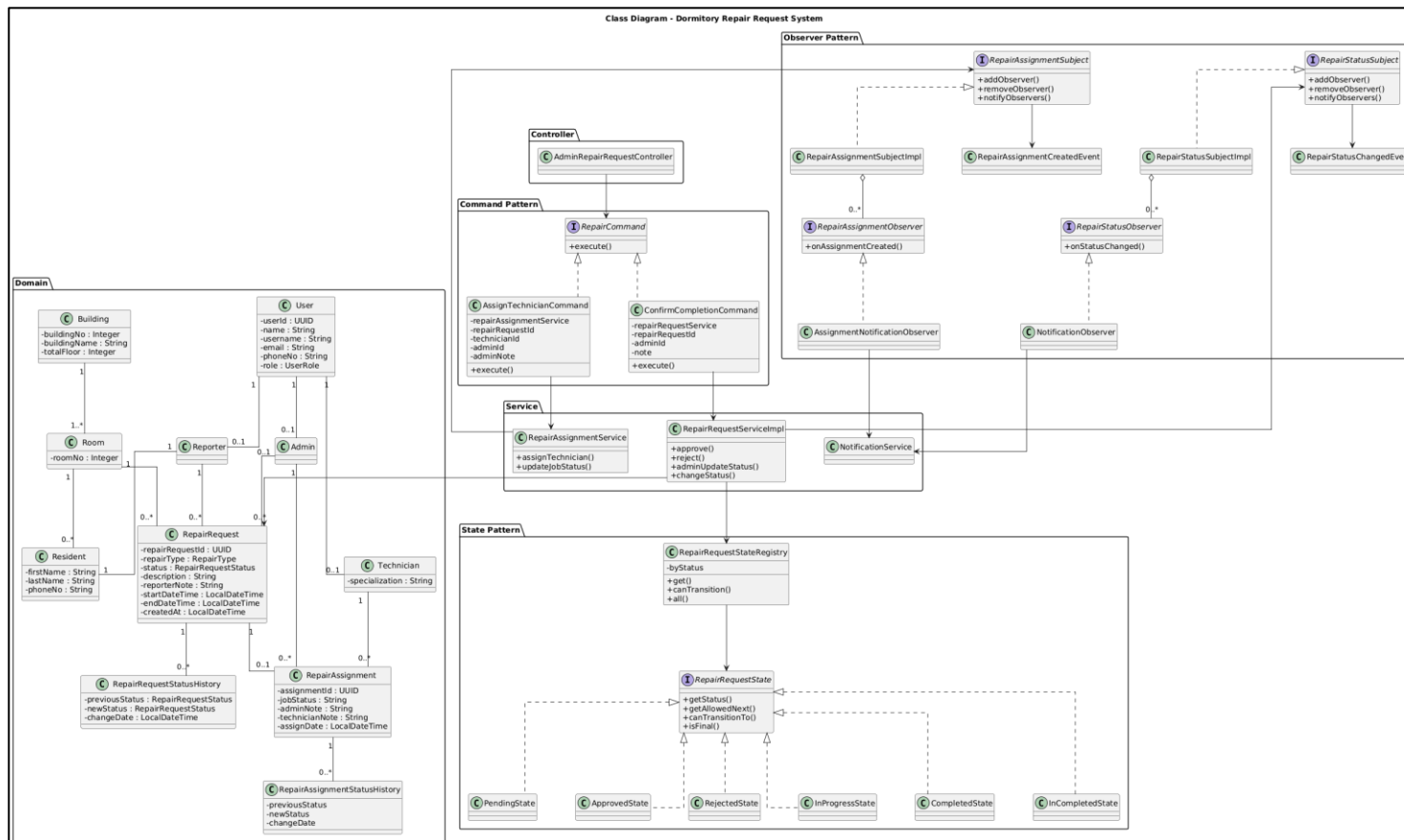
โดย RepairRequest แสดงถึงปัญหาที่ผู้พักอาศัยแจ้งเข้ามา ขณะที่ RepairAssignment แสดงถึงงานที่ถูกส่งต่อให้ช่างดำเนินการ

(5) การเก็บประวัติการเปลี่ยนแปลงสถานะ

- RepairRequestStatusHistory ใช้เก็บประวัติการเปลี่ยนแปลงสถานะของคำร้องแจ้งซ่อม โดยคำร้องหนึ่งรายการสามารถมีประวัติการเปลี่ยนสถานะได้หลายรายการ และแต่ละรายการจะระบุสถานะเดิม สถานะใหม่ และวันที่เปลี่ยนแปลง รวมถึง User ที่เป็นผู้ดำเนินการเปลี่ยนสถานะ
- RepairAssignmentStatusHistory ใช้เก็บประวัติการเปลี่ยนแปลงสถานะของงานที่มอบหมายให้ช่าง โดยมีแนวคิดเช่นเดียวกัน แต่แยกออกจากประวัติของ RepairRequest เพื่อให้สามารถติดตามสถานะของคำร้องและสถานะของงานซ่อมได้อย่างเป็นอิสระ

การแยกประวัติทั้งสองประเภทช่วยให้ระบบสามารถตรวจสอบลำดับการดำเนินงานย้อนหลังได้อย่างชัดเจน และสะท้อนกระบวนการทำงานจริงของระบบ

3.2.3. Class Diagram และ Design Pattern



ภาพที่ 6 Class Diagram ของระบบ

ระบบแจ้งซ่อมภายในหอพักพัฒนาด้วย Spring Boot โดยออกแบบตามหลักการของ Design Pattern ซึ่งแบ่งออกเป็น 2 กลุ่ม ได้แก่ (1) Enterprise / Architectural Patterns ซึ่งกำหนดโครงสร้างโดยรวมและการแบ่งความรับผิดชอบของระบบ และ (2) GoF Patterns ซึ่งใช้จัดการพฤติกรรมของออบเจกต์ภายในระบบ

(1) Enterprise / Architectural Patterns

(1.1) Layered Architecture

ระบบจัดโครงสร้างโปรเจกต์แบบ Layered Architecture โดยแบ่งออกเป็นชั้นตามความรับผิดชอบ และกำหนดให้แต่ละชั้นเรียกใช้งานเฉพาะชั้นที่อยู่ถัดลงไปเท่านั้น ประกอบด้วย 4 ชั้น ดังนี้

- Presentation Layer ได้แก่ Controller และ Template ของหน้าจอกำหนดหน้าที่รับคำขอจากผู้ใช้งานและแสดงผล
- Business Layer ได้แก่ Service ทำหน้าที่ประมวลผล Business Logic เช่น การตรวจสอบสิทธิ์ การตรวจสอบเงื่อนไขการเปลี่ยนสถานะ และการบันทึกประวัติสถานะ
- Data Access Layer ได้แก่ Repository ทำหน้าที่ติดต่อกับฐานข้อมูล
- Database ได้แก่ ฐานข้อมูลของระบบ (Supabase) ซึ่งเก็บข้อมูลตามโครงสร้างของ Entity

นอกจากนี้ Entity และ Enum (package domain) ถูกใช้ร่วมกันในทุกชั้น และมี package สนับสนุน ได้แก่ config, dto, exception, event และ util

ตัวอย่างเช่น เมื่อผู้แจ้งส่งคำร้องแจ้งซ่อม ReporterController จะรับคำขอและส่งต่อให้ RepairRequestService ประมวลผล จากนั้น Service เรียกใช้ RepairRequestRepository เพื่อบันทึกข้อมูลลงฐานข้อมูล การแบ่งชั้นดังกล่าวช่วยให้แต่ละส่วนมีหน้าที่ชัดเจน สามารถแก้ไขหรือทดสอบแต่ละชั้นได้อย่างอิสระ และสอดคล้องกับหลักการ Separation of Concerns

(1.2) MVC (Model-View-Controller)

ในชั้น Presentation ระบบใช้รูปแบบ MVC เพื่อแยกข้อมูล การแสดงผล และการควบคุมการทำงานออกจากกัน ดังนี้

- Model ได้แก่ Entity, DTO และ Form ที่ถูกส่งผ่านออบเจกต์ Model ไปยังหน้าจอ

- View ได้แก่ไฟล์ Thymeleaf Template ซึ่งแยกเป็นหมวดตามบทบาท คือ admin, reporter และ technician
- Controller ได้แก่คลาสที่กำหนดด้วย @Controller ใน package controller.web เช่น AdminRepairRequestController ทำหน้าที่รับ HTTP Request เรียกใช้ Service นำผลลัพธ์ใส่ใน Model และส่งชื่อ View กลับไปแสดงผล เช่น admin/assignments-list

ระบบมี Controller อีกกลุ่มหนึ่งที่กำหนดด้วย @RestController ใน package controller.api เช่น RepairRequestRestController ซึ่งให้บริการข้อมูลในรูปแบบ JSON ผ่านเส้นทาง /api/v1 และ /api/admin โดยการเข้าถึงทุกเส้นทางถูกควบคุมสิทธิ์ตามบทบาทด้วย Spring Security ที่ SecurityConfig

(1.3) Repository Pattern

ระบบใช้ Repository Pattern เพื่อแยกการเข้าถึงข้อมูลออกจาก Business Logic โดยกำหนด Repository เป็น Interface ที่สืบทอดจาก JpaRepository ของ Spring Data JPA จำนวนหนึ่ง Interface ต่อหนึ่ง Entity ซึ่ง Spring Data JPA จะสร้างคลาสที่ทำงานจริงให้โดยอัตโนมัติ

- Repository ของระบบกำหนดเมธอดค้นหาตามชื่อเมธอด (Derived Query) เช่น
- findByReporter_ReporterIdOrderByCreatedAtDesc สำหรับค้นหาคำร้องทั้งหมดของผู้แจ้งโดยเรียงจากล่าสุด และ
- existsByRepairRequest_RepairRequestId สำหรับตรวจสอบว่าคำร้องถูกมอบหมายงานแล้วหรือไม่ ทำให้ Service เรียกใช้ข้อมูลได้โดยไม่ต้องทราบรายละเอียดของฐานข้อมูล

(1.4) Service Layer Pattern

ระบบรวม Business Logic ไว้ในชั้น Service ซึ่งกำหนดสัญญาการทำงานเป็น Interface และมีคลาส Implementation ใน package service.impl กำกับด้วย @Service และ @Transactional เพื่อให้การเปลี่ยนแปลงข้อมูลที่เกี่ยวข้องกันสำเร็จหรือยกเลิกพร้อมกัน

ตัวอย่างเช่น RepairRequestServiceImpl รับผิดชอบการตรวจสอบเงื่อนไขการเปลี่ยนสถานะผ่าน RepairRequestStateRegistry การบันทึก RepairRequestStatusHistory และการแจ้งเหตุการณ์ไปยัง RepairStatusSubject

โดย Controller ไม่มี Business Logic และทำหน้าที่เพียงส่งต่อคำขอให้ Service การออกแบบนี้ทำให้ Logic เดียวกันถูกนำไปใช้ซ้ำได้ทั้งจากหน้าเว็บและ REST API

(1.5) DTO Pattern และ Mapping

ระบบใช้ DTO (Data Transfer Object) เพื่อแยก Entity ออกจากข้อมูลที่ได้รับและส่งผ่าน API โดยแบ่งเป็น 2 กลุ่ม ได้แก่

- DTO ขาเข้า (package dto.request) เช่น CreateRepairRequestDto, RegisterRequest และ AdminTechnicianCreateRequest ใช้รับข้อมูลจากผู้ใช้งาน
- DTO ขาออก (package dto.response) เช่น RepairRequestResponse, AdminTechnicianResponse และ DailyRepairJobDto ใช้ส่งข้อมูลกลับไปยังผู้ใช้งาน โดยเลือกเฉพาะข้อมูลที่จำเป็น

การแปลงข้อมูลระหว่าง Entity และ DTO (Mapping) ทำโดยใช้ Static Factory Method ภายใน DTO เช่น RepairRequestResponse.from(RepairRequest) และเมธอด toResponse() ภายใน Service เช่น AdminTechnicianServiceImpl และ AdminResidentServiceImpl การแยก DTO ช่วยป้องกันการเปิดเผยโครงสร้างภายในของ Entity และลดผลกระทบต่อ API เมื่อโครงสร้างฐานข้อมูลเปลี่ยนแปลง

(1.6) Dependency Injection

ระบบใช้ Dependency Injection ของ Spring Framework โดยให้ Spring IoC Container เป็นผู้สร้างออบเจกต์และส่งต่อ Dependency ที่จำเป็นให้แต่ละคลาส ระบบใช้ Constructor Injection โดยประกาศ Dependency เป็นฟิลด์แบบ final และรับค่าผ่าน Constructor ตัวอย่างเช่น RepairRequestServiceImpl รับ Repository, RepairRequestStateRegistry และ RepairStatusSubject ผ่าน Constructor ทั้งหมด และคลาสต่าง ๆ อ้างอิงถึง Service ผ่าน Interface ไม่ได้อ้างอิง Implementation โดยตรง

InCompletedState โดยแต่ละ Concrete State จะกำหนดสถานะปัจจุบันและสถานะถัดไปที่สามารถเปลี่ยนได้

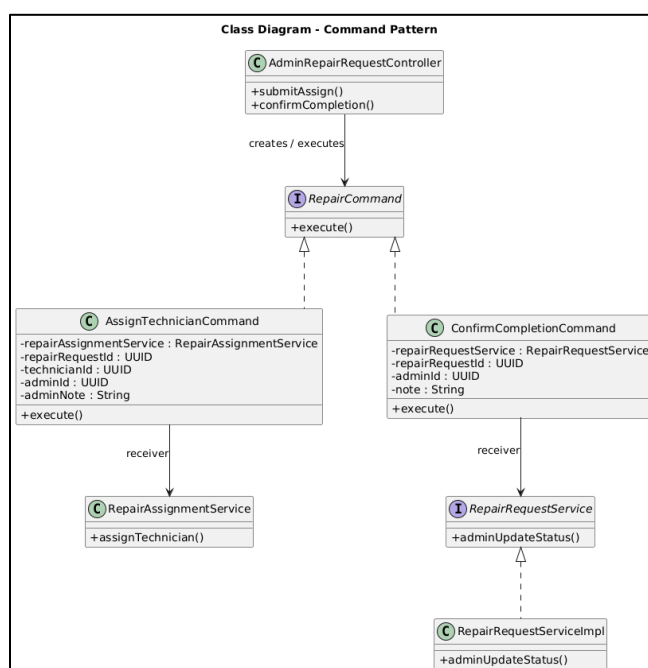
- RepairRequestStateRegistry ทำหน้าที่จัดเก็บและค้นหา State ที่สอดคล้องกับสถานะของคำร้อง ทำให้ RepairRequestServiceImpl สามารถตรวจสอบว่าการเปลี่ยนสถานะจากสถานะหนึ่งไปยังอีกสถานะหนึ่งสามารถทำได้หรือไม่ ก่อนที่จะดำเนินการเปลี่ยนสถานะจริง

ตัวอย่างเช่น เมื่อคำร้องอยู่ในสถานะ PENDING ระบบสามารถเปลี่ยนเป็น APPROVED หรือ REJECTED ได้ แต่ไม่สามารถเปลี่ยนไปยัง COMPLETED ได้โดยตรง การตรวจสอบดังกล่าวจึงเป็นความรับผิดชอบของ State Pattern

การออกแบบนี้ช่วยลดการใช้เงื่อนไข if-else หรือ switch จำนวนมากใน Service และทำให้สามารถเพิ่มหรือปรับกฎของสถานะแต่ละประเภทได้ง่ายขึ้น โดยไม่กระทบกับส่วนอื่นของระบบ

(2.2) Command Pattern

ระบบใช้ Command Pattern เพื่อแยกคำสั่งที่ผู้ใช้งานต้องการดำเนินการออกจากส่วนที่รับผิดชอบการประมวลผลคำสั่ง โดยคำสั่งแต่ละประเภทจะถูกแทนด้วย Object ของ Command ทำให้สามารถจัดการคำสั่งแต่ละประเภทได้อย่างเป็นอิสระ

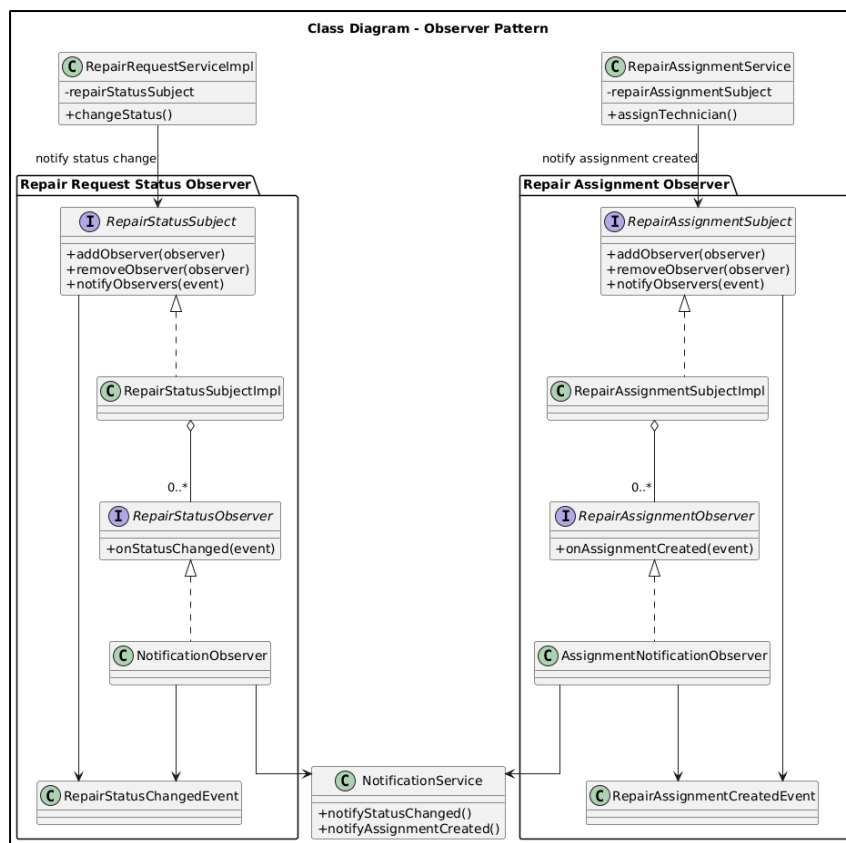


ภาพที่ 8 Class Diagram ส่วน Command Pattern

- RepairCommand เป็น Command Interface ซึ่งกำหนดการทำงานพื้นฐานของคำสั่งผ่าน execute()
- AssignTechnicianCommand และ ConfirmCompletionCommand เป็น Concrete Command โดย AssignTechnicianCommand ใช้แทนคำสั่งที่ผู้ดูแลระบบมอบหมายงานให้ช่างและ ConfirmCompletionCommand ใช้แทนคำสั่งที่ผู้ดูแลระบบยืนยันผลการดำเนินงานของคำร้อง
- RepairAssignmentService และ RepairRequestServiceImpl ทำหน้าที่เป็น Receiver ซึ่งเป็นส่วนที่มี Logic สำหรับดำเนินการจริง โดย Concrete Command จะเรียกใช้ Service ที่เกี่ยวข้องเมื่อมีการเรียก execute()
- AdminRepairRequestController ทำหน้าที่เป็น Invoker เนื่องจากเป็นผู้สร้าง Command และเรียก execute() โดย Controller ไม่จำเป็นต้องทราบรายละเอียดของกระบวนการทำงานภายในคำสั่ง
ตัวอย่างเช่น เมื่อผู้ดูแลระบบมอบหมายงานให้ช่าง Controller จะสร้าง AssignTechnicianCommand และเรียก execute() จากนั้น Command จะส่งต่อการทำงานไปยัง RepairAssignmentService การออกแบบดังกล่าวช่วยแยกคำสั่งออกจากกระบวนการประมวลผล และทำให้สามารถเพิ่มคำสั่งใหม่ได้โดยไม่ต้องแก้ไขโครงสร้างของคำสั่งเดิม

(2.3) Observer Pattern

ระบบใช้ Observer Pattern เพื่อจัดการการแจ้งเตือนเมื่อเกิดเหตุการณ์สำคัญในกระบวนการแจ้งซ่อม โดยแยกผู้ที่สร้างเหตุการณ์ออกจากผู้ที่ต้องการรับรู้เหตุการณ์ ทำให้ส่วนของการเปลี่ยนสถานะหรือการมอบหมายงานไม่จำเป็นต้องผูกกับรายละเอียดของการแจ้งเตือนโดยตรง



ภาพที่ 9 Class Diagram ส่วน Observer Pattern

ระบบมีการใช้งาน Observer Pattern ใน 2 เหตุการณ์หลัก ได้แก่ การเปลี่ยนสถานะของคำร้อง และการสร้างงานมอบหมายให้ช่าง

1. การเปลี่ยนสถานะของคำร้อง

- RepairStatusSubject เป็น Subject Interface ซึ่งกำหนดการเพิ่ม การลบ และการแจ้งเหตุการณ์ไปยัง Observer ส่วน RepairStatusSubjectImpl เป็น Concrete Subject ที่จัดเก็บรายการ Observer
- RepairStatusObserver เป็น Observer Interface และ NotificationObserver เป็น Concrete Observer ที่รับรู้เหตุการณ์เมื่อสถานะของคำร้องเปลี่ยนแปลง โดยได้รับข้อมูลผ่าน RepairStatusChangeEvent และส่งต่อไปให้ NotificationService ดำเนินการสร้างหรือจัดการการแจ้งเตือน

เมื่อ RepairRequestServiceImpl เปลี่ยนสถานะของคำร้อง ระบบจะสร้าง RepairStatusChangedEvent และแจ้งไปยัง RepairStatusSubject จากนั้น Subject จะส่งเหตุการณ์ดังกล่าวไปยัง Observer ที่ลงทะเบียนไว้

2. การสร้างงานมอบหมาย

สำหรับการมอบหมายงานให้ช่าง ระบบใช้ RepairAssignmentSubject และ RepairAssignmentObserver แยกจาก Observer ของการเปลี่ยนสถานะคำร้อง เมื่อ RepairAssignmentService สร้าง RepairAssignment จะสร้าง RepairAssignmentCreatedEvent และส่งไปยัง RepairAssignmentSubject จากนั้น RepairAssignmentSubjectImpl จะแจ้งเหตุการณ์ไปยัง AssignmentNotificationObserver ซึ่งส่งต่อให้ NotificationService ดำเนินการแจ้งเตือนที่เกี่ยวข้อง

การออกแบบด้วย Observer Pattern ทำให้ Service ไม่จำเป็นต้องรู้รายละเอียดว่ามีผู้รับเหตุการณ์กี่ตัวหรือแต่ละ Observer ดำเนินการอย่างไร และช่วยให้สามารถเพิ่ม Observer ใหม่ในอนาคตได้ เช่น Observer สำหรับบันทึก Log, ส่ง Email หรือแจ้งเตือนผ่านช่องทางอื่น โดยไม่ต้องแก้ไข Logic หลักของการเปลี่ยนสถานะหรือการมอบหมายงาน

3.2.4. ER Diagram

ER Diagram ใช้สำหรับออกแบบโครงสร้างฐานข้อมูล โดยแสดง Entity, Attribute, Primary Key, Foreign Key และความสัมพันธ์ระหว่างตาราง เพื่อให้ฐานข้อมูลรองรับกระบวนการทำงานของระบบอย่างถูกต้อง

(1) คำอธิบายเอนทิตี

ตารางที่ 2 คำอธิบายเอนทิตี

เอนทิตี	คำอธิบาย
User (บัญชีผู้ใช้)	เก็บบัญชีของผู้ใช้ทุกบทบาท ได้แก่ ชื่อ นามสกุล เบอร์โทรศัพท์ ชื่อผู้ใช้ อีเมล รหัสผ่าน และบทบาท (ADMIN, REPORTER, TECHNICIAN) ใช้เป็นศูนย์กลางของตัวตนผู้ใช้
Reporter (ผู้แจ้ง)	ข้อมูลเฉพาะของผู้แจ้งซ่อม เชื่อมกับบัญชีผู้ใช้ (User) และผู้พักอาศัย (Resident) อย่างละหนึ่งต่อหนึ่ง ใช้ระบุว่าใครเป็นผู้ส่งคำร้อง
Admin (ผู้ดูแลระบบ)	ข้อมูลเฉพาะของเจ้าหน้าที่หน่วยหอพัก เชื่อมกับบัญชีผู้ใช้แบบหนึ่งต่อหนึ่ง ทำหน้าที่อนุมัติ/ปฏิเสธคำร้อง มอบหมายงาน และยืนยันปิดงาน
Technician (ช่าง)	ข้อมูลเฉพาะของช่างประจำหอพัก เชื่อมกับบัญชีผู้ใช้แบบหนึ่งต่อหนึ่ง และเก็บความเชี่ยวชาญ (specialization) ของช่าง
Building (ตึก)	ตึกในหอพัก เก็บหมายเลขตึก ชื่อตึก และจำนวนชั้นทั้งหมด
Room (ห้อง)	ห้องพักที่อยู่ในแต่ละตึก เก็บเลขห้องและตึกที่ห้องนั้นสังกัด ใช้ระบุตำแหน่งที่ต้องไปซ่อม

ตารางที่ 2 คำอธิบายเอนทิตี (ต่อ)

เอนทิตี	คำอธิบาย
Resident (ผู้พักอาศัย)	ข้อมูลผู้พักอาศัย ได้แก่ ชื่อ นามสกุล เบอร์โทรศัพท์ และห้องที่พัก ผู้แจ้งซ่อมจะผูกกับผู้พักอาศัยเพื่อให้ระบบรู้ห้องของคำร้อง
RepairRequestStatusHistory	ประวัติการเปลี่ยนสถานะของคำร้อง เก็บสถานะเดิม สถานะใหม่ ผู้ทำรายการ (อ้างอิง User) และวันเวลา เพิ่มข้อมูลอย่างเดียวเพื่อใช้ตรวจสอบย้อนหลัง
RepairAssignment (งานที่มอบหมายช่าง)	งานที่ผู้ดูแลระบบมอบหมายให้ช่างตามคำร้อง เก็บคำร้อง ช่าง ผู้ดูแลระบบ วันที่มอบหมาย สถานะงาน และหมายเหตุของผู้ดูแลระบบและช่าง
RepairRequest	คำร้องแจ้งซ่อมของผู้แจ้ง เก็บประเภทงาน รายละเอียดอาการ หมายเหตุ ช่วงเวลาที่สะดวก สถานะ วันที่สร้าง ผู้แจ้ง ห้อง และผู้ดูแลระบบที่ดำเนินการ
RepairAssignmentStatusHistory	ประวัติการเปลี่ยนสถานะของงานที่มอบหมาย เก็บสถานะเดิม สถานะใหม่ ผู้ทำรายการ และวันเวลา
Notification (การแจ้งเตือน)	ข้อมูลการแจ้งเตือนที่ส่งถึงผู้ใช้งาน ประกอบด้วยผู้รับ ประเภท หัวข้อ ข้อความ รายการที่อ้างอิง สถานะการอ่าน และวันเวลาที่สร้าง

(2) ความสัมพันธ์ระหว่างเอนทิตี

ตารางที่ 3 ความสัมพันธ์ระหว่างเอนทิตี

เอนทิตีดั้งทาง	เอนทิตีปลายทาง	ความสัมพันธ์	ความหมาย
User	Admin	1 : 1	บัญชีผู้ใช้หนึ่งบัญชีเป็นผู้ดูแลระบบได้หนึ่งคน
User	Technician	1 : 1	บัญชีผู้ใช้หนึ่งบัญชีเป็นช่างได้หนึ่งคน
User	Reporter	1 : 1	บัญชีผู้ใช้หนึ่งบัญชีเป็นผู้แจ้งได้หนึ่งคน
Resident	Reporter	1 : 1	ผู้แจ้งหนึ่งคนคือผู้พักอาศัยหนึ่งคน
Building	Room	1 : N	ตึกหนึ่งมีหลายห้อง
Room	Resident	1 : 1	ห้องพักหนึ่งห้องมีผู้พักอาศัยได้หนึ่งคน และผู้พักอาศัยหนึ่งคนพักได้หนึ่งห้อง
Reporter	RepairRequest	1 : N	ผู้แจ้งส่งได้หลายคำร้อง
Room	RepairRequest	1 : N	ห้องหนึ่งมีคำร้องได้หลายคำร้อง
Admin	RepairRequest	1 : N	ผู้ดูแลระบบดำเนินการได้หลายคำร้อง (วางไว้ได้จนกว่าจะมีคนดำเนินการ)
RepairRequest	RepairRequestStatusHistory	1 : N	คำร้องหนึ่งมีประวัติสถานะหลายรายการ
User	RepairRequestStatusHistory	1 : N	ผู้ใช้หนึ่งคนเป็นผู้เปลี่ยนสถานะได้หลายครั้ง
RepairRequest	RepairAssignment	1 : 0..1	คำร้องหนึ่งรายการมีงานที่มอบหมายได้ไม่เกินหนึ่งรายการ
Technician	RepairAssignment	1 : N	ช่างหนึ่งคนรับได้หลายงาน
Admin	RepairAssignment	1 : N	ผู้ดูแลระบบหนึ่งคนมอบหมายได้หลายงาน
RepairAssignment	RepairAssignmentStatusHistory	1 : N	งานหนึ่งมีประวัติสถานะหลายรายการ
User	RepairAssignmentStatusHistory	1 : N	ผู้ใช้หนึ่งคนเป็นผู้เปลี่ยนสถานะงานได้หลายครั้ง
User	Notification	1 : N	ผู้ใช้หนึ่งคนรับการแจ้งเตือนได้หลายรายการ

3.2.5. Data Dictionary

ตารางที่ 4 บัญชีผู้ใช้ (users)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
user_id (PK)	UUID	รหัสบัญชีผู้ใช้	ajfls56dkjiuewwa544
first_name	varchar	ชื่อ	สมหญิง
last_name	varchar	นามสกุล	ใจดี
phone_no	varchar	เบอร์โทรศัพท์	0914257619
username	varchar	ชื่อบัญชีผู้ใช้	SMYinG
password	varchar	รหัสผ่านบัญชีผู้ใช้	0aksdm4a5wd8
role	varchar	บทบาทของบัญชีผู้ใช้	ADMIN
email	varchar	อีเมล	admin55@gmail.com

ตารางที่ 4 ผู้แจ้ง (reporter)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
reporter_id (PK)	UUID	รหัสผู้แจ้ง	ajfls56dkjiuewwa544
user_id (FK)	UUID	รหัสบัญชีผู้ใช้	สมหญิง
resident_id (FK)	UUID	รหัสผู้เช่า	ใจดี

ตารางที่ 5 ผู้ดูแลระบบ (admin)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
admin_id (PK)	UUID	รหัสแอดมิน	ajfls56dkjiuewwa544
user_id (FK)	UUID	รหัสบัญชีผู้ใช้	ajfls56dkjiuewwa544

ตารางที่ 6 ช่าง (technician)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
technician_id(PK)	UUID	รหัสช่าง	ajfls56dkjiuewwa544
specialization	varchar	ความสามารถพิเศษของช่าง	ประปา
user_id (FK)	UUID	รหัสบัญชีผู้ใช้	ajfls56dkjiuewwa544
specialization	varchar	งานที่ถนัด	งานไฟฟ้า

ตารางที่ 7 ผู้พักอาศัย (resident)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
resident_id (PK)	UUID	รหัสผู้เช่า	ajfls56dkjiuewwa544
room_no (FK)	UUID	เลขห้อง	1201
firstName	varchar	ชื่อผู้เช่า	สมชาย
lastName	varchar	นามสกุลผู้เช่า	ฮาจิวาเระ
phoneNo	varchar	หมายเลขโทรศัพท์	088-124-5679 09216424563

ตารางที่ 8 ตึก (building)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
building_no (PK)	Integer	หมายเลขตึก	1
building_name	varchar	ชื่อตึก	ตึก A
total_floor	Integer	จำนวนชั้นทั้งหมด	4
building_no (PK)	Integer	หมายเลขตึก	1

ตารางที่ 9 ห้อง (room)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
admin_id (PK)	UUID	รหัสแอดมิน	ajfls56dkjiuewwa544
user_id (FK)	UUID	รหัสบัญชีผู้ใช้	ajfls56dkjiuewwa544

ตารางที่ 10 คำร้องแจ้งซ่อม (repair_request)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
repair_request_id (PK)	UUID	รหัสคำร้องแจ้งซ่อม	ajfls56dkjiuewwa544
reporter_id (FK)	UUID	รหัสผู้แจ้ง	ajfls56dkjiuewwa544
admin_id (FK)	UUID	รหัสแอดมิน	ajfls56dkjiuewwa544
room_no (FK)	Integer	เลขห้อง	1201
repair_type	varchar	ประเภทการซ่อม	PLUMBING
status	varchar	สถานะ	SUBMITTED

ตารางที่ 11 คำร้องแจ้งซ่อม (repair_request) (ต่อ)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
description	String	อาการแจ้งซ่อม	ขาโต๊ะหัก
reporter_note	varchar	หมายเหตุ	ไม่มีคนอยู่ห้อง ใช้กุญแจสำรองได้
start_date_time	timestamp	ช่วงเวลาที่สะดวก (เริ่ม)	2026-09-25 09:00:00
end_date_time	timestamp	ช่วงเวลาที่สะดวก (สิ้นสุด)	2026-09-25 10:00:00
created_at	timestamp	ช่วงเวลาที่ส่งคำร้อง	2026-10-04 16:03:56.945246

ตารางที่ 11 ประวัติการเปลี่ยนแปลงสถานะคำร้อง (repair_request_status_history)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
resident_id (PK)	UUID	รหัสผู้เช่า	ajfls56dkjiuewwa544
request_history_id (PK)	UUID	รหัสประวัติการเปลี่ยนแปลงคำร้อง	ajfls56dkjiuewwa544
repair_request_id (FK)	UUID	รหัสคำร้องแจ้งซ่อม	ajfls56dkjiuewwa544
change_by (FK)	UUID	รหัสผู้เปลี่ยนสถานะคำร้อง	ajfls56dkjiuewwa544
new_status	varchar	สถานะคำร้องใหม่	IN-PROGESS
previous_status	varchar	สถานะคำร้องปัจจุบัน	SUBMITTED
change_date	DateTime	วันเวลาเปลี่ยนสถานะ	2026-09-15 19:58:00

ตารางที่ 12 รายการมอบหมายงาน (repair_assignment)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
assignment_id (PK)	UUID	รหัสรายการมอบหมายงาน	ajfls56dkjiuewwa544
repair_request_id (FK)	UUID	รหัสคำร้องแจ้งซ่อม	ajfls56dkjiuewwa544
technician_id (FK)	UUID	รหัสช่าง	ajfls56dkjiuewwa544
admin_id (FK)	UUID	รหัสแอดมิน	ajfls56dkjiuewwa544
job_status	varchar	สถานะงาน	COMPLETED
admin_note	varchar	บันทึกจากผู้ดูแลระบบ	มาเอากุญแจสำรองที่สำนักงานหอก่อน
technician_note	varchar	บันทึกการทำงานของช่าง	ตรวจสอบหลอดไฟไม่ติด สาเหตุจากรอย หนูกัด เปลี่ยนสายไฟใหม่
assign_date	timestamp	วัน เวลาที่มอบหมายงาน	2026-09-15 19:58:00

ตารางที่ 13 ประวัติการเปลี่ยนแปลงสถานะรายการมอบหมายงาน (repair_assignment_status_history)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
history_id	UUID	รหัสประวัติการเปลี่ยนแปลงสถานะ รายการมอบหมายงาน	ajfls56dkjiuewwa544
assignment_id (FK)	UUID	รหัสรายการมอบหมายงาน	ajfls56dkjiuewwa544
change_by (FK)	UUID	รหัสผู้เปลี่ยนสถานะงาน	ajfls56dkjiuewwa544
new_status	varchar	สถานะงานใหม่	COMPLETED
change_date	timestamp	วันที่ดำเนินการ	2026-09-15 19:58:00
previous_status	varchar	สถานะงานเดิม	IN-PROGRESS

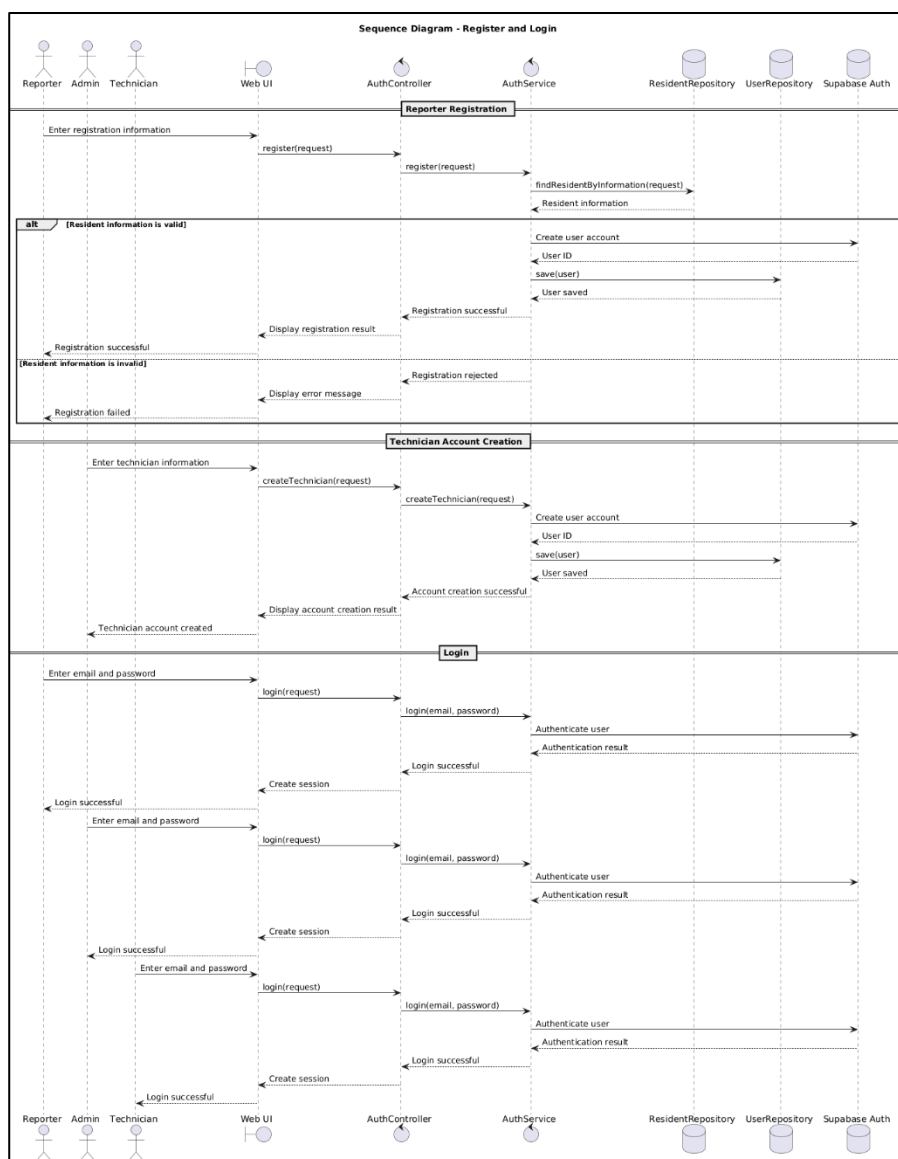
ตารางที่ 14 การแจ้งเตือน (notification)

ชื่อคอลัมน์	ชนิดของข้อมูล	คำอธิบาย	ตัวอย่าง
notification_id (PK)	UUID	รหัสการแจ้งเตือน	1b3d5f7a-9c2e-4a6b-8d0f-2a4c6e8a0b1c
user_id (FK)	UUID	รหัสผู้รับการแจ้งเตือน	5b8a2c4d-6e1f-4a3b-9c7d-2e4f6a8b0c1d
type	varchar	ประเภทการแจ้งเตือน	STATUS_CHANGED
title	varchar	หัวข้อการแจ้งเตือน	คำร้องได้รับการอนุมัติ
message	text	ข้อความการแจ้งเตือน	คำร้องแจ้งซ่อมห้อง 1201 ได้รับการอนุมัติแล้ว
reference_id	UUID	รหัสรายการที่อ้างอิง	e5a7c9e1-3b5d-4f7a-9c1e-3a5c7e9b1d3f
is_read	boolean	สถานะการอ่าน	false
created_at	timestamp	เวลาที่สร้าง	2026-09-24 18:30:00
notification_id (PK)	UUID	รหัสการแจ้งเตือน	1b3d5f7a-9c2e-4a6b-8d0f-2a4c6e8a0b1c

3.2.6. Sequence Diagram

Sequence Diagram ใช้อธิบายขั้นตอนการทำงานของฟังก์ชันหลักของระบบ ได้แก่ การสมัครสมาชิกและเข้าสู่ระบบ การส่งคำร้องแจ้งซ่อม การดูแลจัดการคำร้อง การตรวจสอบคำร้องโดยผู้ดูแลระบบ การมอบหมายงานให้ช่าง การดำเนินงานซ่อม การยืนยันผลการซ่อม และการจัดการข้อมูลบัญชีและผู้พักอาศัย

(1) การสมัครสมาชิกและเข้าสู่ระบบ (Register and Login)



ภาพที่ 10 Sequence Diagram การสมัครสมาชิกและเข้าสู่ระบบ

Sequence Diagram: Register and Login แสดงลำดับการทำงานของกระบวนการสมัครบัญชีผู้ใช้และการเข้าสู่ระบบของระบบแจ้งซ่อมในหอพัก โดยแสดงการติดต่อสื่อสารระหว่างผู้ใช้งาน ส่วนติดต่อผู้ใช้ (Web UI), Controller, Service, Repository และ Supabase Authentication ตามลำดับการทำงานของระบบ

ในส่วนของการ Register ผู้แจ้งกรอกข้อมูลสำหรับสมัครสมาชิกผ่าน Web UI จากนั้น Web UI ส่งข้อมูลไปยัง AuthController และ AuthController ส่งต่อข้อมูลให้ AuthService ตรวจสอบข้อมูลผู้สมัครกับข้อมูลผู้พักอาศัยในตาราง “ผู้พักอาศัย” ผ่าน ResidentRepository เพื่อยืนยันว่าผู้สมัครมีรายชื่อเป็นผู้พักอาศัยที่อยู่ในระบบจริง

หากข้อมูลของผู้สมัครตรงกับข้อมูลในตาราง “ผู้พักอาศัย” ระบบจะดำเนินการสร้างบัญชีผู้ใช้ผ่าน Supabase Authentication และได้รับ User ID กลับมา จากนั้น AuthService จะบันทึกข้อมูลผู้ใช้งานในฐานข้อมูลผ่าน UserRepository และส่งผลการสมัครสมาชิกกลับไป Web UI เพื่อแจ้งว่าการสมัครสมาชิกสำเร็จ

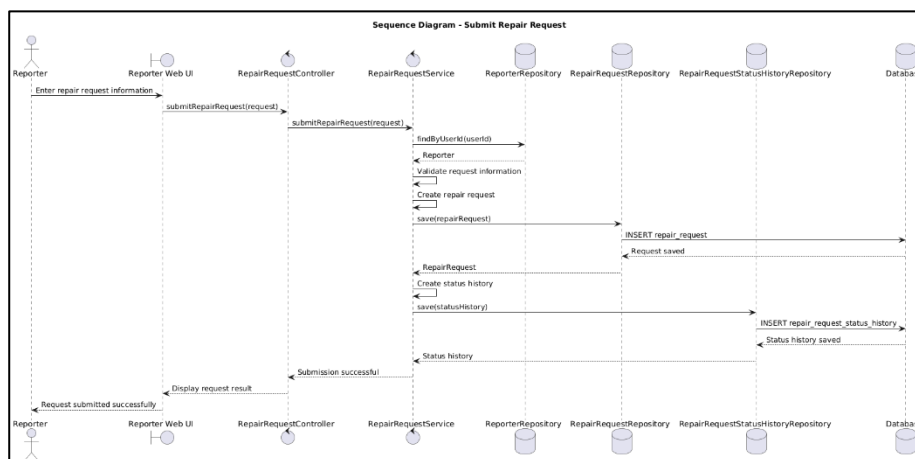
ในกรณีที่ไม่มีพบข้อมูลผู้สมัครในตาราง “ผู้พักอาศัย” หรือข้อมูลไม่ตรงกับข้อมูลที่ระบบกำหนด ระบบจะปฏิเสธการสมัครสมาชิกและแสดงข้อความแจ้งเตือนแก่ผู้ใช้งาน เพื่อป้องกันบุคคลที่ไม่มีรายชื่อเป็นผู้พักอาศัยในระบบจากการสมัครบัญชีในฐานผู้แจ้ง

สำหรับ “การสร้างบัญชีของช่าง” ผู้ดูแลระบบเป็นผู้สร้างบัญชีสำหรับช่างประจำหอพัก โดยกรอกข้อมูลของช่างผ่าน Web UI แล้วส่งข้อมูลไปยัง AuthController และ AuthService จากนั้นระบบสร้างบัญชีผ่าน Supabase Authentication และบันทึกข้อมูลผู้ใช้งานในฐานข้อมูลผ่าน UserRepository เมื่อดำเนินการเสร็จสิ้น ระบบจะแจ้งผลการสร้างบัญชีให้ผู้ดูแลระบบทราบ ทั้งนี้ช่างไม่สามารถสมัครบัญชีด้วยตนเองผ่านกระบวนการสมัครของผู้แจ้งได้

ในส่วนของการลงชื่อเข้าใช้ ผู้ใช้งานทั้ง 3 บทบาท ได้แก่ ผู้แจ้ง, ผู้ดูแลระบบ และ ช่าง สามารถเข้าสู่ระบบด้วยอีเมลและรหัสผ่านของตนเอง โดย Web UI จะส่งข้อมูลไปยัง AuthController และ AuthService ซึ่งทำหน้าที่ตรวจสอบข้อมูลการเข้าสู่ระบบกับ Supabase Authentication เมื่อการยืนยันตัวตนสำเร็จ ระบบจะสร้าง Session สำหรับผู้ใช้งาน และนำผู้ใช้งานเข้าสู่ระบบตามสิทธิ์ของบทบาทที่ได้รับ

Sequence Diagram นี้แสดงให้เห็นว่า ระบบมีการแยกกระบวนการสมัครบัญชีตามบทบาท โดยผู้แจ้งต้องผ่านการตรวจสอบกับข้อมูลผู้พักอาศัย ก่อนสมัครสมาชิก ขณะที่บัญชีของช่างถูกสร้างโดยผู้ดูแลระบบ และผู้ใช้งานทั้งสามบทบาทสามารถเข้าสู่ระบบเพื่อใช้งานฟังก์ชันต่าง ๆ ตามสิทธิ์ของตนเอง

(2) การส่งคำร้องแจ้งซ่อม (Submit Repair Request)



ภาพที่ 11 Sequence Diagram การส่งคำร้องแจ้งซ่อม

Sequence Diagram: Submit Repair Request แสดงลำดับการทำงานของกระบวนการส่งคำร้องแจ้งซ่อมของผู้แจ้ง โดยแสดงการติดต่อสื่อสารระหว่างผู้แจ้ง ส่วนติดต่อผู้ใช้ (Reporter Web UI), Controller, Service, Repository และฐานข้อมูล เพื่อแสดงขั้นตอนตั้งแต่ผู้แจ้งกรอกข้อมูลคำร้องจนกระทั่งระบบบันทึกคำร้องและประวัติสถานะลงในฐานข้อมูลเรียบร้อยแล้ว

เริ่มต้นจากผู้แจ้งกรอกข้อมูลรายละเอียดการแจ้งซ่อมผ่าน Reporter Web UI เช่น ประเภทงานซ่อม รายละเอียดปัญหา และข้อมูลอื่น ๆ ที่เกี่ยวข้อง จากนั้น Web UI จะส่งข้อมูลคำร้องไปยัง RepairRequestController ผ่านเมธอด submitRepairRequest(request) และ Controller จะส่งต่อข้อมูลให้ RepairRequestService เพื่อดำเนินการตาม Business Logic ของระบบ

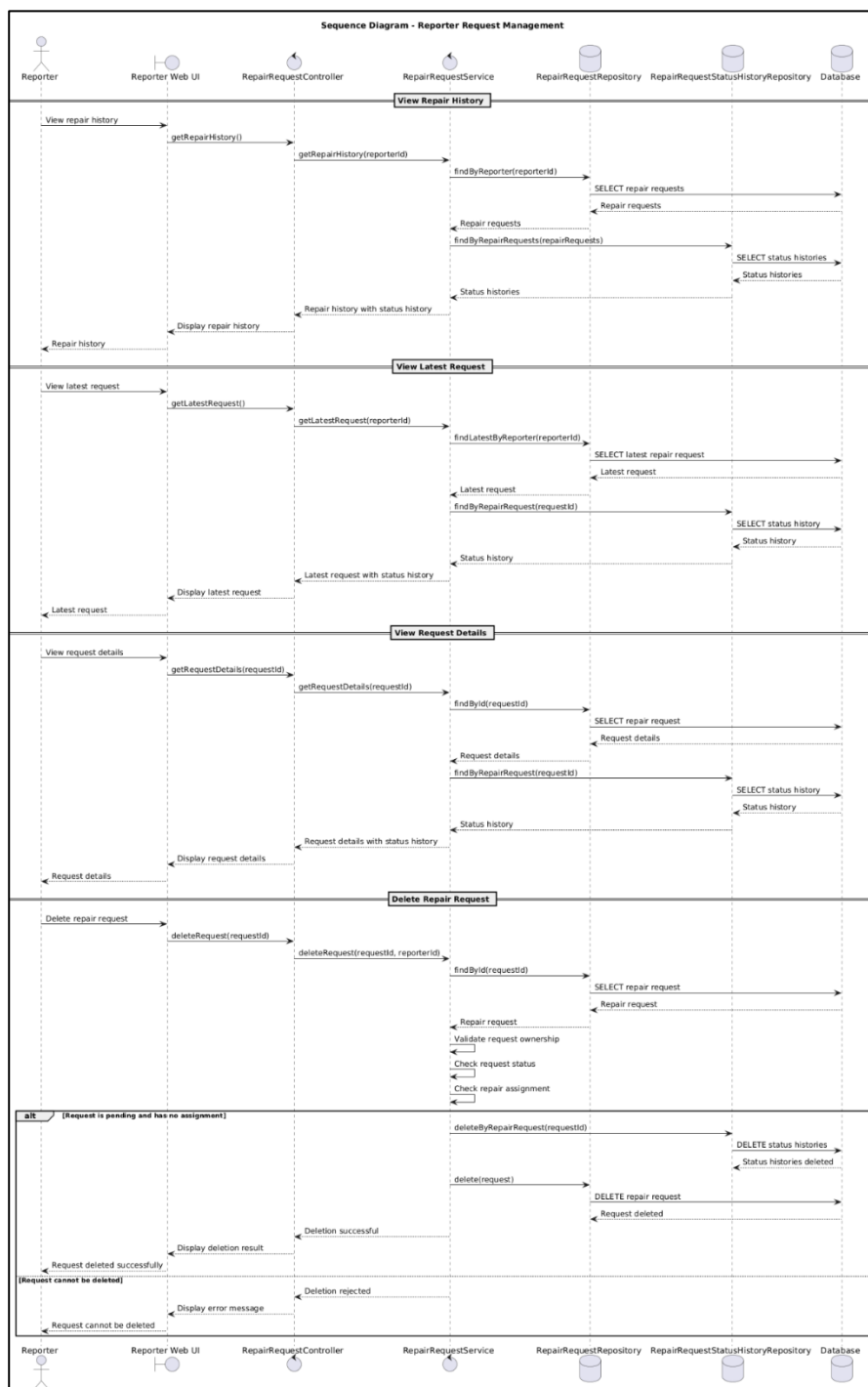
RepairRequestService จะตรวจสอบข้อมูลของผู้แจ้งผ่าน ReporterRepository โดยค้นหาข้อมูลของผู้แจ้ง ผ่าน User ID เพื่อยืนยันว่าผู้ส่งคำร้องเป็นผู้ใช้งานที่มีสิทธิ์ในการแจ้งซ่อม จากนั้น Service จะตรวจสอบความถูกต้องของข้อมูลคำร้อง และสร้างออบเจกต์ RepairRequest สำหรับบันทึกข้อมูลคำร้องใหม่

เมื่อสร้างคำร้องเรียบร้อยแล้ว RepairRequestService จะส่ง RepairRequest ไปยัง RepairRequestRepository เพื่อบันทึกข้อมูลลงในตาราง repair_request ของฐานข้อมูล เมื่อบันทึกสำเร็จ ระบบจะได้รับข้อมูล RepairRequest ที่ถูกสร้างกลับมา

หลังจากบันทึกคำร้องสำเร็จ ระบบจะสร้าง RepairRequestStatusHistory เพื่อ บันทึกประวัติสถานะเริ่มต้นของคำร้อง โดยกำหนดสถานะตามสถานะเริ่มต้นของคำร้อง เช่น PENDING และบันทึกข้อมูลผู้ที่ดำเนินการเปลี่ยนแปลงสถานะ จากนั้น RepairRequestStatusHistoryRepository จะทำหน้าที่บันทึกข้อมูลประวัติสถานะลงในตาราง repair_request_status_history

เมื่อทั้งข้อมูล RepairRequest และ RepairRequestStatusHistory ถูกบันทึกสำเร็จ RepairRequestService จะส่งผลการดำเนินการกลับไปยัง RepairRequestController จากนั้น Controller จะส่งผลลัพธ์ไปยัง Reporter Web UI และระบบจะแสดงข้อความแจ้งให้ ผู้แจ้งทราบว่าการส่งคำร้องแจ้งซ่อมสำเร็จ

(3) การดูแลจัดการคำร้องของผู้แจ้ง (Reporter Request Management)



ภาพที่ 12 Sequence Diagram การดูแลจัดการคำร้องของผู้แจ้ง

Sequence Diagram: Reporter Request Management แสดงลำดับการทำงานของ
 ของผู้แจ้งในการจัดการคำร้องแจ้งซ่อมของตนเอง โดยครอบคลุมการดูประวัติคำร้อง การดูคำ
 ร้องล่าสุด การดูรายละเอียดคำร้อง และการลบคำร้องที่ยังไม่เข้าสู่กระบวนการดำเนินงาน
 โดยที่ Sequence Diagram นี้มีความต่อเนื่องจากกระบวนการ Submit Repair Request
 ซึ่งเมื่อมีการสร้างคำร้อง ระบบจะบันทึกข้อมูล RepairRequest พร้อมกับ
 RepairRequestStatusHistory เพื่อใช้ติดตามการเปลี่ยนแปลงสถานะของคำร้อง

ในส่วนของ View Repair History ผู้แจ้งเลือกดูประวัติการแจ้งซ่อมผ่าน Reporter
 Web UI จากนั้นระบบจะส่งคำขอไปยัง RepairRequestController และ
 RepairRequestService ตามลำดับ โดย Service จะค้นหาคำร้องของผู้แจ้งผ่าน
 RepairRequestRepository และค้นหาประวัติสถานะของคำร้องผ่าน
 RepairRequestStatusHistoryRepository จากนั้นนำข้อมูลคำร้องและประวัติสถานะมา
 ประกอบกันก่อนส่งกลับไปแสดงผลให้ผู้แจ้ง

ในส่วนของ View Latest Request ระบบจะค้นหาคำร้องล่าสุดของผู้แจ้งผ่าน
 RepairRequestRepository จากนั้นใช้ RepairRequestStatusHistoryRepository เพื่อ
 ค้นหาประวัติสถานะที่เกี่ยวข้องกับคำร้องดังกล่าว ทำให้ผู้แจ้งสามารถตรวจสอบทั้งข้อมูลคำ
 ร้องล่าสุดและสถานะหรือประวัติการเปลี่ยนแปลงของคำร้องได้

ในส่วนของ View Request Details ผู้แจ้งสามารถเลือกดูรายละเอียดของคำร้องที่
 ต้องการ ระบบจะค้นหาข้อมูลคำร้องจาก RepairRequestRepository โดยใช้ requestId
 และค้นหาประวัติสถานะที่เกี่ยวข้องผ่าน RepairRequestStatusHistoryRepository จากนั้น
 RepairRequestService จะส่งข้อมูลรายละเอียดคำร้องพร้อมประวัติสถานะกลับไปยัง
 Controller และ Web UI เพื่อแสดงผลให้ผู้แจ้งตรวจสอบ

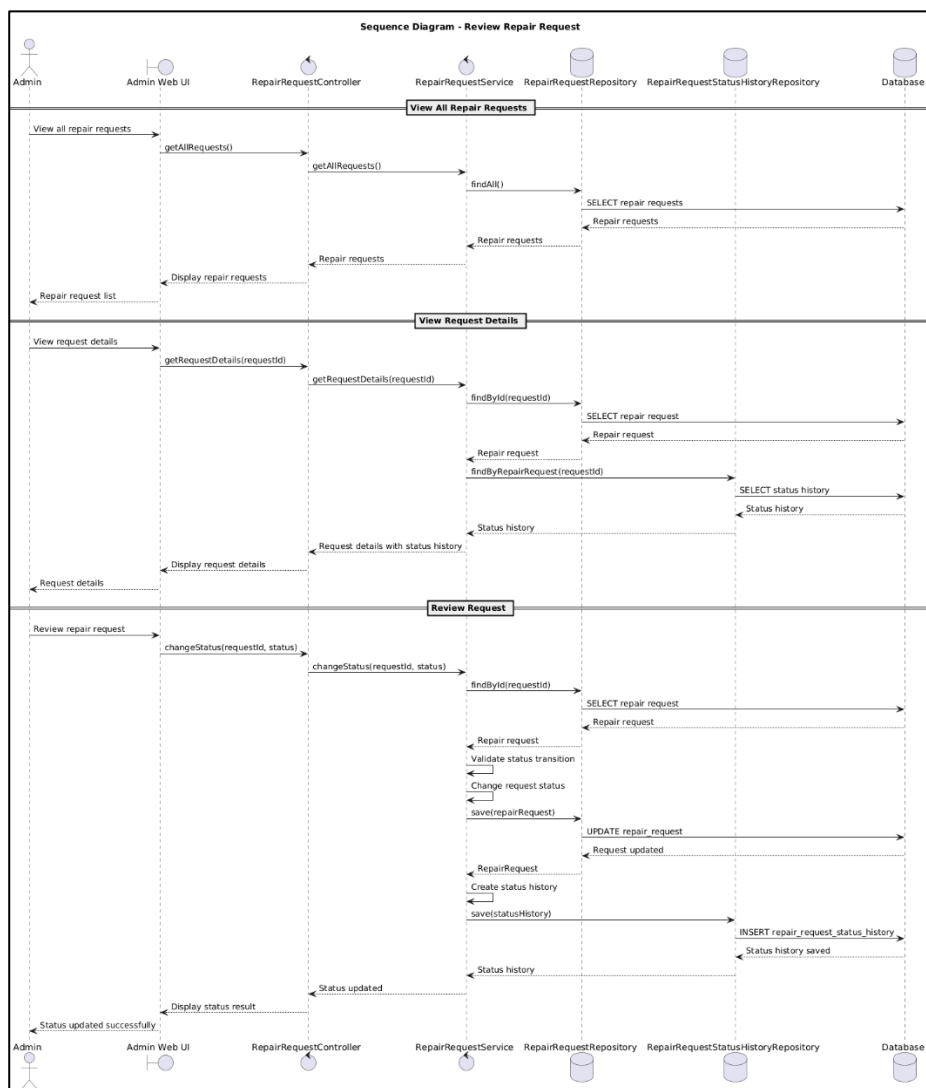
สำหรับ Delete Repair Request ผู้แจ้งสามารถเลือกคำร้องของตนเองเพื่อลบได้ โดย
 RepairRequestController จะส่ง requestId และข้อมูลผู้แจ้งไปยัง
 RepairRequestService ซึ่ง Service จะตรวจสอบก่อนว่าคำร้องดังกล่าวเป็นของผู้แจ้งที่
 กำลังใช้งานระบบจริงหรือไม่ จากนั้นตรวจสอบสถานะของคำร้องและตรวจสอบว่ามี
 RepairAssignment ที่ถูกสร้างขึ้นแล้วหรือไม่

หากคำร้องยังอยู่ในสถานะที่สามารถลบได้ เช่น อยู่ในสถานะรอดำเนินการและยังไม่มี
การมอบหมายงานให้ช่าง ระบบจะลบข้อมูล RepairRequestStatusHistory ที่เกี่ยวข้องก่อน
เพื่อไม่ให้เกิดข้อมูลประวัติสถานะที่ยังอ้างอิงถึงคำร้องที่ถูกลบ จากนั้นจึงลบข้อมูล
RepairRequest ออกจากฐานข้อมูล เมื่อดำเนินการสำเร็จ ระบบจะแจ้งผลการลบคำร้องให้ผู้
แจ้งทราบ

ในกรณีที่คำร้องไม่เป็นไปตามเงื่อนไข เช่น คำร้องได้รับการอนุมัติแล้ว หรือมีการสร้าง
RepairAssignment เพื่อบริการช่างแล้ว ระบบจะปฏิเสธการลบและแสดงข้อความ
แจ้งเตือนแก่ผู้แจ้ง เพื่อป้องกันการลบข้อมูลคำร้องที่เข้าสู่กระบวนการดำเนินงาน

Sequence Diagram นี้แสดงให้เห็นการทำงานของระบบในการจัดการคำร้องของผู้
แจ้งตั้งแต่การเรียกดูข้อมูลคำร้องและประวัติสถานะ ไปจนถึงการลบคำร้องตามเงื่อนไขที่
ระบบกำหนด โดย RepairRequest และ RepairRequestStatusHistory มีความสัมพันธ์กัน
ในการจัดเก็บข้อมูลคำร้องและประวัติการเปลี่ยนแปลงสถานะอย่างต่อเนื่อง

(4) การตรวจสอบคำร้องโดยผู้ดูแลระบบ (Review Repair Request)



ภาพที่ 13 Sequence Diagram การตรวจสอบคำร้องโดยผู้ดูแลระบบ

Sequence Diagram : Review Repair Request แสดงลำดับการทำงานของผู้ดูแลระบบในการตรวจสอบและพิจารณาคำร้องแจ้งซ่อม โดยครอบคลุมการดูรายการคำร้องทั้งหมด การดูรายละเอียดคำร้อง และการเปลี่ยนแปลงสถานะของคำร้อง รวมถึงการบันทึกประวัติการเปลี่ยนแปลงสถานะลงใน RepairRequestStatusHistory เพื่อให้สามารถตรวจสอบการดำเนินงานย้อนหลังได้

ในส่วนของ View All Repair Requests ผู้ดูแลระบบเลือกดูรายการคำร้องแจ้งซ่อมทั้งหมดผ่าน Admin Web UI จากนั้นระบบส่งคำขอไปยัง RepairRequestController และ RepairRequestService ตามลำดับ โดย Service เรียกใช้ RepairRequestRepository เพื่อค้นหาข้อมูลคำร้องทั้งหมดจากฐานข้อมูลผ่านคำสั่ง findAll() เมื่อได้รับข้อมูลแล้ว ระบบจะส่งรายการคำร้องกลับไปยัง Web UI เพื่อแสดงให้ผู้ดูแลระบบตรวจสอบ

ในส่วนของ View Request Details ผู้ดูแลระบบจะเลือกคำร้องที่ต้องการตรวจสอบระบบจะค้นหาข้อมูล RepairRequest ผ่าน RepairRequestRepository โดยใช้ requestId จากนั้น RepairRequestService จะเรียกใช้ RepairRequestStatusHistoryRepository เพื่อค้นหาประวัติการเปลี่ยนแปลงสถานะของคำร้องดังกล่าว ทำให้ผู้ดูแลระบบสามารถตรวจสอบทั้งรายละเอียดของคำร้องและลำดับประวัติสถานะที่เกิดขึ้นก่อนหน้านี้ได้

สำหรับส่วน Review Request ผู้ดูแลระบบเลือกพิจารณาคำร้องและกำหนดสถานะใหม่ เช่น APPROVED หรือ REJECTED จากนั้น Web UI จะส่ง requestId และสถานะที่ต้องการเปลี่ยนไปยัง RepairRequestController และส่งต่อไปยัง RepairRequestService

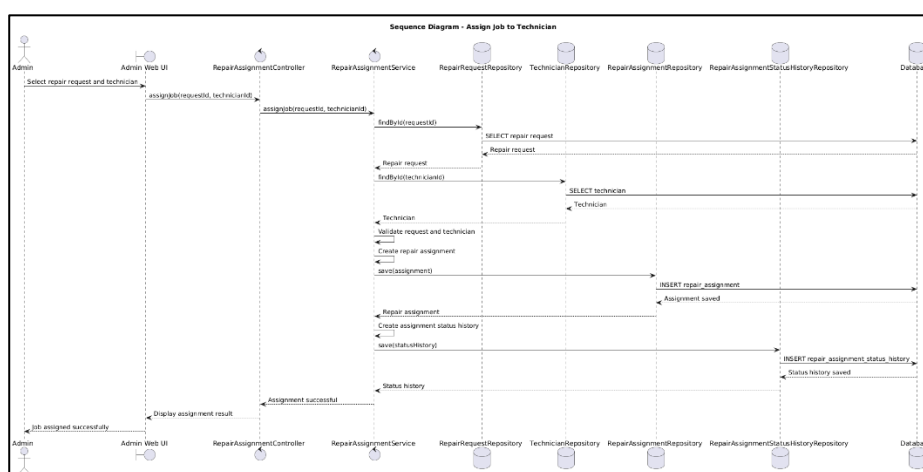
RepairRequestService จะค้นหาคำร้องจาก RepairRequestRepository ก่อนตรวจสอบว่าการเปลี่ยนสถานะดังกล่าวเป็นไปตามเงื่อนไขของระบบหรือไม่ เมื่อผ่านการตรวจสอบแล้ว Service จะเปลี่ยนสถานะของ RepairRequest และบันทึกข้อมูลที่เปลี่ยนแปลงลงในฐานข้อมูลผ่าน RepairRequestRepository

หลังจากบันทึกสถานะใหม่ของ RepairRequest สำเร็จ ระบบจะสร้าง RepairRequestStatusHistory เพื่อบันทึกประวัติการเปลี่ยนแปลงสถานะครั้งนั้น โดยข้อมูลประวัติจะเชื่อมโยงกับคำร้องที่เกี่ยวข้อง และสามารถบันทึกข้อมูลสำคัญ เช่น สถานะใหม่ ผู้ที่ดำเนินการเปลี่ยนสถานะ และเวลาที่ เกิดการเปลี่ยนแปลง จากนั้น RepairRequestStatusHistoryRepository จะทำหน้าที่บันทึกประวัติดังกล่าวลงในตาราง repair_request_status_history

เมื่อทั้งข้อมูล RepairRequest และ RepairRequestStatusHistory ถูกบันทึกเรียบร้อยแล้ว ระบบจะส่งผลการดำเนินการกลับไปยัง RepairRequestController และแสดงผลผ่าน Admin Web UI เพื่อแจ้งให้ผู้ดูแลระบบทราบว่าการเปลี่ยนสถานะสำเร็จ

ในการบันทึก RepairRequestStatusHistory ขั้นตอนนี้มีความสำคัญ เนื่องจากการเปลี่ยนสถานะของคำร้องแต่ละครั้งถือเป็นเหตุการณ์ที่ควรมีข้อมูลสำหรับตรวจสอบย้อนหลัง เช่น คำร้องถูกส่งเข้าระบบเมื่อใด ได้รับการอนุมัติหรือไม่อนุมัติโดยผู้ใด และมีการเปลี่ยนสถานะในลำดับใด ทำให้ระบบสามารถตรวจสอบกระบวนการดำเนินงานของคำร้องได้อย่างเป็นระบบ

(5) การมอบหมายงานให้ช่าง (Assign Job to Technician)



ภาพที่ 14 Sequence Diagram การมอบหมายงานให้ช่าง

Sequence Diagram: Assign Job to Technician แสดงลำดับการทำงานของผู้อยู่ดูแลระบบในการมอบหมายงานซ่อมให้แก่ช่างประจำหอพัก โดยแสดงการทำงานตั้งแต่การเลือกคำร้องและช่างที่จะรับผิดชอบ การตรวจสอบข้อมูล การสร้าง RepairAssignment ตลอดจนการบันทึกประวัติสถานะของงานที่ได้รับมอบหมายลงในฐานข้อมูล

เริ่มต้นจากผู้ดูแลระบบเลือกคำร้องแจ้งซ่อมและช่างที่ต้องการมอบหมายผ่าน Admin Web UI จากนั้น Web UI จะส่ง requestId และ technicianId ไปยัง RepairAssignmentController ผ่านคำสั่ง assignJob(requestId, technicianId) และ Controller จะส่งข้อมูลต่อไปยัง RepairAssignmentService เพื่อดำเนินการตาม Business Logic ของระบบ

RepairAssignmentService จะค้นหาข้อมูลคำร้องจาก RepairRequestRepository โดยใช้ requestId เพื่อยืนยันว่าคำร้องที่ต้องการมอบหมายมีอยู่ในระบบ จากนั้นค้นหาข้อมูล

ช่างผ่าน TechnicianRepository โดยใช้ technicianId เพื่อยืนยันว่าช่างที่เลือกมีอยู่ในระบบ และสามารถรับงานได้

เมื่อได้รับข้อมูลคำร้องและช่างแล้ว Service จะตรวจสอบเงื่อนไขที่เกี่ยวข้อง เช่น ความถูกต้องของคำร้องและข้อมูลช่าง จากนั้นสร้างออบเจกต์ RepairAssignment เพื่อเชื่อมโยงคำร้องกับช่างผู้รับผิดชอบงาน

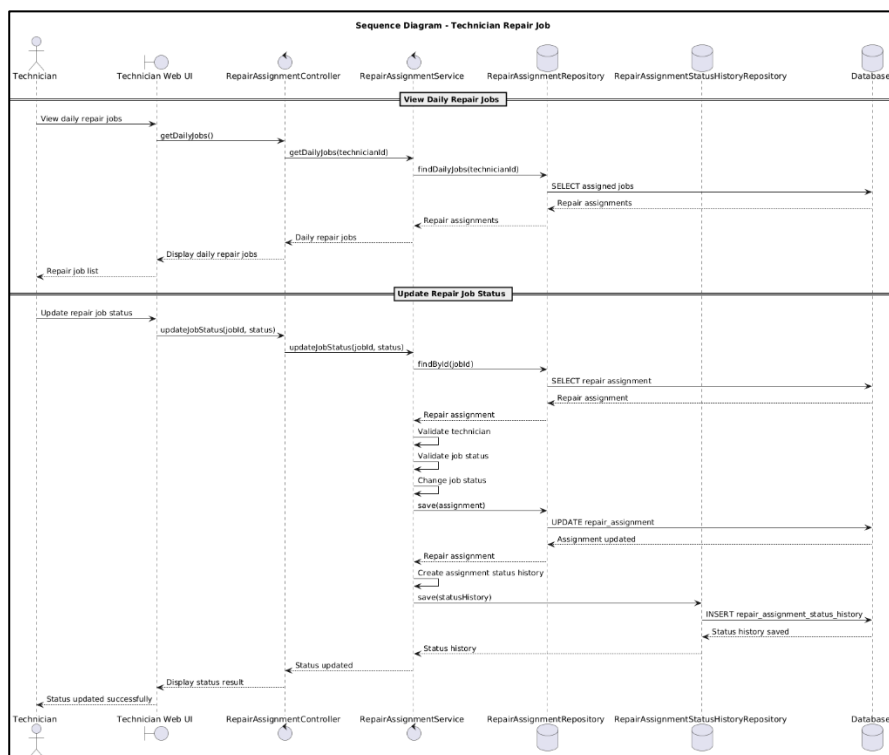
หลังจากสร้าง RepairAssignment แล้ว ระบบจะบันทึกข้อมูลลงในฐานข้อมูลผ่าน RepairAssignmentRepository โดยเพิ่มข้อมูลลงในตาราง repair_assignment เมื่อการบันทึกสำเร็จ ระบบจะได้รับข้อมูล RepairAssignment กลับมายัง Service

จากนั้น RepairAssignmentService จะสร้าง RepairAssignmentStatusHistory เพื่อบันทึกสถานะเริ่มต้นของงานที่ได้รับมอบหมาย โดยประวัติดังกล่าวจะเชื่อมโยงกับ RepairAssignment ที่เพิ่งสร้างขึ้น และสามารถใช้สำหรับติดตามการเปลี่ยนแปลงสถานะของงานในภายหลัง จากนั้น RepairAssignmentStatusHistoryRepository จะบันทึกข้อมูลดังกล่าวลงในตาราง repair_assignment_status_history

การบันทึก Status History ในขั้นตอนการมอบหมายงานมีความสำคัญ เนื่องจากทำให้ระบบสามารถติดตามประวัติของงานซ่อมแยกออกจากประวัติของคำร้องได้ โดย RepairRequestStatusHistory ใช้ติดตามการเปลี่ยนแปลงสถานะของคำร้องแจ้งซ่อม ขณะที่ RepairAssignmentStatusHistory ใช้ติดตามการเปลี่ยนแปลงสถานะของงานที่มอบหมายให้ช่าง

เมื่อข้อมูล RepairAssignment และ RepairAssignmentStatusHistory ถูกบันทึกสำเร็จ RepairAssignmentService จะส่งผลการดำเนินการกลับไปยัง RepairAssignmentController จากนั้น Controller จะแจ้งผลผ่าน Admin Web UI เพื่อแสดงให้ผู้ดูแลระบบทราบว่าการมอบหมายงานให้ช่างสำเร็จ

(6) การดำเนินงานซ่อม (Technician Repair Job)



ภาพที่ 15 Sequence Diagram การดำเนินงานซ่อม

Sequence Diagram: Technician Repair Job แสดงลำดับการทำงานของช่างประจำหอพักในการตรวจสอบงานซ่อมที่ได้รับมอบหมายและการปรับปรุงสถานะของงาน โดยแสดงการติดต่อสื่อสารระหว่าง Technician, Technician Web UI, RepairAssignmentController, RepairAssignmentService, RepairAssignmentRepository และ RepairAssignmentStatusHistoryRepository

ในส่วนของ View Daily Repair Jobs ช่างเข้าสู่หน้าแสดงรายการงานซ่อมประจำวันผ่าน Technician Web UI จากนั้นระบบจะส่งคำขอไปยัง RepairAssignmentController และส่งต่อไปยัง RepairAssignmentService พร้อมข้อมูล technicianId เพื่อระบุช่างที่กำลังเข้าสู่ระบบ

RepairAssignmentService จะเรียกใช้ RepairAssignmentRepository เพื่อค้นหา รายการ RepairAssignment ที่ได้รับมอบหมายให้กับช่าง จากนั้น Repository จะดึงข้อมูล จากฐานข้อมูลและส่งรายการงานกลับมายัง Service เมื่อได้รับข้อมูลแล้ว Service จะส่ง รายการงานกลับไปยัง Controller และ Web UI เพื่อแสดงรายการงานซ่อมให้ช่างตรวจสอบ

ในส่วนของการ Update Repair Job Status ช่างสามารถเปลี่ยนสถานะของงานซ่อมที่ ตนเองได้รับมอบหมายผ่าน Web UI เช่น เปลี่ยนสถานะเป็น COMPLETED หรือ IN_COMPLETED ตามเงื่อนไขของระบบ เมื่อช่างเลือกสถานะใหม่ ระบบจะส่ง jobId และ สถานะที่ต้องการเปลี่ยนไปยัง RepairAssignmentController และส่งต่อไปยัง RepairAssignmentService

RepairAssignmentService จะค้นหา RepairAssignment จาก RepairAssignmentRepository โดยใช้ jobId จากนั้นตรวจสอบว่าช่างที่กำลังดำเนินการเป็น ช่างที่ได้รับมอบหมายให้กับงานดังกล่าวจริงหรือไม่ และตรวจสอบว่าสถานะใหม่สามารถ เปลี่ยนได้ตามกฎของระบบหรือไม่ เมื่อผ่านการตรวจสอบ Service จะเปลี่ยนสถานะของ RepairAssignment และบันทึกข้อมูลลงในฐานข้อมูลผ่าน RepairAssignmentRepository

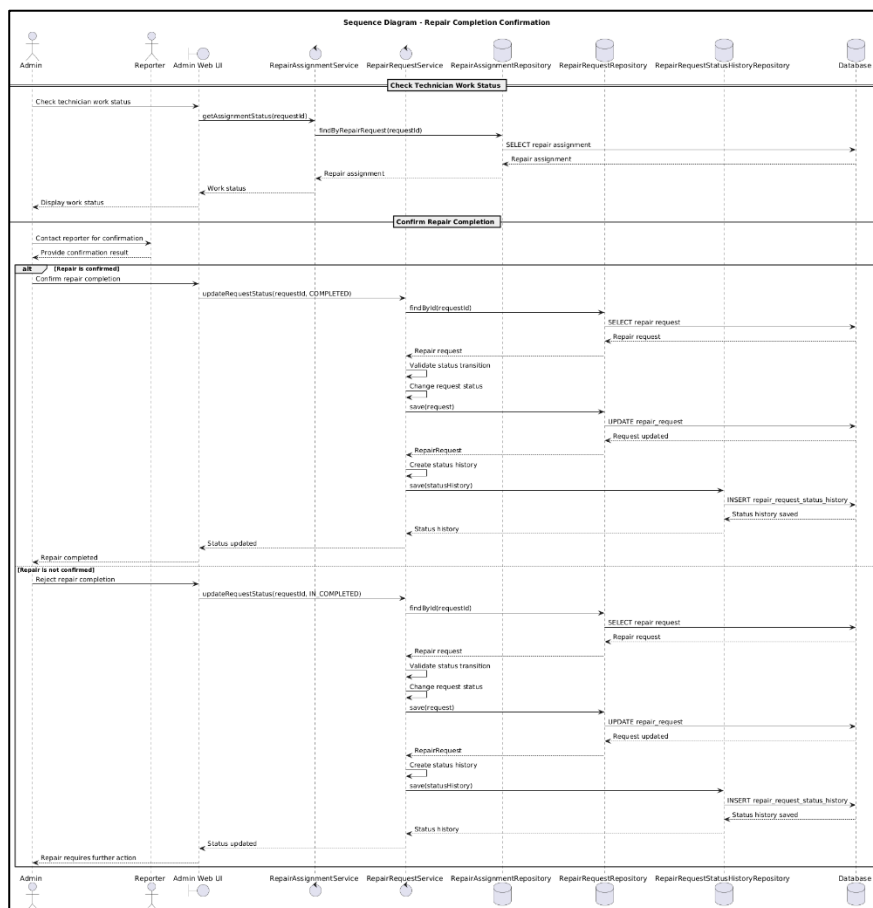
หลังจากบันทึกสถานะใหม่ของ RepairAssignment สำเร็จ ระบบจะสร้าง RepairAssignmentStatusHistory เพื่อเก็บประวัติการเปลี่ยนแปลงสถานะของงาน โดย ประวัติดังกล่าวจะเชื่อมโยงกับ RepairAssignment ที่เกี่ยวข้อง และสามารถใช้ตรวจสอบได้ ว่างานถูกเปลี่ยนสถานะเมื่อใด และมีการดำเนินการเปลี่ยนสถานะอย่างไร จากนั้น RepairAssignmentStatusHistoryRepository จะบันทึกข้อมูลลงในตาราง repair_assignment_status_history

เมื่อการเปลี่ยนสถานะและการบันทึกประวัติสถานะเสร็จสมบูรณ์ RepairAssignmentService จะส่งผลการดำเนินการกลับไปยัง RepairAssignmentController และ Web UI จะแสดงผลให้ช่างทราบว่าการเปลี่ยนสถานะ งานสำเร็จ

การแยก RepairAssignment และ RepairAssignmentStatusHistory ออกจาก RepairRequest และ RepairRequestStatusHistory ทำให้ระบบสามารถติดตามสถานะ ของคำร้องและสถานะของงานที่มอบหมายให้ช่างได้อย่างอิสระ โดยช่างมีหน้าที่ปรับปรุง

สถานะของงานใน RepairAssignment ส่วนสถานะของ RepairRequest จะถูกปรับตาม กระบวนการของระบบในขั้นตอนการยืนยันผลการซ่อมภายหลัง

(7) การยืนยันผลการซ่อม (Repair Completion Confirmation)



ภาพที่ 16 Sequence Diagram การยืนยันผลการซ่อม

Sequence Diagram: Repair Completion Confirmation แสดงลำดับการทำงานในขั้นตอนการตรวจสอบผลการปฏิบัติงานของช่างและการยืนยันผลการซ่อมร่วมกับผู้แจ้ง โดยผู้ดูแลระบบเป็นผู้ตรวจสอบสถานะงานของช่าง ติดต่อผู้แจ้งเพื่อยืนยันว่าการซ่อมเสร็จสมบูรณ์หรือไม่ และปรับปรุงสถานะของคำร้องตามผลการยืนยัน พร้อมบันทึกประวัติการเปลี่ยนแปลงสถานะของคำร้อง

ในส่วนของ Check Technician Work Status ผู้ดูแลระบบเลือกตรวจสอบสถานะงานของช่างผ่าน Admin Web UI จากนั้นระบบเรียกใช้ RepairAssignmentService เพื่อค้นหา RepairAssignment ที่เกี่ยวข้องกับคำร้อง โดยใช้ RepairAssignmentRepository

และ requestId ระบบจะดึงข้อมูลจากฐานข้อมูลและส่งสถานะของงานกลับไปยัง Web UI เพื่อให้ผู้ดูแลระบบตรวจสอบว่างานของช่างอยู่ในสถานะใด

หลังจากตรวจสอบสถานะงานแล้ว ผู้ดูแลระบบจะติดต่อผู้แจ้งเพื่อสอบถามและยืนยันผลการซ่อม โดยผู้แจ้งสามารถแจ้งผลกลับมายังผู้ดูแลระบบว่า การซ่อมเสร็จสมบูรณ์ หรือ ยังไม่สามารถยืนยันว่าการซ่อมเสร็จสมบูรณ์ได้

หาก Repair is confirmed ผู้ดูแลระบบจะยืนยันการซ่อมผ่าน Admin Web UI และระบบจะส่งคำขอไปยัง RepairRequestService เพื่อเปลี่ยนสถานะของ RepairRequest เป็น COMPLETED จากนั้น Service จะค้นหาคำร้องผ่าน RepairRequestRepository และตรวจสอบว่าการเปลี่ยนสถานะเป็นไปตามเงื่อนไขของระบบหรือไม่ เมื่อผ่านการตรวจสอบแล้ว ระบบจะปรับปรุงสถานะของคำร้องและบันทึกข้อมูลลงในฐานข้อมูล

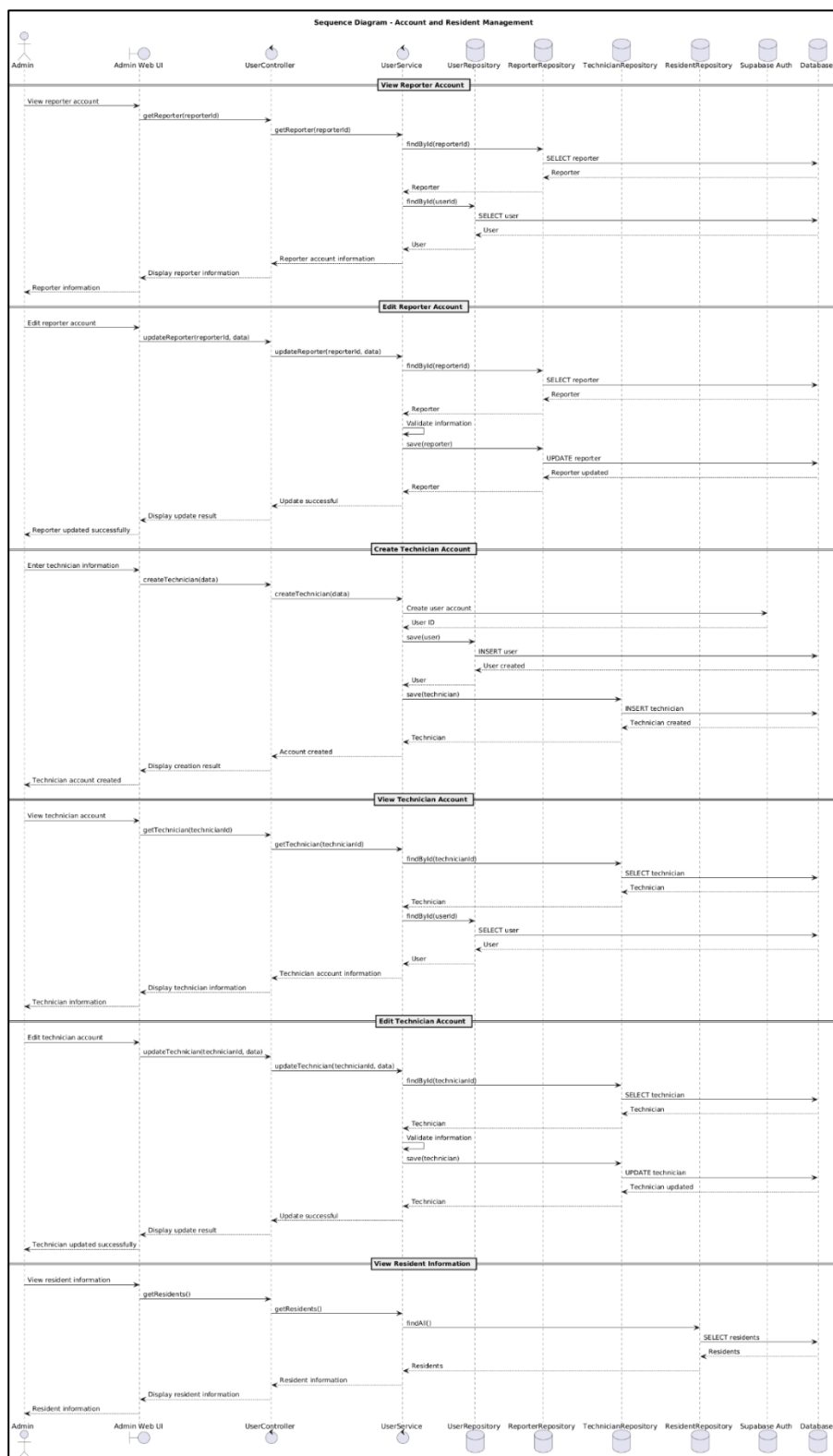
หลังจากเปลี่ยนสถานะ RepairRequest สำเร็จ ระบบจะสร้าง RepairRequestStatusHistory เพื่อบันทึกการเปลี่ยนแปลงสถานะครั้งนี้ โดย RepairRequestStatusHistoryRepository จะทำหน้าที่บันทึกข้อมูลประวัติลงในตาราง repair_request_status_history ทำให้สามารถตรวจสอบย้อนหลังได้ว่าคำร้องถูกเปลี่ยนเป็นสถานะ COMPLETED เมื่อใด และเกิดจากการดำเนินการของผู้ใด

ในกรณีที่ Repair is not confirmed ผู้แจ้งไม่ยืนยันว่าการซ่อมเสร็จสมบูรณ์ ผู้ดูแลระบบจะเลือกดำเนินการผ่าน Admin Web UI และระบบจะส่งคำขอไปยัง RepairRequestService เพื่อเปลี่ยนสถานะของ RepairRequest เป็น IN_COMPLETED จากนั้น Service จะค้นหาคำร้อง ตรวจสอบความถูกต้องของการเปลี่ยนสถานะ และบันทึกสถานะใหม่ลงในฐานข้อมูล

เช่นเดียวกับกรณีการยืนยันผล ระบบจะสร้าง RepairRequestStatusHistory หลังจากเปลี่ยนสถานะสำเร็จ เพื่อบันทึกประวัติ IN_COMPLETED ไว้ในระบบ ทำให้สามารถตรวจสอบลำดับการดำเนินงานของคำร้องได้อย่างต่อเนื่อง

Sequence Diagram นี้แสดงให้เห็นถึงการแยกความรับผิดชอบระหว่าง RepairAssignment และ RepairRequest โดย RepairAssignment ใช้สำหรับติดตามสถานะการปฏิบัติงานของช่าง ส่วน RepairRequest จะถูกเปลี่ยนเป็น COMPLETED หรือ IN_COMPLETED ตามผลการยืนยันจากผู้แจ้ง ดังนั้นช่างจะไม่ได้เป็นผู้เปลี่ยนสถานะของ RepairRequest เป็น COMPLETED โดยตรง แต่สถานะดังกล่าวจะเกิดขึ้นภายหลังจากกระบวนการยืนยันผลการซ่อม

(8) การจัดการข้อมูลบัญชีและผู้พักอาศัย



ภาพที่ 17 Sequence Diagram การจัดการข้อมูลบัญชีและผู้พักอาศัย

Sequence Diagram: Account and Resident Management แสดงลำดับการทำงานของ ผู้ดูแลระบบในการจัดการข้อมูลบัญชีผู้ใช้งานและข้อมูลผู้พักอาศัยภายในระบบ โดยครอบคลุมการ ดูและแก้ไขข้อมูลบัญชีของผู้แจ้ง การสร้างและจัดการบัญชีของช่างประจำหอพัก และการดูข้อมูลผู้ พักอาศัย

ในส่วนของ View Reporter Account ผู้ดูแลระบบเลือกดูข้อมูลบัญชีของผู้แจ้งผ่าน Admin Web UI จากนั้น Web UI จะส่งคำขอไปยัง UserController และ UserService ตามลำดับ โดย Service จะค้นหาข้อมูลผู้แจ้งผ่าน ReporterRepository และค้นหาข้อมูลบัญชีผู้ใช้งานที่เกี่ยวข้อง ผ่าน UserRepository จากนั้นนำข้อมูลที่ได้มาประกอบกันและส่งกลับไปยัง Web UI เพื่อแสดง ข้อมูลบัญชีของผู้แจ้งให้ผู้ดูแลระบบตรวจสอบ

สำหรับ Edit Reporter Account ผู้ดูแลระบบเลือกแก้ไขข้อมูลบัญชีของผู้แจ้ง ระบบจะค้นหา ข้อมูลผู้แจ้งผ่าน ReporterRepository และตรวจสอบความถูกต้องของข้อมูลก่อนดำเนินการแก้ไข เมื่อข้อมูลผ่านการตรวจสอบ UserService จะบันทึกข้อมูลที่แก้ไขผ่าน Repository และปรับปรุง ข้อมูลในฐานข้อมูล จากนั้นระบบจะแจ้งผลการแก้ไขให้ผู้ดูแลระบบทราบ

ในส่วนของ Create Technician Account ผู้ดูแลระบบเป็นผู้สร้างบัญชีสำหรับช่างประจำ หอพัก โดยกรอกข้อมูลของช่างผ่าน Admin Web UI จากนั้นข้อมูลจะถูกส่งไปยัง UserController และ UserService ระบบจะสร้างบัญชีผู้ใช้งานผ่าน Supabase Authentication และได้รับ User ID กลับมา จากนั้น Service จะบันทึกข้อมูลบัญชีผู้ใช้ผ่าน UserRepository และบันทึกข้อมูล เฉพาะของช่างผ่าน TechnicianRepository เพื่อเชื่อมโยงบัญชีผู้ใช้กับข้อมูลช่างในระบบ เมื่อ ดำเนินการสำเร็จ ระบบจะแจ้งผลการสร้างบัญชีให้ผู้ดูแลระบบทราบ

สำหรับ View Technician Account ผู้ดูแลระบบสามารถเรียกดูข้อมูลบัญชีของช่างได้ โดย UserService จะค้นหาข้อมูลช่างผ่าน TechnicianRepository และค้นหาข้อมูลบัญชีผู้ใช้งานที่ เกี่ยวข้องผ่าน UserRepository จากนั้นนำข้อมูลมาประกอบกันและแสดงผลผ่าน Admin Web UI

ในส่วนของ Edit Technician Account ผู้ดูแลระบบสามารถแก้ไขข้อมูลของช่างได้ โดยระบบ จะค้นหาข้อมูลช่างผ่าน TechnicianRepository ตรวจสอบความถูกต้องของข้อมูล และบันทึก ข้อมูลที่แก้ไขกลับเข้าสู่ฐานข้อมูล จากนั้นระบบจะแจ้งผลการแก้ไขให้ผู้ดูแลระบบทราบ

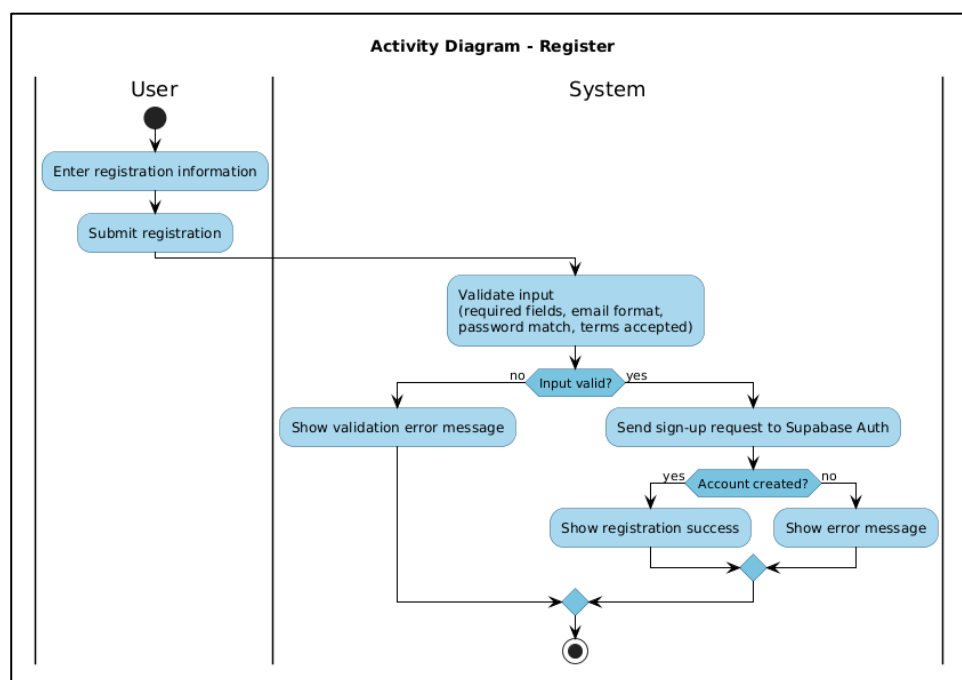
สำหรับ View Resident Information ผู้ดูแลระบบสามารถดูข้อมูลผู้พักอาศัยในหอพักได้ โดย Web UI จะส่งคำขอไปยัง UserController และ UserService จากนั้น Service เรียกใช้

ResidentRepository เพื่อค้นหาข้อมูลผู้พักอาศัยทั้งหมดจากฐานข้อมูล เมื่อได้รับข้อมูลแล้วระบบจะส่งข้อมูลกลับไปยัง Web UI เพื่อแสดงรายการผู้พักอาศัยให้ผู้ดูแลระบบตรวจสอบ

Sequence Diagram นี้แสดงให้เห็นการแบ่งความรับผิดชอบของแต่ละส่วนอย่างชัดเจน โดย Controller ทำหน้าที่รับคำขอจาก Web UI, Service ทำหน้าที่ประมวลผล Business Logic, Repository ทำหน้าที่ติดต่อกับฐานข้อมูล และ Supabase Authentication ทำหน้าที่จัดการการสร้างบัญชีผู้ใช้งานภายนอกระบบฐานข้อมูลหลัก การแบ่งส่วนดังกล่าวช่วยให้โครงสร้างระบบมีความเป็นระเบียบและสอดคล้องกับแนวคิด Separation of Concerns และ Layered Architecture

3.2.7. Activity Diagram

(1) Register



ภาพที่ 18 Activity Diagram – Register

Activity Diagram: Register แสดงลำดับกิจกรรมของกระบวนการสมัครบัญชีผู้ใช้งานของระบบ แจ้งข้อมูลภายในห้องพัก โดยแบ่งความรับผิดชอบออกเป็น 2 ส่วน (Swimlane) ได้แก่ User ซึ่งเป็นผู้สมัครใช้งาน และ System ซึ่งเป็นส่วนประมวลผลของระบบ

กระบวนการเริ่มต้นที่ฝั่ง User โดยผู้สมัครกรอกข้อมูลสำหรับการสมัครบัญชี (Enter registration information) ได้แก่ ชื่อ นามสกุล ชื่อผู้ใช้ อีเมล เบอร์โทรศัพท์ หมายเลขห้อง

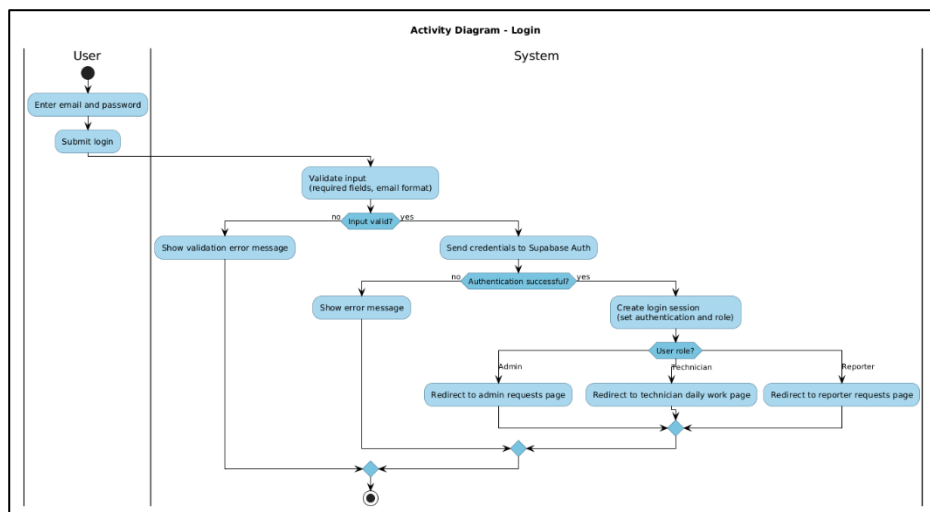
รหัสผ่าน และยืนยันรหัสผ่าน พร้อมยอมรับเงื่อนไขการใช้งาน จากนั้นกดยืนยันการสมัคร (Submit registration)

เมื่อระบบได้รับข้อมูล ฝั่ง System จะตรวจสอบความถูกต้องของข้อมูลที่กรอก (Validate input) ได้แก่ การกรอกข้อมูลที่จำเป็นครบถ้วน รูปแบบอีเมลที่ถูกต้อง รหัสผ่านและการยืนยันรหัสผ่านตรงกัน และการยอมรับเงื่อนไขการใช้งาน แล้วเข้าสู่จุดตัดสินใจ "Input valid?" ซึ่งแบ่งออกเป็น 2 ทาง

- (1.1) ข้อมูลไม่ถูกต้อง (no): ระบบแสดงข้อความแจ้งข้อผิดพลาดของการตรวจสอบข้อมูล (Show validation error message) และไม่ดำเนินการสร้างบัญชี ผู้สมัครต้องแก้ไขข้อมูลแล้วสมัครใหม่
- (1.2) ข้อมูลถูกต้อง (yes): ระบบส่งคำขอสร้างบัญชีไปยัง Supabase Authentication (Send sign-up request to Supabase Auth) แล้วเข้าสู่จุดตัดสินใจ "Account created?"
 - สร้างบัญชีสำเร็จ (yes): ระบบแสดงผลว่าการสมัครสำเร็จ (Show registration success)
 - สร้างบัญชีไม่สำเร็จ (no): ระบบแสดงข้อความแจ้งข้อผิดพลาดที่ได้รับจากบริการยืนยันตัวตน (Show error message)

ทุกเส้นทางมาบรรจบกันที่ Merge Node ก่อนเข้าสู่จุดสิ้นสุดของกระบวนการ (Activity Final) แผนภาพนี้แสดงให้เห็นว่าการสมัครบัญชีมีการตรวจสอบข้อมูลสองระดับ คือ ระดับระบบที่ตรวจสอบความถูกต้องของข้อมูลก่อน และระดับบริการยืนยันตัวตนภายนอก (Supabase Authentication) ที่เป็นผู้สร้างและจัดเก็บบัญชีผู้ใช้

(2) Login



ภาพที่ 19 Activity Diagram: Login

Activity Diagram: Login แสดงลำดับกิจกรรมของกระบวนการเข้าสู่ระบบ โดยแบ่งเป็น 2 ส่วน ได้แก่ User ซึ่งเป็นผู้ใช้งานทั้งสามบทบาท (Reporter, Admin, Technician) และ System ซึ่งเป็นส่วนประมวลผลของระบบ

กระบวนการเริ่มต้นที่ User กรอกอีเมลและรหัสผ่าน (Enter email and password) แล้วกดเข้าสู่ระบบ (Submit login) ฝั่ง System ตรวจสอบความถูกต้องของข้อมูลที่กรอก (Validate input) ได้แก่ การกรอกข้อมูลครบและรูปแบบอีเมล แล้วเข้าสู่จุดตัดสินใจ "Input valid?"

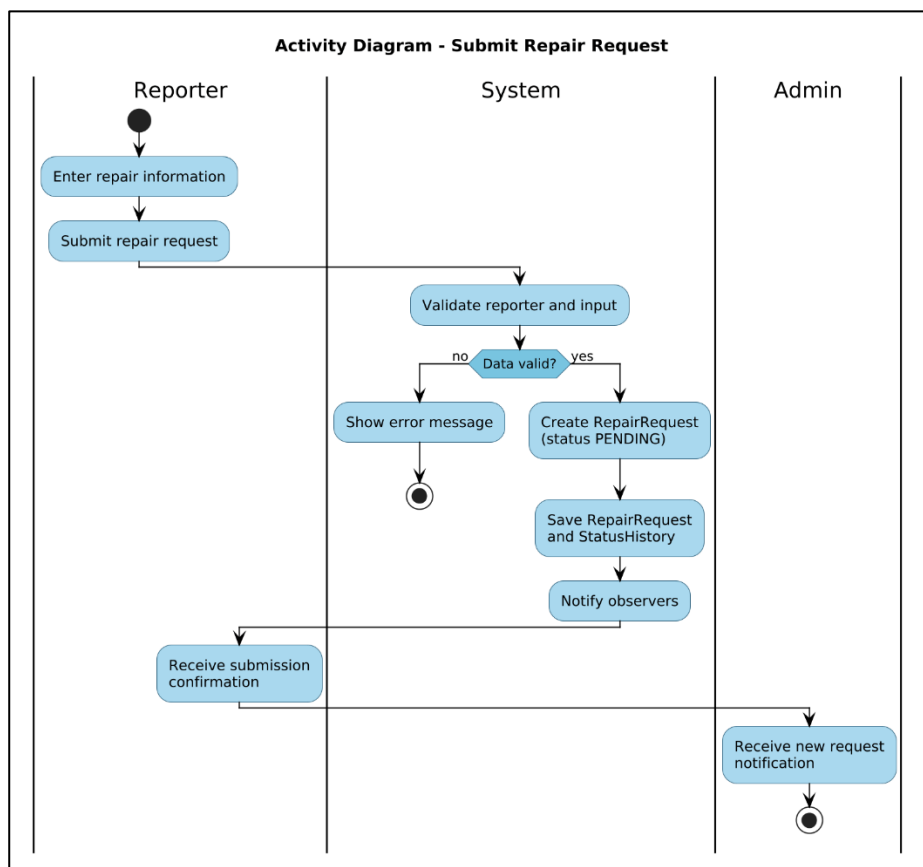
(2.1) ข้อมูลไม่ถูกต้อง (no): ระบบแสดงข้อความแจ้งข้อผิดพลาดและสิ้นสุดกระบวนการ

(2.2) ข้อมูลถูกต้อง (yes): ระบบส่งอีเมลและรหัสผ่านไปยัง Supabase Authentication เพื่อยืนยันตัวตน แล้วเข้าสู่จุดตัดสินใจ "Authentication successful?"

- **ไม่สำเร็จ (no):** ระบบแสดงข้อความแจ้งข้อผิดพลาด
- **สำเร็จ (yes):** ระบบสร้างเซสชันการเข้าสู่ระบบและกำหนดสิทธิ์ตามบทบาท แล้วนำผู้ไปยังหน้าหลักของบทบาท ได้แก่ Admin ไปหน้าจัดการคำร้อง Technician ไปหน้างานประจำวัน และ Reporter ไปหน้าประวัติคำร้องของตนเอง

แผนภาพนี้แสดงให้เห็นว่าการยืนยันตัวตนทำโดยบริการภายนอก (Supabase Authentication) ส่วนระบบรับผิดชอบกำหนดสิทธิ์และนำทางตามบทบาทของผู้ใช้หลังเข้าสู่ระบบสำเร็จ

(3) Submit Repair Request



ภาพที่ 20 Activity Diagram: Submit Repair Request

Activity Diagram: Submit Repair Request แสดงลำดับกิจกรรมของกระบวนการส่งคำร้องแจ้งซ่อม โดยแบ่งความรับผิดชอบเป็น 3 ส่วน (Swimlane) ได้แก่ Reporter ซึ่งเป็นผู้แจ้งซ่อม System ซึ่งเป็นส่วนประมวลผลของระบบ และ Admin ซึ่งเป็นผู้ดูแลระบบ

กระบวนการเริ่มต้นที่ Reporter กรอกข้อมูลการแจ้งซ่อม (Enter repair information) ได้แก่ ประเภทงานซ่อม รายละเอียดปัญหา หมายเหตุ วันที่และช่วงเวลาที่จะตรวจให้เข้าดำเนินการ แล้วกดส่งคำร้อง (Submit repair request)

เมื่อระบบได้รับคำร้อง ฟังก์ชัน System ตรวจสอบข้อมูลผู้แจ้งและข้อมูลที่กรอก (Validate reporter and input) ได้แก่ การมีข้อมูลผู้แจ้งและห้องพัก การเลือกประเภทงาน การกรอกรายละเอียด และการระบุวันที่ถูกต้อง แล้วเข้าสู่จุดตัดสินใจ "Data valid?" ซึ่งแบ่งเป็น 2 ทาง

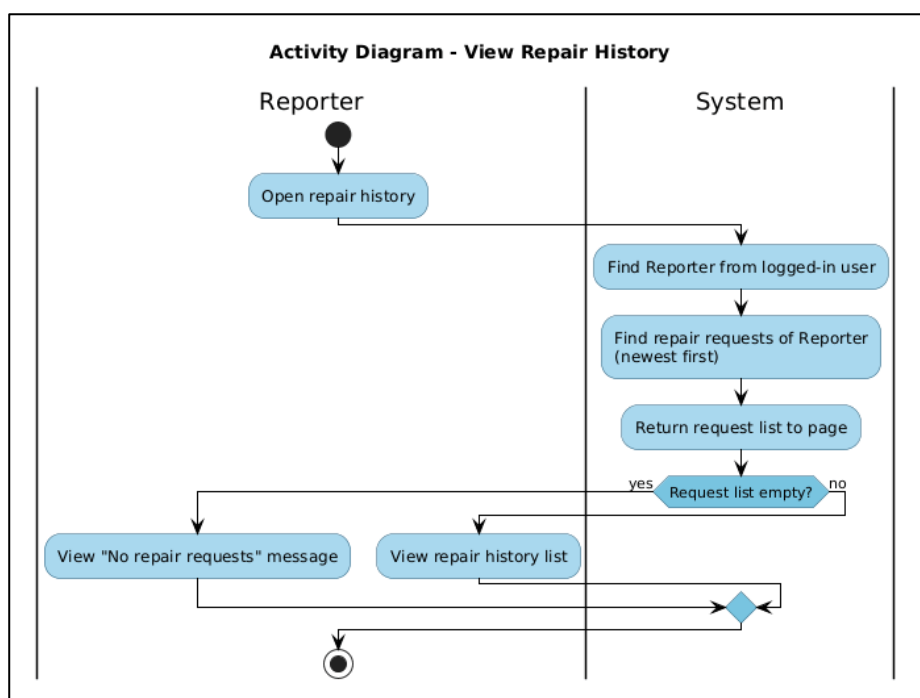
(3.1) ข้อมูลไม่ถูกต้อง (no): ระบบแสดงข้อความแจ้งข้อผิดพลาด (Show error message) และสิ้นสุดกระบวนการโดยไม่สร้างคำร้อง

(3.2) ข้อมูลถูกต้อง (yes): ระบบสร้างคำร้องแจ้งซ่อมโดยกำหนดสถานะเริ่มต้นเป็น PENDING (Create RepairRequest) บันทึกคำร้องและประวัติสถานะแรกลงฐานข้อมูล (Save RepairRequest and StatusHistory) แล้วส่งเหตุการณ์ให้ Observer เพื่อแจ้งเตือน (Notify observers)

หลังจากนั้น Reporter ได้รับการยืนยันการส่งคำร้อง (Receive submission confirmation) และ Admin ได้รับการแจ้งเตือนว่ามีคำร้องใหม่ (Receive new request notification) กระบวนการจึงสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าระบบบันทึกประวัติสถานะตั้งแต่ขั้นตอนสร้างคำร้อง เพื่อให้ตรวจสอบย้อนหลังได้ และแจ้งผู้ดูแลระบบทันทีที่มีคำร้องใหม่ ซึ่งเริ่มต้นที่สถานะ PENDING เพื่อรอ Admin พิจารณาต่อไป

(4) View Repair History



ภาพที่ 21 Activity Diagram: View Repair History

Activity Diagram: View Repair History แสดงลำดับกิจกรรมของกระบวนการดูประวัติคำร้องแจ้งซ่อมของผู้แจ้ง โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Reporter ซึ่งเป็นผู้แจ้งซ่อม และ System ซึ่งเป็นส่วนประมวลผลของระบบ

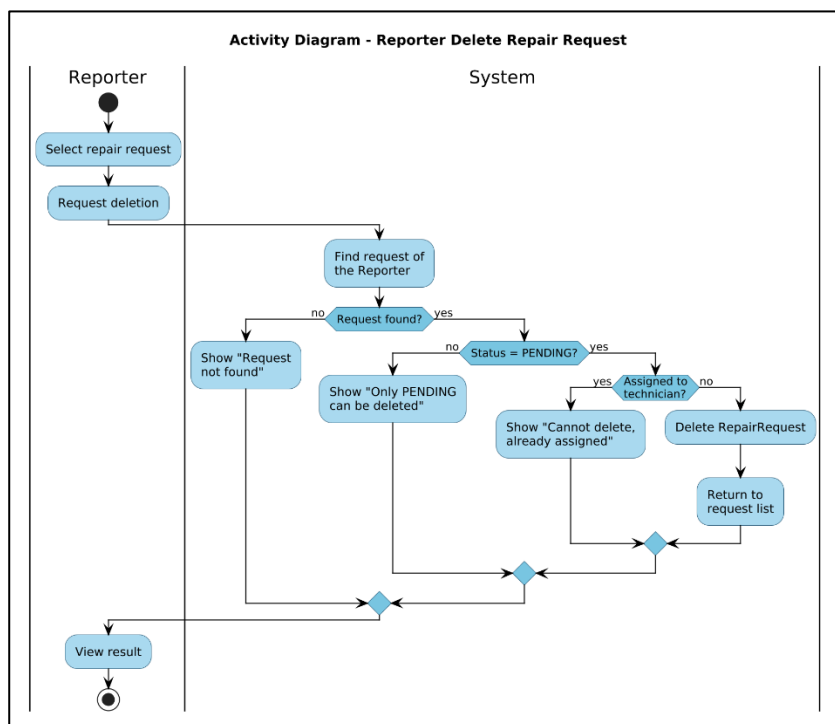
กระบวนการเริ่มต้นที่ Reporter เปิดหน้าประวัติการแจ้งซ่อม (Open repair history) ฝั่ง System ระบุตัวผู้แจ้งจากบัญชีที่เข้าสู่ระบบ (Find Reporter from logged-in user) ค้นหาคำร้องทั้งหมดของผู้แจ้งรายนั้นโดยเรียงจากรายการล่าสุด (Find repair requests of Reporter) และส่งรายการคำร้องกลับไปแสดงที่หน้าเว็บ (Return request list to page) จากนั้นเข้าสู่จุดตัดสินใจ "Request list empty?" ซึ่งแบ่งเป็น 2 ทาง

(4.1) รายการว่าง (yes): หน้าเว็บแสดงข้อความแจ้งว่ายังไม่มีคำร้องแจ้งซ่อม

(4.2) มีรายการ (no): หน้าเว็บแสดงรายการประวัติคำร้องของผู้แจ้งให้ดู

หลังจากนั้นกระบวนการสิ้นสุด (Activity Final) แผนภาพนี้แสดงให้เห็นว่าผู้แจ้งดูได้เฉพาะคำร้องของตนเอง เรียงจากใหม่ไปเก่า และระบบจัดการกรณีที่ยังไม่เคยแจ้งซ่อมด้วยข้อความที่เหมาะสมแทนการแสดงผลหน้าว่าง

(5) Reporter Delete Repair Request



ภาพที่ 22 Activity Diagram: Reporter Delete Repair Request

Activity Diagram: Reporter Delete Repair Request แสดงลำดับกิจกรรมของกระบวนการที่ผู้แจ้งลบคำร้องแจ้งซ่อมของตนเอง โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Reporter ซึ่งเป็นผู้แจ้งซ่อม และ System ซึ่งเป็นส่วนประมวลผลของระบบ

กระบวนการเริ่มต้นที่ Reporter เลือกคำร้องแจ้งซ่อมที่ต้องการ (Select repair request) แล้วสั่งลบคำร้อง (Request deletion) ฝั่ง System ค้นหาคำร้องจากรหัสคำร้องโดยจำกัดเฉพาะคำร้องที่เป็นของผู้แจ้งที่เข้าสู่ระบบ แล้วตรวจสอบเงื่อนไขตามลำดับ ดังนี้

(5.1) ไม่พบคำร้อง (Request found? = no): ระบบแจ้งว่าไม่พบคำร้อง ซึ่งรวมถึงกรณี คำร้องเป็นของผู้แจ้งรายอื่น

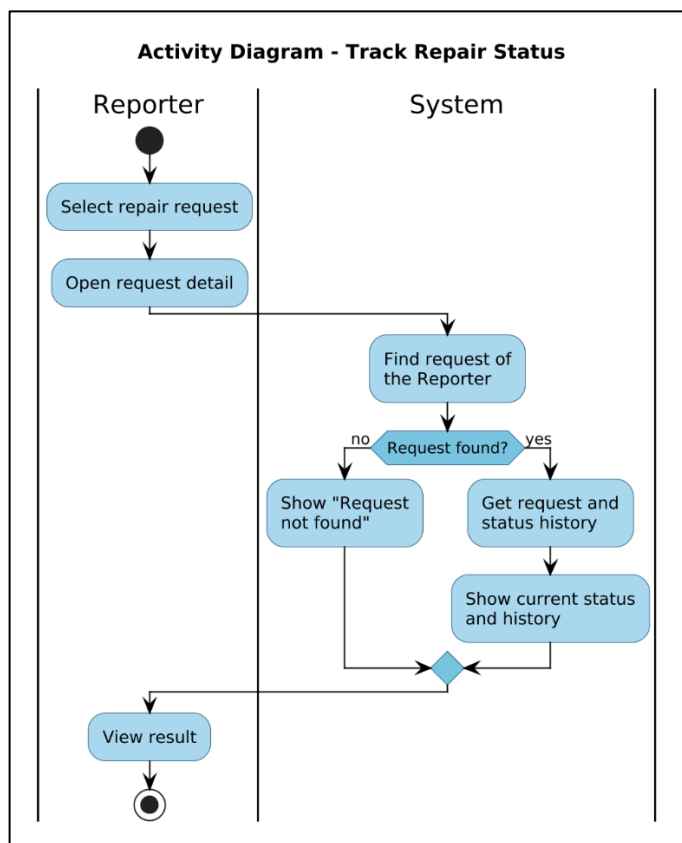
(5.2) สถานะไม่ใช่ PENDING (Status = PENDING? = no): ระบบแจ้งว่าลบได้เฉพาะ คำร้องที่ยังอยู่ในสถานะ PENDING เนื่องจากหลังจาก Admin ดำเนินการแล้วคำร้อง ต้องคงอยู่เพื่อใช้ติดตามงาน

(5.3) คำร้องถูกมอบหมายให้ช่างแล้ว (Already assigned to technician? = yes): ระบบแจ้งว่าไม่สามารถลบได้

(5.4) ผ่านทุกเงื่อนไข: ระบบลบคำร้อง (Delete RepairRequest) แล้วนำผู้แจ้งกลับไป ยังหน้ารายการคำร้อง

แผนภาพนี้แสดงให้เห็นว่าผู้แจ้งลบคำร้องได้เฉพาะคำร้องของตนเองที่ยังอยู่ในสถานะ PENDING และยังไม่มีมอบหมายงาน ซึ่งสอดคล้องกับ State Diagram ที่กำหนดว่าการลบทำได้เฉพาะสถานะ PENDING เท่านั้น

(6) Track Repair Status



ภาพที่ 23 Activity Diagram: Track Repair Status

Activity Diagram: Track Repair Status แสดงลำดับกิจกรรมของกระบวนการติดตามสถานะคำร้องแจ้งซ่อมของผู้แจ้ง โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Reporter ซึ่งเป็นผู้แจ้งซ่อม และ System ซึ่งเป็นส่วนประมวลผลของระบบ

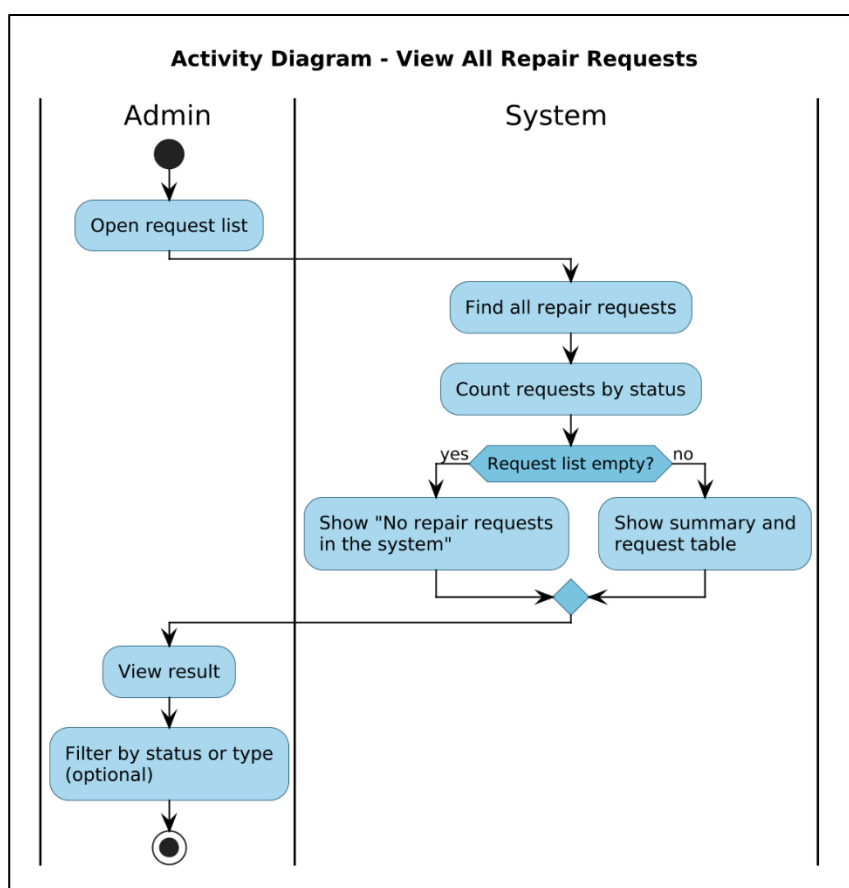
กระบวนการเริ่มต้นที่ Reporter เลือกคำร้องแจ้งซ่อมที่ต้องการติดตาม (Select repair request) และเปิดหน้ารายละเอียดคำร้อง (Open request detail) ฝั่ง System ค้นหาคำร้องจากรหัสคำร้อง โดยจำกัดเฉพาะคำร้องที่เป็นของผู้แจ้งที่เข้าสู่ระบบ แล้วเข้าสู่จุดตัดสินใจ "Request found?"

(6.1) ไม่พบคำร้อง (no): ระบบแจ้งว่าไม่พบคำร้อง ซึ่งรวมถึงกรณีที่คำร้องเป็นของผู้แจ้งรายอื่น

(6.2) พบคำร้อง (yes): ระบบดึงข้อมูลคำร้องและประวัติการเปลี่ยนสถานะ (Get request and status history) แล้วแสดงสถานะปัจจุบันของคำร้องพร้อมประวัติการเปลี่ยนสถานะที่ผ่านมา (Show current status and history)

จากนั้น Reporter ดูผลลัพธ์ที่แสดง (View result) และกระบวนการสิ้นสุด แผนภาพนี้แสดงให้เห็นว่าผู้แจ้งติดตามได้เฉพาะคำร้องของตนเอง และสามารถเห็นลำดับการเปลี่ยนสถานะของคำร้องตั้งแต่ถูกส่งจนถึงปัจจุบัน ซึ่งสอดคล้องกับการบันทึกประวัติสถานะ (RepairRequestStatusHistory) ที่ระบบสร้างทุกครั้งที่สถานะเปลี่ยน

(7) View All Repair Requests



ภาพที่ 24 Activity Diagram: View All Repair Requests

Activity Diagram: View All Repair Requests แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบดูรายการคำร้องแจ้งซ่อมทั้งหมด โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ และ System ซึ่งเป็นส่วนประมวลผลของระบบ

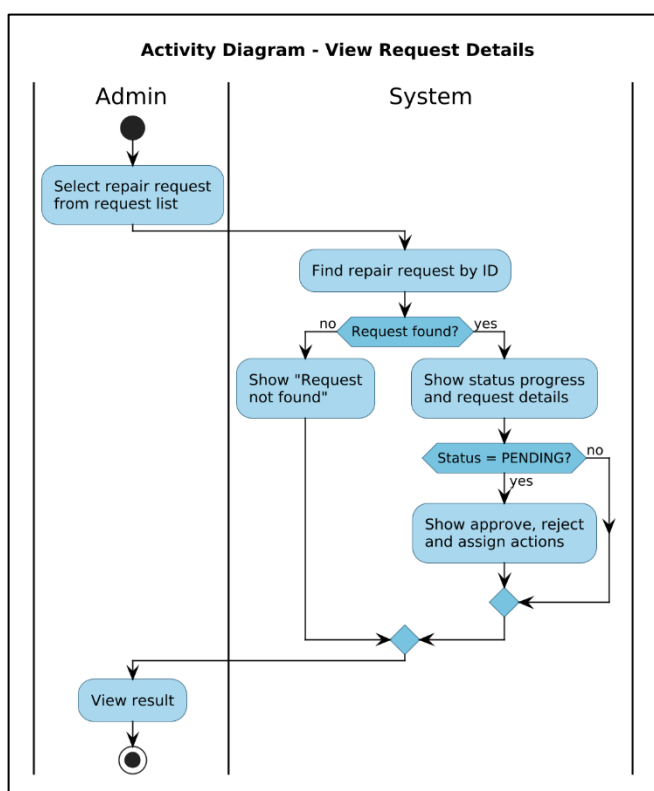
กระบวนการเริ่มต้นที่ Admin เปิดหน้ารายการคำร้อง (Open request list) ฝั่ง System ดึงคำร้องแจ้งซ่อมทั้งหมดในระบบ (Find all repair requests) และนับจำนวนคำร้องแยกตามสถานะ (Count requests by status) ได้แก่ PENDING, APPROVED, REJECTED, IN_PROGRESS, COMPLETED และ IN_COMPLETED เพื่อใช้แสดงเป็นการ์ดสรุป จากนั้นเข้าสู่จุดตัดสินใจ "Request list empty?"

(7.1) ไม่มีคำร้องในระบบ (yes): หน้าเว็บแสดงข้อความแจ้งว่ายังไม่มีคำร้องแจ้งซ่อมในระบบ

(7.2) มีคำร้อง (no): หน้าเว็บแสดงการ์ดสรุปจำนวนตามสถานะและตารางรายการคำร้อง

Admin ดูผลลัพธ์ที่แสดง (View result) และสามารถกรองรายการตามสถานะหรือประเภทงานได้ตามต้องการ (Filter by status or type) ซึ่งเป็นการกรองที่หน้าเว็บโดยไม่ต้องเรียกระบบซ้ำ แผนภาพนี้แสดงให้เห็นว่าผู้ดูแลระบบเห็นภาพรวมของคำร้องทุกรายการและจำนวนในแต่ละสถานะได้จากหน้าเดียว เพื่อตรวจสอบและดำเนินการต่อ เช่น อนุมัติ ปฏิเสธ หรือมอบหมายงานให้ช่าง

(8) View Request Details



ภาพที่ 25 Activity Diagram: View Request Details

Activity Diagram: View Request Details แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบเรียกดูรายละเอียดคำร้องแจ้งซ่อม โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ และ System ซึ่งเป็นส่วนประมวลผลของระบบ

กระบวนการเริ่มต้นที่ Admin เลือกคำร้องแจ้งซ่อมจากรายการคำร้อง (Select repair request from request list) ฝั่ง System ค้นหาคำร้องจากรหัสคำร้องที่เลือก (Find repair request by ID) แล้วเข้าสู่จุดตัดสินใจ “Request found?” ซึ่งแบ่งเป็น 2 ทาง

(8.1) ไม่พบคำร้อง (no): ระบบแสดงข้อความว่าไม่พบคำร้อง (Show “Request not found”)

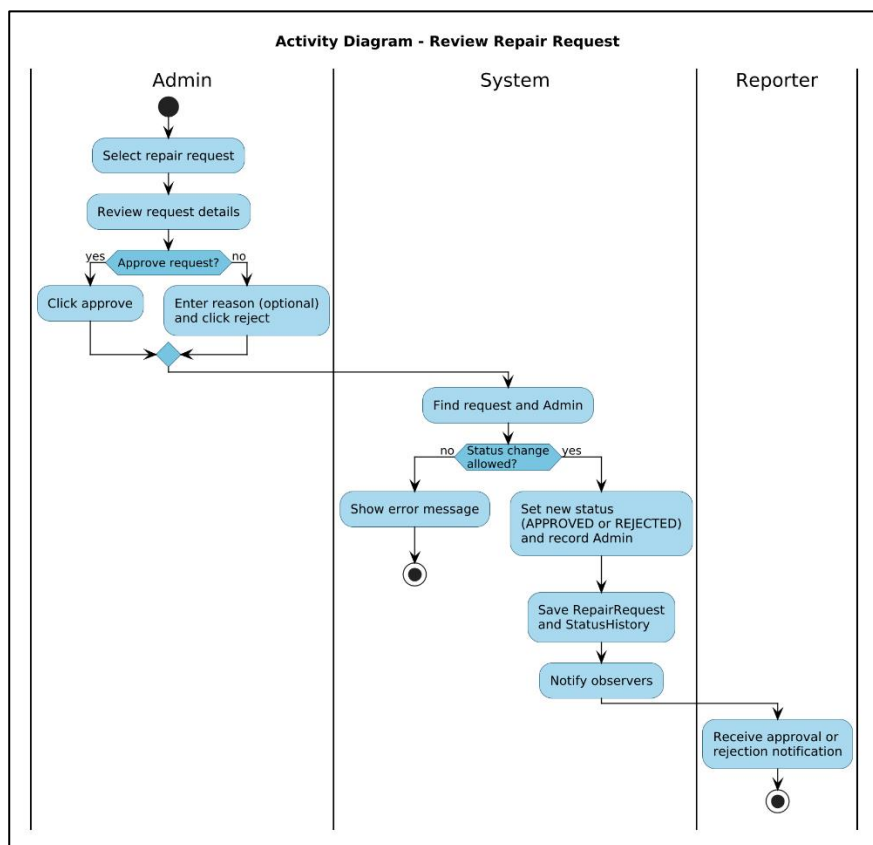
(8.2) พบคำร้อง (yes): ระบบแสดงความคืบหน้าของสถานะและรายละเอียดคำร้อง (Show status progress and request details) แล้วเข้าสู่จุดตัดสินใจ “Status = PENDING?”

- o สถานะเป็น PENDING (yes): ระบบแสดงปุ่มอนุมัติและปฏิเสธคำร้อง (Show approve and reject actions)
- o สถานะไม่ใช่ PENDING (no): ระบบแสดงเฉพาะรายละเอียดคำร้อง โดยไม่แสดงปุ่มดำเนินการ

ทุกเส้นทางมาบรรจบกันที่ Merge Node จากนั้น Admin ดูผลลัพธ์ที่แสดง (View result) และกระบวนการสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าผู้ดูแลระบบสามารถพิจารณาอนุมัติหรือปฏิเสธได้เฉพาะคำร้องที่อยู่ในสถานะ PENDING ซึ่งสอดคล้องกับกฎของ State Pattern ส่วนการมอบหมายงานให้ช่างทำได้หลังจากคำร้องได้รับการอนุมัติแล้วเท่านั้น

(9) Review Repair Request



ภาพที่ 26 Activity Diagram: Review Repair Request

Activity Diagram: Review Repair Request แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบตรวจสอบและพิจารณาคำร้องแจ้งซ่อม โดยแบ่งความรับผิดชอบเป็น 3 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ System ซึ่งเป็นส่วนประมวลผลของระบบ และ Reporter ซึ่งเป็นผู้แจ้งซ่อม

กระบวนการเริ่มต้นที่ Admin เลือกคำร้องแจ้งซ่อม (Select repair request) และตรวจสอบรายละเอียดของคำร้อง (Review request details) แล้วตัดสินใจ "Approve request?" ซึ่งแบ่งเป็น 2 ทาง

(9.1) อนุมัติ (yes): Admin กดอนุมัติคำร้อง (Click approve)

(9.2) ปฏิเสธ (no): Admin ระบุเหตุผลการปฏิเสธ (ถ้ามี) แล้วกดปฏิเสธคำร้อง (Enter reason (optional) and click reject)

ทั้งสองทางมาบรรจบกันที่ Merge Node แล้วส่งคำสั่งเข้าสู่ระบบ ฝั่ง System ค้นหาคำร้องและข้อมูลผู้ดูแลระบบที่ทำรายการ (Find request and Admin) จากนั้นตรวจสอบว่า

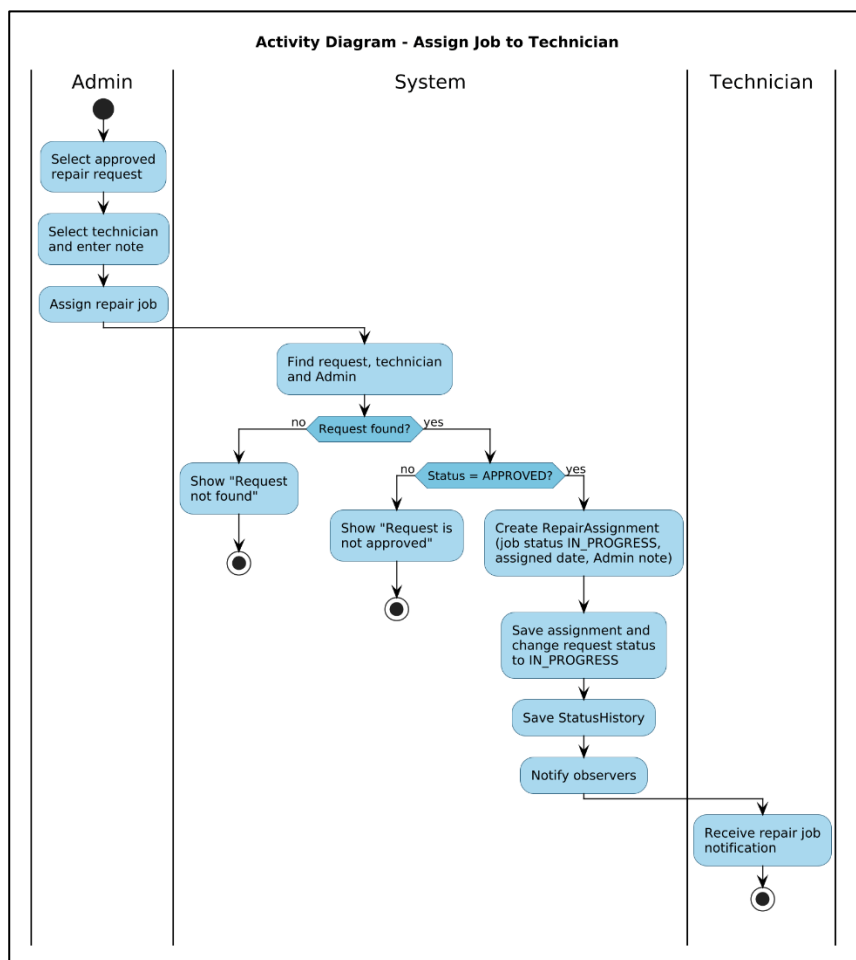
สถานะปัจจุบันเปลี่ยนเป็นสถานะใหม่ได้หรือไม่ตามกฎหมายของ State Pattern (Status change allowed?) ซึ่งอนุญาตให้อนุมัติหรือปฏิเสธได้เฉพาะคำร้องที่อยู่ในสถานะ PENDING

- เปลี่ยนสถานะไม่ได้ (no): ระบบแสดงข้อความแจ้งข้อผิดพลาด (Show error message) และสิ้นสุดกระบวนการโดยไม่เปลี่ยนแปลงข้อมูล
- เปลี่ยนสถานะได้ (yes): ระบบกำหนดสถานะใหม่เป็น APPROVED หรือ REJECTED พร้อมบันทึกผู้ดูแลระบบที่ทำรายการ (Set new status and record Admin) บันทึกคำร้องและประวัติการเปลี่ยนสถานะ (Save RepairRequest and StatusHistory) แล้วส่งเหตุการณ์ให้ Observer เพื่อแจ้งเตือน (Notify observers)

จากนั้น Reporter ได้รับการแจ้งเตือนผลการพิจารณาว่าคำร้องได้รับการอนุมัติหรือถูกปฏิเสธ (Receive approval or rejection notification) กระบวนการจึงสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าการอนุมัติและการปฏิเสธใช้ลำดับการประมวลผลเดียวกันต่างกันเพียงสถานะที่กำหนด โดยทุกการเปลี่ยนสถานะต้องผ่านการตรวจสอบตามกฎหมายของ State Pattern ถูกบันทึกประวัติพร้อมผู้ดำเนินการเพื่อการตรวจสอบย้อนหลัง และผู้แจ้งได้รับแจ้งผลทันที

(10) Assign Job to Technician



ภาพที่ 27 Activity Diagram: Assign Job to Technician

Activity Diagram: Assign Job to Technician แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบมอบหมายงานซ่อมให้ช่าง โดยแบ่งความรับผิดชอบเป็น 3 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ System ซึ่งเป็นส่วนประมวลผลของระบบ และ Technician ซึ่งเป็นช่างประจำหอพัก

กระบวนการเริ่มต้นที่ Admin เลือกคำร้องที่ได้รับการอนุมัติแล้ว (Select approved repair request) เลือกช่างและระบุหมายเหตุถึงช่าง (Select technician and enter note) แล้วกดมอบหมายงาน (Assign repair job) ฝั่ง System ค้นหาคำร้อง ช่าง และผู้ดูแลระบบที่ทำรายการ แล้วตรวจสอบเงื่อนไขตามลำดับ

(10.1) ไม่พบคำร้อง (Request found? = no): ระบบแจ้งว่าไม่พบคำร้องและสิ้นสุดกระบวนการ

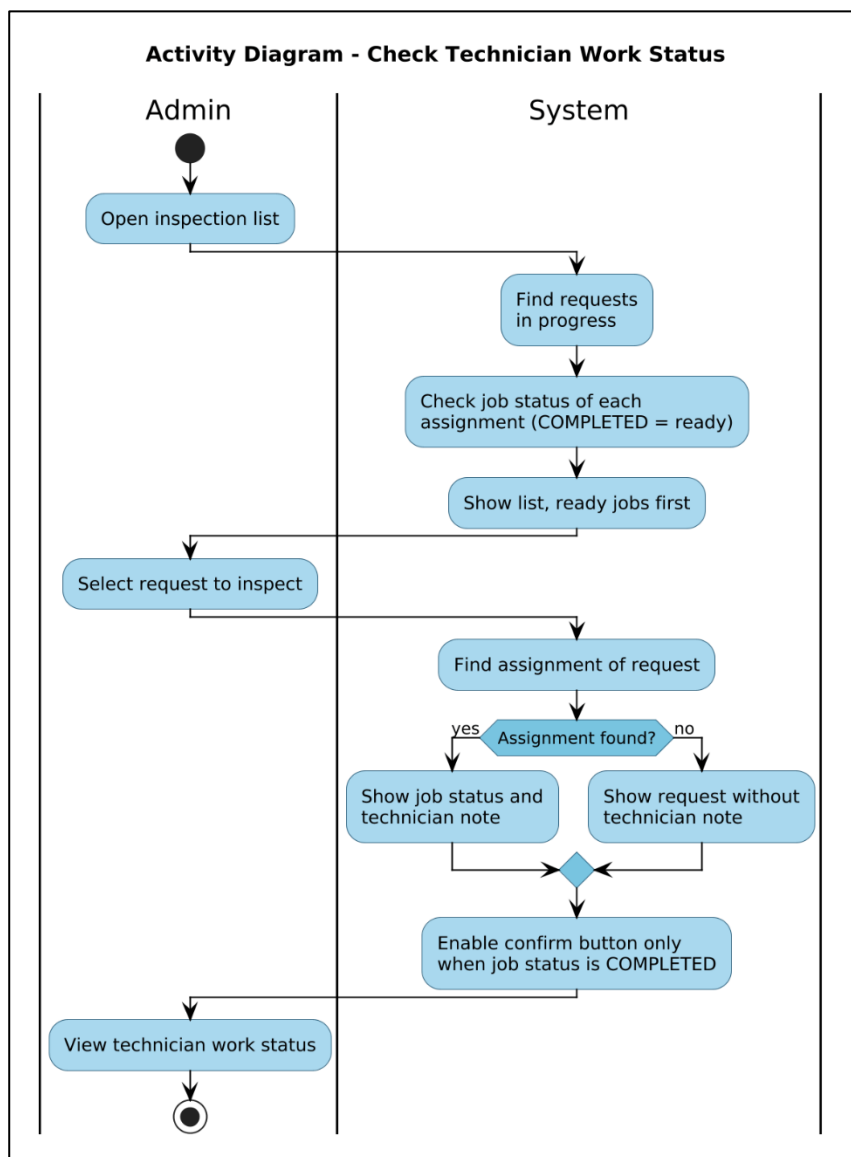
(10.2) คำร้องยังไม่อยู่ในสถานะ APPROVED (Status = APPROVED? = no): ระบบแจ้งว่าคำร้องยังไม่ได้รับการอนุมัติ และไม่บันทึกข้อมูลใด ๆ ซึ่งป้องกันการมอบหมายซ้ำเมื่อคำร้องอยู่ระหว่างดำเนินการแล้ว

(10.3) ผ่านทุกเงื่อนไข: ระบบสร้างรายการมอบหมายงาน (Create RepairAssignment) โดยกำหนดสถานะงานเป็น IN_PROGRESS วันที่มอบหมายและหมายเหตุของ Admin บันทึกรายการมอบหมายพร้อมเปลี่ยนสถานะคำร้องจาก APPROVED เป็น IN_PROGRESS (Save assignment and change request status) บันทึกประวัติการเปลี่ยนสถานะ (Save StatusHistory) แล้วส่งเหตุการณ์ให้ Observer (Notify observers)

จากนั้น Technician ได้รับการแจ้งเตือนว่ามีงานซ่อมใหม่ (Receive repair job notification) กระบวนการจึงสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าการมอบหมายงานทำได้เฉพาะคำร้องที่อนุมัติแล้ว และการสร้างรายการมอบหมายกับการเปลี่ยนสถานะคำร้องเกิดขึ้นพร้อมกันในธุรกรรมเดียว หากขั้นตอนใดล้มเหลวจะไม่มีข้อมูลถูกบันทึก

(11) Check Technician Work Status



ภาพที่ 28 Activity Diagram: Check Technician Work Status

Activity Diagram: Check Technician Work Status แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบตรวจสอบสถานะการทำงานของช่าง โดยแบ่งความรับผิดชอบเป็น 2 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ และ System ซึ่งเป็นส่วนประมวลผลของระบบ

กระบวนการเริ่มต้นที่ Admin เปิดหน้ารายการตรวจงาน (Open inspection list) ฝั่ง System ดึงคำร้องที่อยู่ในสถานะ IN_PROGRESS (Find requests in progress) ตรวจสอบสถานะงานของรายการมอบหมายแต่ละรายการ โดยถือว่างานที่ช่างอัปเดตเป็น COMPLETED พร้อมให้ตรวจสอบ (Check job status of each assignment) แล้วแสดงรายการโดยเรียงงานที่พร้อมตรวจขึ้นก่อน (Show list, ready jobs first)

จากนั้น Admin เลือกคำร้องที่ต้องการตรวจสอบ (Select request to inspect) ฝั่ง System ค้นหาการมอบหมายของคำร้องนั้น (Find assignment of request) แล้วเข้าสู่จุดตัดสินใจ "Assignment found?"

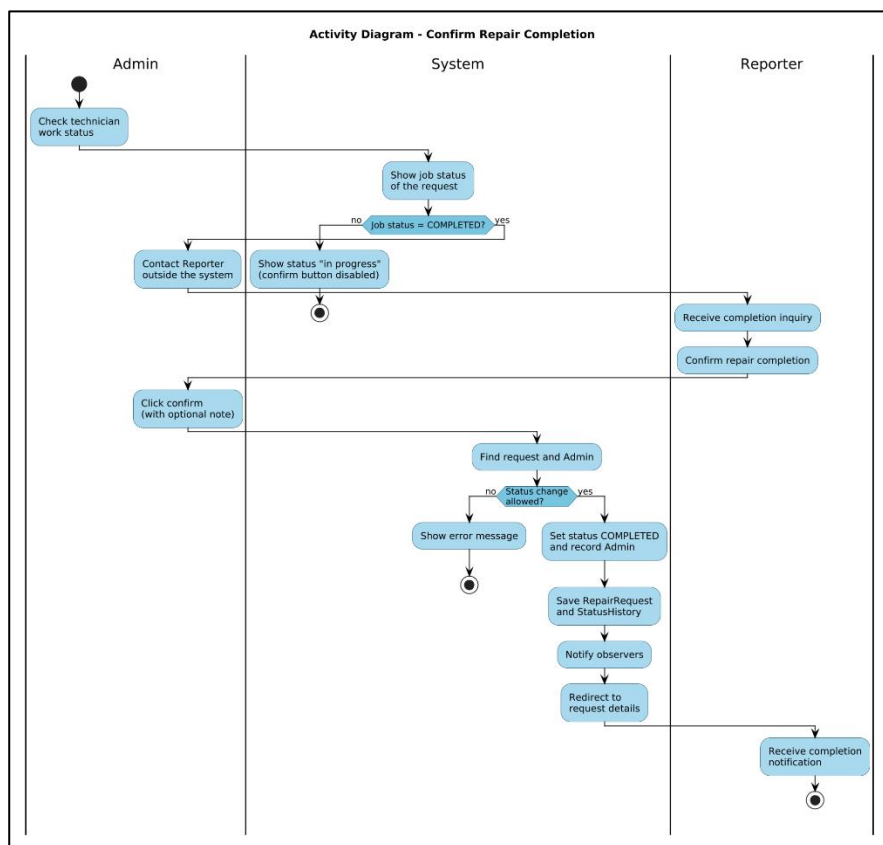
(11.1) พบรายการมอบหมาย (yes): ระบบแสดงสถานะงานและหมายเหตุจากช่าง (Show job status and technician note)

(11.2) ไม่พบรายการมอบหมาย (no): ระบบแสดงข้อมูลคำร้องโดยไม่มีหมายเหตุของช่าง (Show request without technician note)

ทั้งสองทางมาบรรจบกัน ระบบเปิดใช้งานปุ่มยืนยันผลเฉพาะเมื่อสถานะงานเป็น COMPLETED เท่านั้น (Enable confirm button only when job status is COMPLETED) แล้ว Admin ดูสถานะการทำงานของช่าง (View technician work status) กระบวนการจึงสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าผู้ดูแลระบบติดตามความคืบหน้าของงานที่อยู่ระหว่างดำเนินการได้จากหน้าเดียว และระบบป้องกันการยืนยันงานก่อนที่ช่างจะรายงานว่าซ่อมเสร็จ

(12) Confirm Repair Completion



ภาพที่ 29 Activity Diagram: Confirm Repair Completion

Activity Diagram: Confirm Repair Completion แสดงลำดับกิจกรรมของกระบวนการที่ผู้ดูแลระบบยืนยันว่าการซ่อมเสร็จสมบูรณ์ โดยแบ่งความรับผิดชอบเป็น 3 ส่วน (Swimlane) ได้แก่ Admin ซึ่งเป็นผู้ดูแลระบบ System ซึ่งเป็นส่วนประมวลผลของระบบ และ Reporter ซึ่งเป็นผู้แจ้งซ่อม

กระบวนการเริ่มต้นที่ Admin ตรวจสอบสถานะการทำงานของช่าง (Check technician work status) ฝั่ง System แสดงสถานะงานของคำร้อง (Show job status of the request) แล้วเข้าสู่จุดตัดสินใจ "Job status = COMPLETED?"

(12.1) ช่างยังไม่รายงานว่ายี่เสร็จ (no): ระบบแสดงสถานะ "in progress" และปิดปุ่มยืนยัน ผู้ดูแลระบบยังยืนยันผลไม่ได้ และกระบวนการสิ้นสุด

(12.2) ช่างรายงานว่ายี่เสร็จแล้ว (yes): Admin ติดต่อผู้แจ้งนอกระบบเพื่อสอบถามผลการซ่อม (Contact Reporter outside the system) ผู้แจ้งได้รับการสอบถามและยืนยันว่าการซ่อมเสร็จสมบูรณ์ (Receive completion inquiry, Confirm

repair completion) จากนั้น Admin กดยืนยันในระบบพร้อมหมายเหตุ (ถ้ามี)
(Click confirm)

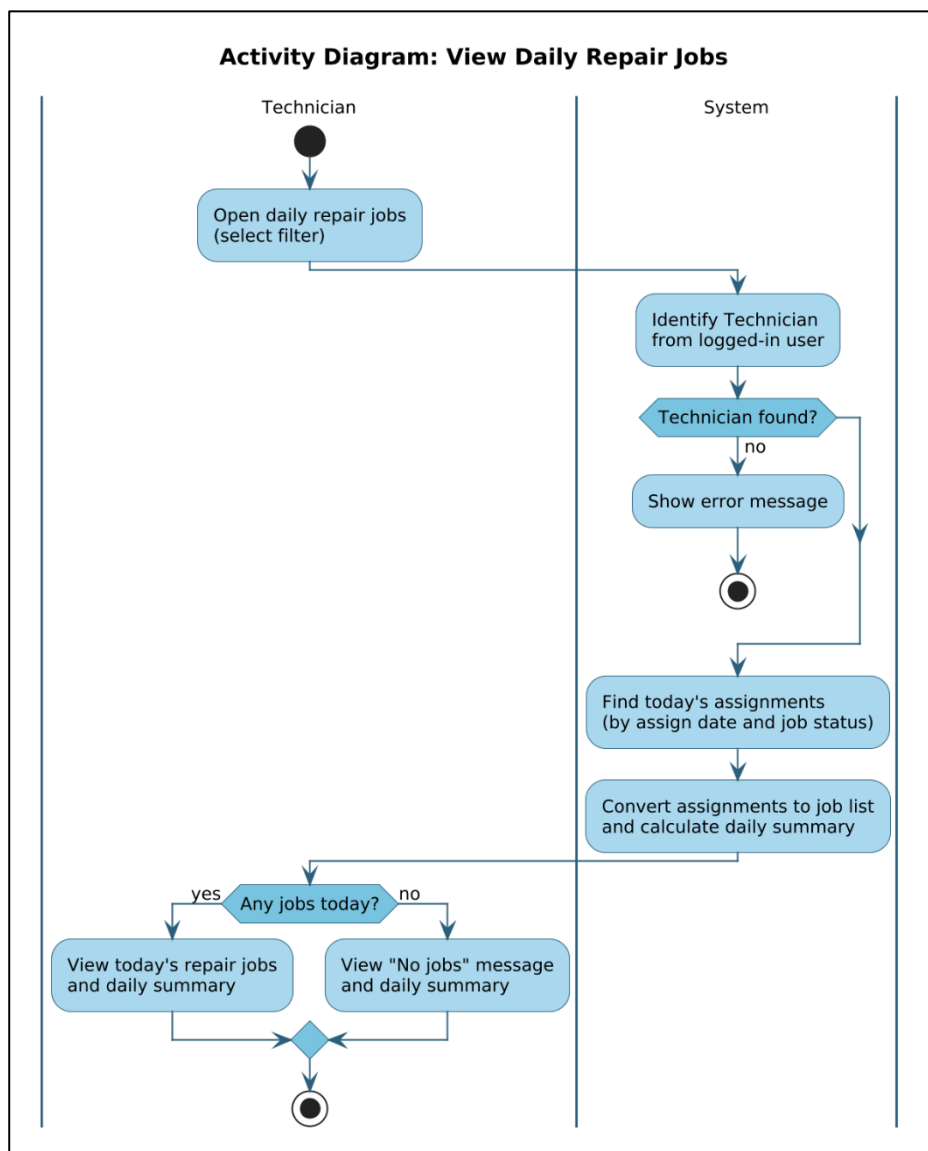
ฝั่ง System ค้นหาคำร้องและข้อมูลผู้ดูแลระบบ แล้วตรวจสอบว่าสถานะ IN_PROGRESS เปลี่ยนเป็น COMPLETED ได้หรือไม่ตามกฎของ State Pattern (Status change allowed?)

- เปลี่ยนไม่ได้ (no): ระบบแสดงข้อความแจ้งข้อผิดพลาด
- เปลี่ยนได้ (yes): ระบบกำหนดสถานะคำร้องเป็น COMPLETED พร้อมบันทึกผู้ดูแลระบบ บันทึกคำร้องและประวัติสถานะ ส่งเหตุการณ์ให้ Observer แล้วนำ Admin กลับไปยังหน้ารายละเอียดคำร้อง

จากนั้น Reporter ได้รับการแจ้งเตือนว่าคำร้องเสร็จสมบูรณ์ (Receive completion notification) กระบวนการจึงสิ้นสุด

แผนภาพนี้แสดงให้เห็นว่าการปิดงานต้องผ่านสองฝ่ายตามลำดับ คือช่างรายงานว่าซ่อมเสร็จ และผู้ดูแลระบบยืนยันหลังตรวจสอบกับผู้แจ้ง โดยสถานะของคำร้องเปลี่ยนเป็น COMPLETED ได้ต่อเมื่อ Admin ยืนยันเท่านั้น

(13) View Daily Repair Jobs



ภาพที่ 30 Activity Diagram: View Daily Repair Jobs

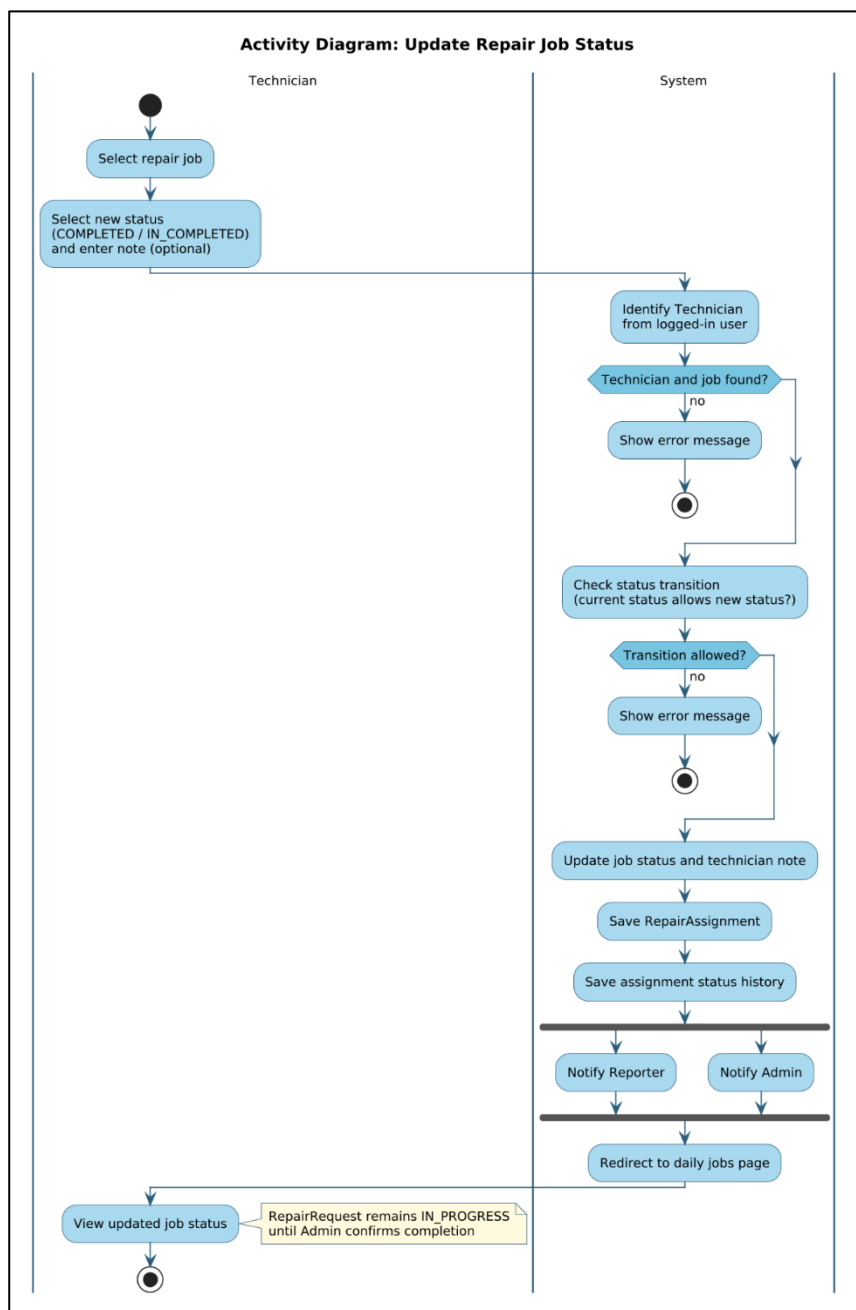
Activity Diagram: View Daily Repair Jobs แสดงขั้นตอนที่ช่างซ่อมเปิดดูรายการงานซ่อมประจำวันของตนเอง โดยมี 2 ส่วนรับผิดชอบ คือ Technician และ System

เริ่มจากช่างเปิดหน้างานซ่อมประจำวัน และสามารถเลือกตัวกรองสถานะงานได้ ได้แก่ ทั้งหมด กำลังดำเนินการ ซ่อมเสร็จ และซ่อมไม่เสร็จ โดยค่าเริ่มต้นคือทั้งหมด ระบบระบุตัวช่างจากผู้ใช้ที่เข้าสู่ระบบและตรวจสอบว่ามีข้อมูลช่างอยู่จริง หากไม่พบ ระบบแสดงข้อความผิดพลาดและสิ้นสุดการทำงาน

เมื่อพบข้อมูลช่าง ระบบค้นหางานที่ได้รับมอบหมายให้ช่างคนนั้นโดยใช้เวลาที่มอบหมายงานอยู่ในช่วงของวันนี้ ร่วมกับสถานะงานตามตัวกรอง แล้วแปลงข้อมูลเป็นรายการงาน ซึ่งประกอบด้วยเวลา รายละเอียดปัญหา อาคาร ห้อง เบอร์โทรผู้แจ้ง และหมายเหตุ พร้อมคำนวณสรุปจำนวนงานประจำวัน

จากนั้นหน้าจอของช่างจะแสดงผลตามเงื่อนไข หากมีงานในวันนี้ ช่างจะเห็นรายการงานและสรุปประจำวัน หากไม่มีงาน ช่างจะเห็นข้อความว่าไม่มีงานในวันนี้พร้อมสรุปประจำวัน แล้วสิ้นสุดการทำงาน

(14) Update Repair Job Status



ภาพที่ 31 Activity Diagram: Update Repair Job Status

Activity Diagram: Update Repair Job Status แสดงขั้นตอนที่ช่างซ่อมอัปเดตสถานะงานซ่อมที่ได้รับมอบหมาย โดยมี 2 ส่วนรับผิดชอบ คือ Technician และ System เริ่มจากช่างเลือกงานซ่อมจากรายการงานประจำวัน แล้วเลือกสถานะใหม่ของงาน ได้แก่ ช่อมเสร็จ (COMPLETED) หรือซ่อมไม่เสร็จ (IN_COMPLETED) พร้อมกรอกหมายเหตุ

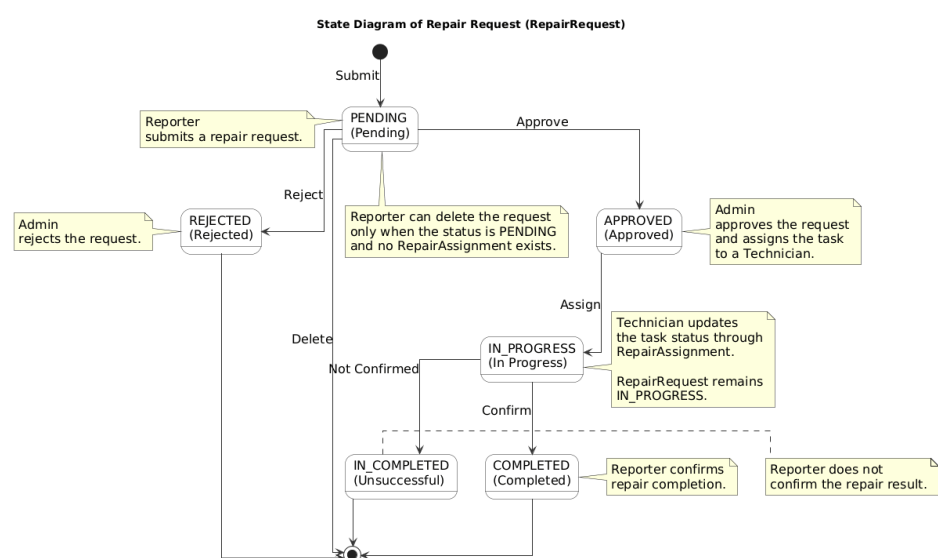
จากช่าง (ถ้ามี) ระบบระบุตัวช่างจากผู้ที่ใช้เข้าสู่ระบบ แล้วตรวจสอบว่าพบข้อมูลช่างและงานที่เลือกหรือไม่ หากไม่พบ ระบบแสดงข้อความผิดพลาดและสิ้นสุดการทำงาน

จากนั้นระบบตรวจสอบว่าสถานะปัจจุบันของงานอนุญาตให้เปลี่ยนเป็นสถานะใหม่หรือไม่ ตามกฎของ State Pattern ซึ่งงานที่กำลังดำเนินการ (IN_PROGRESS) เปลี่ยนเป็น COMPLETED หรือ IN_COMPLETED ได้ แต่งานที่เป็น COMPLETED หรือ IN_COMPLETED แล้วจะเปลี่ยนต่อไม่ได้ หากไม่อนุญาต ระบบแสดงข้อความผิดพลาดและสิ้นสุดการทำงาน

หากอนุญาต ระบบอัปเดตสถานะงานและหมายเหตุของช่าง บันทึกข้อมูลการมอบหมายงาน และบันทึกประวัติการเปลี่ยนสถานะของงาน จากนั้นระบบส่งการแจ้งเตือนไปยังผู้แจ้งซ่อมและผู้ดูแลระบบพร้อมกันผ่าน Observer Pattern แล้วพาช่างกลับไปยังหน้างานประจำวัน ช่างจึงเห็นสถานะงานที่อัปเดตแล้ว

3.2.8. State Diagram

การออกแบบระบบจัดการแจ้งซ่อมในหอพัก ระบบจำเป็นต้องกำหนดสถานะของคำร้องแจ้งซ่อม (Repair Request) และควบคุมการเปลี่ยนแปลงสถานะให้เป็นไปตามลำดับและเงื่อนไขที่ระบบกำหนด เพื่อป้องกันการเปลี่ยนสถานะที่ไม่ถูกต้อง และทำให้สามารถติดตามความคืบหน้าของคำร้องได้อย่างชัดเจน ดังนั้นจึงออกแบบ State Diagram สำหรับคลาส RepairRequest เพื่อแสดงสถานะต่าง ๆ ของคำร้อง รวมถึงเหตุการณ์หรือการดำเนินการที่ทำให้คำร้องเปลี่ยนจากสถานะหนึ่งไปยังอีกสถานะหนึ่ง



ภาพที่ 32 State Diagram

State Diagram ของ RepairRequest ประกอบด้วยสถานะหลัก 6 สถานะ ได้แก่ PENDING, APPROVED, REJECTED, IN_PROGRESS, COMPLETED และ IN_COMPLETED โดยมีรายละเอียดดังนี้

(1) PENDING (รอดำเนินการ)

เป็นสถานะเริ่มต้นของคำร้อง หลังจากผู้แจ้งส่งคำร้องแจ้งซ่อมเข้าสู่ระบบ คำร้องจะอยู่ในสถานะ PENDING เพื่อรอการตรวจสอบและพิจารณาจากผู้ดูแลระบบในระหว่างที่อยู่ในสถานะนี้ผู้แจ้งสามารถลบคำร้องได้ โดยเงื่อนไขสำคัญคือคำร้องต้องยังไม่มี RepairAssignment ซึ่งหมายความว่ายังไม่ได้ถูกมอบหมายงานให้แก่ช่าง

(2) APPROVED (อนุมัติ)

เมื่อผู้ดูแลระบบตรวจสอบคำร้องและอนุมัติคำร้อง ระบบจะเปลี่ยนสถานะจาก PENDING เป็น APPROVED จากนั้นผู้ดูแลระบบสามารถดำเนินการมอบหมายงานให้แก่ช่างเพื่อดำเนินการซ่อมต่อไป

(3) REJECTED (ปฏิเสธ)

หากผู้ดูแลระบบตรวจสอบแล้วเห็นว่าคำร้องไม่สามารถดำเนินการได้หรือไม่ผ่านเงื่อนไขที่กำหนดผู้ดูแลระบบสามารถปฏิเสธคำร้องได้ โดยสถานะจะเปลี่ยนจาก PENDING เป็น REJECTED และถือเป็นสถานะสิ้นสุดของคำร้องดังกล่าว

(4) IN_PROGRESS (กำลังดำเนินการ)

เมื่อผู้ดูแลระบบมอบหมายงานให้แก่ช่างคำร้องที่อยู่ในสถานะ APPROVED จะเปลี่ยนเป็น IN_PROGRESS เพื่อแสดงว่าคำร้องกำลังอยู่ในกระบวนการดำเนินงานซ่อม โดยในขั้นตอนนี้ช่างจะดำเนินการและอัปเดตสถานะของงานผ่าน RepairAssignment อย่างไรก็ตาม สถานะของ RepairRequest ยังคงเป็น IN_PROGRESS จนกว่าผู้แจ้งจะยืนยันผลการซ่อม

(5) COMPLETED (ดำเนินการสำเร็จ)

เมื่อช่างดำเนินการซ่อมเสร็จแล้วผู้ดูแลระบบจะติดต่อผู้แจ้งเพื่อจะตรวจสอบผลการซ่อมและยืนยันว่าการซ่อมเสร็จสมบูรณ์ หากผู้แจ้งยืนยันผลการซ่อม ผู้ดูแลระบบจะยืนยันสถานะคำร้อง โดยระบบจะเปลี่ยนสถานะของ RepairRequest จาก IN_PROGRESS เป็น COMPLETED ซึ่งหมายถึงคำร้องได้รับการดำเนินการเสร็จสิ้น และเป็นสถานะสิ้นสุดของคำร้อง

(6) IN_COMPLETED (ดำเนินการไม่สำเร็จ)

หากปัญหายังไม่ได้รับการแก้ไขอย่างสมบูรณ์ ระบบสามารถเปลี่ยนสถานะจาก IN_PROGRESS เป็น IN_COMPLETED เพื่อแสดงว่าการดำเนินการซ่อมไม่สำเร็จตามผลที่คาดหวัง โดยสถานะดังกล่าวเป็นสถานะสิ้นสุดของคำร้องในกระบวนการที่ออกแบบไว้

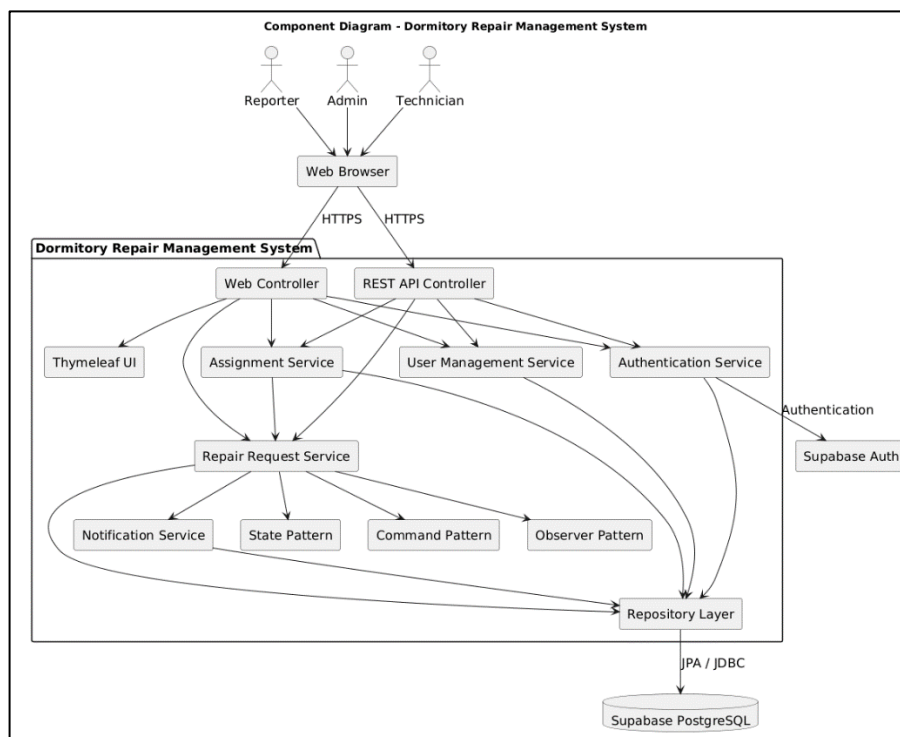
ลำดับการเปลี่ยนสถานะ

ลำดับการทำงานเริ่มจากสถานะเริ่มต้น เมื่อผู้แจ้งส่งคำร้อง ระบบจะกำหนดสถานะเป็น PENDING จากนั้นผู้ดูแลระบบจะตรวจสอบและเลือกดำเนินการได้สองแนวทาง คือ อนุมัติคำร้องเพื่อเปลี่ยนเป็น APPROVED หรือปฏิเสธคำร้องเพื่อเปลี่ยนเป็น REJECTED หากคำร้องได้รับการอนุมัติและผู้ดูแลระบบจะมอบหมายงานให้ช่าง ระบบจะเปลี่ยนสถานะเป็น IN_PROGRESS

เมื่ออยู่ในสถานะ IN_PROGRESS ช่างจะดำเนินการซ่อมผ่าน RepairAssignment โดยสถานะของ RepairRequest จะยังคงเป็น IN_PROGRESS จนกว่าผู้ดูแลระบบจะตรวจสอบและยืนยันผลการซ่อมจากผู้แจ้ง หากผู้แจ้งยืนยันว่าซ่อมเสร็จสมบูรณ์ สถานะจะเปลี่ยนเป็น COMPLETED แต่หากซ่อมไม่สมบูรณ์ สถานะจะเปลี่ยนเป็น IN_COMPLETED

สำหรับสถานะ REJECTED, COMPLETED และ IN_COMPLETED ถือเป็นสถานะสิ้นสุดของกระบวนการตาม State Diagram ที่ออกแบบไว้ ขณะที่ PENDING มีเงื่อนไขพิเศษ คือผู้แจ้งสามารถลบคำร้องออกจากระบบได้เฉพาะในช่วงที่คำร้องยังอยู่ในสถานะ PENDING และยังไม่มี การสร้าง RepairAssignment

3.2.9. Component Diagram



ภาพที่ 33 Component Diagram

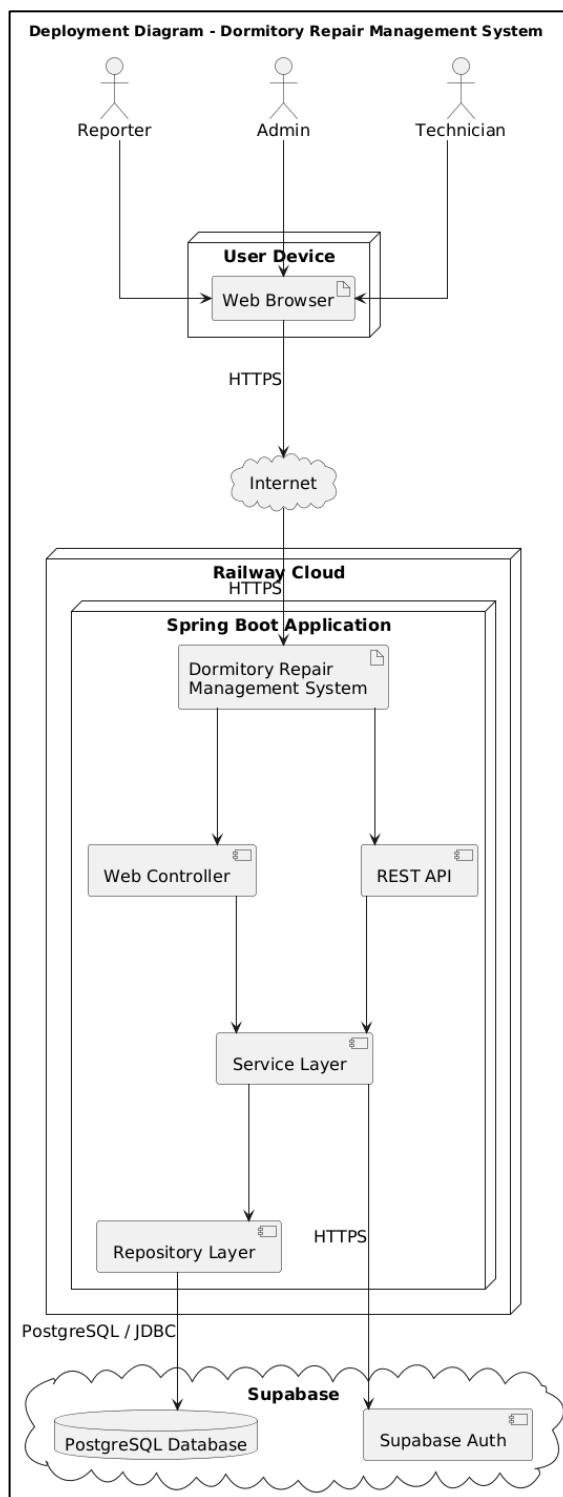
ระบบพัฒนาด้วย Spring Boot และแบ่งส่วนประกอบตามหน้าที่ เพื่อให้แต่ละส่วนมีความรับผิดชอบที่ชัดเจนและสามารถทำงานร่วมกันได้อย่างเป็นระบบ ผู้ใช้งานของระบบประกอบด้วย Reporter, Admin และ Technician ซึ่งเข้าถึงระบบผ่าน Web Browser โดยมีรายละเอียดของส่วนประกอบดังนี้

- (1) Web Controller ทำหน้าที่รับและประมวลผลคำขอจาก Web Browser สำหรับการดำเนินงานของส่วนติดต่อผู้ใช้ เช่น การแสดงหน้าเว็บ การส่งแบบฟอร์ม และการจัดการคำขอของผู้ใช้งาน
- (2) REST API Controller ทำหน้าที่จัดเตรียม API สำหรับรับและส่งข้อมูลระหว่าง Client กับระบบ โดยเรียกใช้ Service Layer เพื่อดำเนินการตาม Business Logic ของระบบ
- (3) Thymeleaf UI ทำหน้าที่เป็นส่วนติดต่อผู้ใช้ของระบบ โดยใช้ Thymeleaf ในการสร้างหน้าเว็บแบบ Dynamic HTML และแสดงข้อมูลที่ได้รับจาก Web Controller

- (4) Authentication Service ทำหน้าที่จัดการกระบวนการยืนยันตัวตนของผู้ใช้งาน และเชื่อมต่อกับ Supabase Auth เพื่อดำเนินการเกี่ยวกับบัญชีผู้ใช้และการเข้าสู่ระบบ
- (5) Repair Request Service เป็นส่วนสำคัญในการจัดการกระบวนการของคำร้องแจ้งซ่อม เช่น การสร้าง แก้ไข ตรวจสอบ อนุมัติ ปฏิเสธ และเปลี่ยนสถานะของคำร้อง โดยทำงานร่วมกับ State Pattern, Command Pattern และ Observer Pattern ตามลักษณะการทำงานของระบบ
- (6) User Management Service ทำหน้าที่จัดการข้อมูลผู้ใช้งานและข้อมูลที่เกี่ยวข้องกับ Reporter, Admin และ Technician รวมถึงการดำเนินการเกี่ยวกับบัญชีผู้ใช้งานตามสิทธิ์ของแต่ละบทบาท
- (7) Assignment Service ทำหน้าที่จัดการการมอบหมายงานซ่อมให้แก่ Technician รวมถึงการจัดการข้อมูลของ RepairAssignment และประสานงานกับ Repair Request Service เพื่อให้สถานะและข้อมูลของคำร้องสอดคล้องกับการดำเนินงาน
- (8) Notification Service ทำหน้าที่จัดการการแจ้งเตือนเมื่อเกิดเหตุการณ์สำคัญในระบบ เช่น การเปลี่ยนแปลงสถานะของคำร้อง โดยทำงานร่วมกับ Observer Pattern เพื่อให้ผู้ที่เกี่ยวข้องได้รับข้อมูลตามเหตุการณ์ที่เกิดขึ้น
- (9) State Pattern ใช้สำหรับจัดการพฤติกรรมของ RepairRequest ตามสถานะปัจจุบัน เช่น PENDING, APPROVED, REJECTED, IN_PROGRESS, COMPLETED และ IN_COMPLETED ทำให้กฎการเปลี่ยนสถานะถูกแยกออกจากตัวคำร้องและสามารถจัดการได้อย่างเป็นระบบ
- (10) Command Pattern ใช้สำหรับแยกคำสั่งหรือการดำเนินการที่มีต่อระบบออกจากผู้ใช้ ทำให้สามารถจัดการคำสั่งต่าง ๆ ที่เกี่ยวข้องกับกระบวนการแจ้งซ่อมได้อย่างเป็นอิสระ และช่วยลดการผูกติดระหว่างส่วนที่เรียกใช้คำสั่งกับส่วนที่ดำเนินการ
- (11) Observer Pattern ใช้สำหรับจัดการการแจ้งเตือนเมื่อเกิดเหตุการณ์สำคัญ โดยเมื่อ RepairRequest มีการเปลี่ยนแปลงสถานะ ระบบสามารถแจ้งไปยัง Observer ที่เกี่ยวข้อง เช่น Reporter, Technician หรือ Admin ตามเงื่อนไขของระบบ
- (12) Repository Layer ทำหน้าที่เป็นชั้นสำหรับติดต่อและจัดการข้อมูลในฐานข้อมูล โดยใช้กลไกของ Spring Data JPA เพื่อดำเนินการเพิ่ม แก้ไข ค้นหา และลบข้อมูล
- (13) Supabase PostgreSQL ทำหน้าที่จัดเก็บข้อมูลหลักของระบบ เช่น ข้อมูลผู้ใช้งาน ข้อมูล Resident, Room, RepairRequest, RepairAssignment และข้อมูลประวัติสถานะของคำร้อง

จากความสัมพันธ์ของ Component ดังกล่าว ระบบมีการแบ่งความรับผิดชอบออกเป็นแต่ละชั้นอย่างชัดเจน โดย Controller ทำหน้าที่รับคำขอ Service ทำหน้าที่ประมวลผล Business Logic และ Repository ทำหน้าที่เข้าถึงข้อมูล ขณะที่ Design Pattern ช่วยจัดการพฤติกรรมเฉพาะของระบบ ทำให้โครงสร้างของระบบมีความเป็นระเบียบ ลดการพึ่งพาระหว่างส่วนประกอบ และง่ายต่อการบำรุงรักษาและพัฒนาต่อในอนาคต

3.2.10. Deployment Diagram



ภาพที่ 34 Deployment Diagram

ระบบถูก Deploy บน Railway Cloud ในรูปแบบ Web Application ผู้ใช้งานสามารถเข้าถึงระบบผ่าน Web Browser โดยระบบมีการเชื่อมต่อกับบริการภายนอก ได้แก่ Supabase Auth สำหรับการยืนยันตัวตน และ Supabase PostgreSQL สำหรับจัดเก็บข้อมูลของระบบ โดยมีรายละเอียดดังนี้

- (1) User Device เป็นอุปกรณ์ของผู้ใช้งาน เช่น คอมพิวเตอร์หรืออุปกรณ์ที่สามารถเปิด Web Browser ได้ โดยผู้ใช้งานหลักประกอบด้วย Reporter, Admin และ Technician
- (2) Web Browser เป็นส่วนที่ผู้ใช้งานใช้ในการเข้าถึงระบบ โดยส่งคำขอไปยังระบบผ่าน Internet ด้วยโปรโตคอล HTTPS เพื่อให้สามารถใช้งานระบบแอดมินและฟังก์ชันต่าง ๆ ตามสิทธิ์ของผู้ใช้งานแต่ละประเภท
- (3) Internet ทำหน้าที่เป็นเครือข่ายสำหรับเชื่อมต่อระหว่างอุปกรณ์ของผู้ใช้งานกับ Application ที่ถูก Deploy บน Railway รวมถึงเป็นช่องทางสำหรับติดต่อกับบริการภายนอกของ Supabase
- (4) Railway Cloud เป็นแพลตฟอร์ม Cloud ที่ใช้สำหรับ Deploy และให้บริการ Spring Boot Application ของระบบ โดย Application สามารถทำงานและรับคำขอจากผู้ใช้งานผ่าน Internet
- (5) Spring Boot Application เป็นส่วนประมวลผลหลักของระบบ ซึ่งประกอบด้วย Web Controller, REST API, Service Layer และ Repository Layer โดย Controller ทำหน้าที่รับคำขอจากผู้ใช้งาน Service Layer ประมวลผล Business Logic และ Repository Layer ติดต่อกับฐานข้อมูล
- (6) Web Controller ทำหน้าที่จัดการคำขอที่มาจาก Web Browser และควบคุมการทำงานของส่วนติดต่อผู้ใช้ เช่น การแสดงหน้าเว็บและการส่งข้อมูลจากแบบฟอร์ม
- (7) REST API ทำหน้าที่รับส่งข้อมูลผ่าน API สำหรับการทำงานที่ต้องการรูปแบบการสื่อสารแบบ REST และเป็นช่องทางให้ Client ติดต่อกับ Business Logic ของระบบ
- (8) Service Layer ทำหน้าที่ประมวลผลกฎและตรรกะทางธุรกิจของระบบ รวมถึงการจัดการกระบวนการแจ้งซ่อม การมอบหมายงาน การจัดการผู้ใช้งาน และการเปลี่ยนแปลงสถานะของคำร้อง
- (9) Repository Layer ทำหน้าที่เชื่อมต่อระหว่าง Service Layer กับฐานข้อมูล โดยใช้ JPA/JDBC ในการจัดการข้อมูลของระบบ

(10) Supabase Auth เป็นบริการภายนอกที่ใช้สำหรับการยืนยันตัวตนและจัดการบัญชีผู้ใช้งาน โดย Spring Boot Application ติดต่อกับ Supabase Auth ผ่าน HTTPS

(11) Supabase PostgreSQL Database เป็นฐานข้อมูลสำหรับจัดเก็บข้อมูลของระบบ โดย Repository Layer ติดต่อกับฐานข้อมูลผ่าน PostgreSQL/JDBC เพื่อดำเนินการเกี่ยวกับข้อมูล

จากสถาปัตยกรรมดังกล่าว ระบบสามารถให้บริการผ่าน Internet โดย Application หลักถูก Deploy บน Railway Cloud ขณะที่บริการด้าน Authentication และ Database ถูกแยกออกไปอยู่บน Supabase ทำให้แต่ละบริการมีหน้าที่เฉพาะและสามารถบริหารจัดการแยกจากกันได้ โครงสร้างนี้ช่วยให้ระบบสามารถนำไปใช้งานจริงบน Cloud และรองรับการเข้าถึงจากผู้ใช้งานตามสิทธิ์ของ Reporter, Admin และ Technician

3.3 พัฒนาระบบ

ดำเนินการพัฒนาระบบตามแบบจำลองที่ออกแบบไว้ โดยใช้ Spring Boot เป็น Framework หลักในการพัฒนา Web Application และ REST API

การพัฒนาระบบแบ่งออกเป็นส่วนสำคัญ ได้แก่

- (1) พัฒนาระบบจัดการบัญชีและการยืนยันตัวตน โดยเชื่อมต่อกับ Supabase Authentication และกำหนดสิทธิ์การใช้งานตามบทบาทของผู้ใช้งาน
- (2) พัฒนาระบบจัดการคำร้องแจ้งซ่อม สำหรับการสร้าง ดู แก้ไข และลบคำร้องตามเงื่อนไขที่กำหนด
- (3) พัฒนาระบบตรวจสอบคำร้อง สำหรับผู้ดูแลระบบ (Admin) เพื่อพิจารณาอนุมัติหรือปฏิเสธคำร้อง
- (4) พัฒนาระบบมอบหมายงาน สำหรับสร้าง RepairAssignment และกำหนด Technician ที่รับผิดชอบ
- (5) พัฒนาระบบดำเนินงานซ่อม สำหรับให้ Technician ตรวจสอบงานและอัปเดตสถานะของงาน
- (6) พัฒนาระบบยืนยันผลการซ่อม สำหรับให้ Admin ดำเนินการยืนยันผลร่วมกับ Reporter และปรับปรุงสถานะของ RepairRequest
- (7) พัฒนาระบบบันทึกประวัติสถานะ สำหรับเก็บ RepairRequestStatusHistory และ RepairAssignmentStatusHistory

- (8) พัฒนาระบบแจ้งเตือน โดยประยุกต์ใช้ Observer Pattern เพื่อแจ้งเหตุการณ์ที่เกี่ยวข้องกับผู้ใช้งาน
 - (9) พัฒนาส่วนติดต่อผู้ใช้ ด้วย Thymeleaf และ HTML/CSS เพื่อให้ผู้ใช้งานแต่ละบทบาทสามารถเข้าถึงฟังก์ชันตามสิทธิ์ของตนเอง
 - (10) พัฒนา REST API เพื่อรองรับการรับส่งข้อมูลระหว่าง Client กับระบบในรูปแบบ JSON
- ในการพัฒนาระบบมีการแบ่งโครงสร้างเป็น Controller, Service, Repository และ Domain เพื่อแยกความรับผิดชอบของแต่ละส่วน และใช้ Constructor Injection ของ Spring ในการจัดการ Dependency

3.4 พัฒนารฐานข้อมูล

พัฒนารฐานข้อมูลตาม ER Diagram และ Data Dictionary ที่ออกแบบไว้ โดยใช้ PostgreSQL เป็นระบบจัดการฐานข้อมูล และใช้งานผ่าน Supabase PostgreSQL

ตารางในระบบถูกออกแบบให้มี Primary Key และ Foreign Key เพื่อรักษาความถูกต้องของความสัมพันธ์ระหว่างข้อมูล รวมถึงกำหนด Constraint และ Index ที่เหมาะสมกับการค้นหาและการเชื่อมโยงข้อมูล

นอกจากนี้มีการกำหนดความสัมพันธ์ระหว่าง Entity ตามลักษณะของข้อมูล เช่น ความสัมพันธ์ระหว่าง User กับข้อมูลบทบาท ความสัมพันธ์ระหว่าง Building กับ Room ความสัมพันธ์ระหว่าง Room กับ Resident และความสัมพันธ์ระหว่าง RepairRequest กับ RepairAssignment และ Status History

การออกแบบยังคำนึงถึงการกำหนด Cascade และ Fetch Type ให้เหมาะสม เพื่อควบคุมการโหลดข้อมูลและการจัดการข้อมูลที่มีความสัมพันธ์กัน รวมถึงป้องกันข้อมูลที่ไม่สามารถอ้างอิงถึงกันได้หลังจากมีการลบข้อมูล

3.5 ทดสอบระบบ

ดำเนินการทดสอบระบบเพื่อค้นหาข้อผิดพลาดและตรวจสอบว่าระบบทำงานตรงตามความต้องการที่กำหนดไว้ โดยแบ่งการทดสอบเป็น 33 ชุดทดสอบ (TS-01 ถึง TS-33) รวม 497 กรณีทดสอบ ครอบคลุม 4 ระดับ ได้แก่ Unit Test 152 กรณี Integration Test 151 กรณี Functional Test 113 กรณี และ User Acceptance Test 81 กรณี โดยสองระดับแรกใช้ JUnit 5 และ Mockito ส่วนสองระดับหลังทดสอบผ่านเบราว์เซอร์ด้วย Selenium WebDriver

การทดสอบครอบคลุมการทำงานของผู้ใช้งานทั้ง 3 บทบาท ได้แก่ ผู้แจ้ง (TS-01 ถึง TS-07) ผู้ดูแลระบบ (TS-08 ถึง TS-27) และช่าง (TS-28 ถึง TS-33) ทั้งกรณีการทำงานปกติ และกรณีที่ไม่เป็นไปตามเงื่อนไขของระบบ เช่น การกรอกข้อมูลไม่ครบถ้วน การเข้าใช้งานโดยไม่ได้เข้าสู่ระบบ หรือการเปลี่ยนสถานะงานที่ระบบไม่อนุญาต ตัวอย่างการทดสอบ ได้แก่

1. การสมัครสมาชิกของ Reporter (TS-01) ทั้งการตรวจสอบความถูกต้องของข้อมูลที่กรอก และกรณีระบุข้อมูลผู้พักอาศัย (Resident) ที่ไม่มีอยู่ในระบบ
2. การสร้างบัญชี Technician โดย Admin (TS-18) รวมถึงกรณีผู้ใช้ซ้ำและกรณีที่ Supabase ตอบข้อผิดพลาด
3. การเข้าสู่ระบบ (TS-02) และการนำผู้ใช้ไปยังหน้าที่เหมาะสมตามบทบาท โดยทดสอบเส้นทางของ Admin (TS-08) รวมถึงกรณีข้อมูลเข้าสู่ระบบไม่ถูกต้อง
4. การส่งคำร้องแจ้งซ่อม (TS-03) ทั้งกรณีข้อมูลถูกต้องและข้อมูลไม่ผ่านการตรวจสอบ
5. การลบคำร้องเฉพาะกรณีที่อยู่ในสถานะ PENDING และยังไม่มี RepairAssignment (TS-06) โดยทดสอบระดับ Controller ว่าระบบไม่ลบเมื่อ Service ปฏิเสธทั้งสองเงื่อนไข
6. การอนุมัติและปฏิเสธคำร้อง (TS-11) ทั้งกรณีปฏิเสธพร้อมเหตุผลและไม่ระบุเหตุผล
7. การมอบหมายงานให้ Technician (TS-12)
8. การอัปเดตสถานะของ RepairAssignment และการบันทึกหมายเหตุการดำเนินงาน (TS-31, TS-32) รวมถึงกรณีเปลี่ยนสถานะที่ไม่ถูกต้อง
9. การบันทึกประวัติการเปลี่ยนสถานะของงาน (RepairAssignmentStatusHistory) ใน TS-31 และการแสดงประวัติสถานะของคำร้อง (RepairRequestStatusHistory) ใน TS-05 และ TS-07
10. การตรวจสอบและยืนยันผลการซ่อม (TS-13 ถึง TS-15)
11. การแจ้งเตือนเมื่อสถานะงานเปลี่ยน โดยตรวจสอบการเรียกใช้ Subject ตาม Observer Pattern (TS-32)
12. การเข้าถึงโดยไม่มีสิทธิ์ ได้แก่ การใช้งานโดยไม่ได้เข้าสู่ระบบ (ถูกส่งกลับไปหน้าเข้าสู่ระบบ) และการเข้าถึงคำร้องของผู้ใช้อื่น (TS-07)

3.6 นำระบบไปใช้งานบน Cloud

ดำเนินการนำระบบไป Deploy บน Railway Cloud เพื่อให้สามารถเข้าถึงระบบผ่าน Internet ได้ Source Code ของระบบถูกจัดเก็บและควบคุมเวอร์ชันด้วย Git และ GitHub จากนั้น Railway เชื่อมต่อกับ Repository เพื่อสร้างและ Deploy Spring Boot Application

ระบบที่ Deploy บน Railway เชื่อมต่อกับบริการ Supabase ได้แก่ Supabase Auth สำหรับการยืนยันตัวตน และ Supabase PostgreSQL สำหรับจัดเก็บข้อมูล โดยกำหนดค่า Configuration และ Environment Variables สำหรับข้อมูลที่เป็นต่อการเชื่อมต่อบริการภายนอก

หลังจาก Deploy ระบบจะตรวจสอบการทำงานของ Application บนสภาพแวดล้อมจริง เช่น การเข้าสู่ระบบ การส่งคำร้อง การเชื่อมต่อฐานข้อมูล การเรียกใช้ API และการทำงานของฟังก์ชันต่าง ๆ เพื่อให้มั่นใจว่าระบบสามารถทำงานได้เหมือนกับสภาพแวดล้อมที่ใช้ในการพัฒนา

บทที่ 4

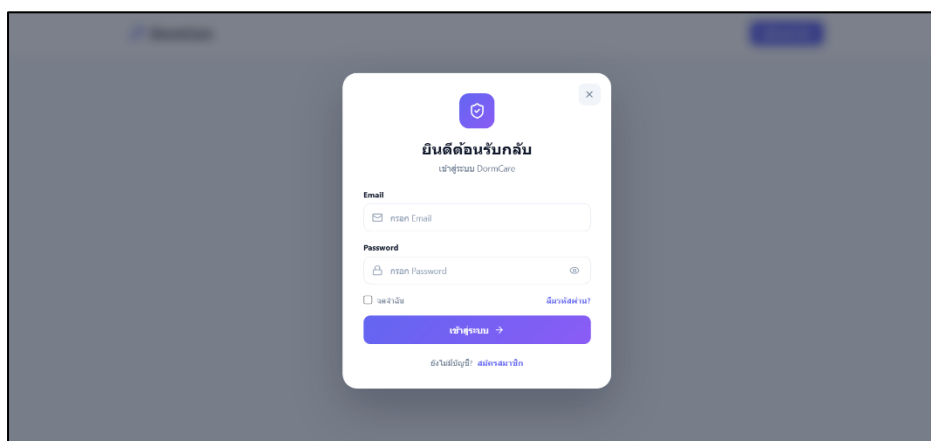
ผลการดำเนินงาน

4.1 ผลการพัฒนาเว็บแอปพลิเคชัน

ระบบแจ้งซ่อมหอพักที่พัฒนาขึ้นรองรับผู้ใช้งาน 3 บทบาท ได้แก่ ผู้แจ้ง ผู้ดูแลระบบ และช่าง โดยครอบคลุมตั้งแต่การสมัครสมาชิกและเข้าสู่ระบบ การแจ้งซ่อม การอนุมัติและมอบหมายงาน การอัปเดตสถานะงานซ่อม ไปจนถึงการตรวจสอบและยืนยันผลการซ่อม พร้อมระบบแจ้งเตือนสำหรับทุกบทบาท ผลการดำเนินงานแสดงตามหน้าจอการใช้งานของแต่ละบทบาทดังนี้

4.1.1 การเข้าสู่ระบบและสมัครสมาชิก

ผู้ใช้งานทุกบทบาทเข้าสู่ระบบด้วยอีเมลและรหัสผ่าน เมื่อสำเร็จระบบจะนำไปยังหน้าหลักของบทบาทนั้น ได้แก่ หน้าจัดการคำร้อง (ผู้ดูแลระบบ) หน้างานประจำวัน (ช่าง) และหน้ารายการคำร้อง (ผู้แจ้ง)



ภาพที่ 35 หน้าเข้าสู่ระบบ

ผู้แจ้งสมัครสมาชิกโดยกรอกข้อมูลส่วนตัวที่ต้องตรงกับข้อมูลผู้พักอาศัย (Resident) ที่มีอยู่ในระบบ และต้องยอมรับเงื่อนไขการใช้งานก่อนสมัคร

✕

สร้างบัญชีใหม่

สมัครสมาชิกเพื่อใช้งาน DormCare

First Name

Last Name

ชื่อ

นามสกุล

Username

ชื่อ Username

Phone Number

เบอร์ XXX-XXXX

Email

example@email.com

Room Number

ระบุ A101

Password

ภาพที่ 36 หน้าสมัครสมาชิกของผู้แจ้ง

4.1.2 ผู้แจ้ง (Reporter)

ผู้แจ้งกรอกรายละเอียดปัญหา ประเภทงานซ่อม และช่วงเวลาที่จะสะดวกให้เข้าซ่อม เพื่อส่งคำร้อง
แจ้งซ่อม

หน้าข้อมูลส่วนตัว

ประวัติการติดต่อ

ข้อมูลส่วนตัว

←

แบบฟอร์มแจ้งข้อมูลเบื้องต้น

ข้อมูลเบื้องต้น

ชื่อ-สกุล : นาย/นางสาว/นาง/นาย/นางสาว/นางสาว
นามสกุล : 4/001 ปี : 1

เบอร์โทรศัพท์ : 0953648757

📎 อัปโหลดรูป

คุณเคยมีประวัติการติดต่อจากหน่วยงานนี้หรือไม่?

ใช่

ไม่

ไม่แน่ใจ

สถานที่ / จังหวัด

ส่วนราชการ/หน่วยงานที่เกี่ยวข้อง...

วันเวลาที่ติดต่อ

ใน 13/04/2563

เวลา 10:00 - 11:00

หมายเหตุ

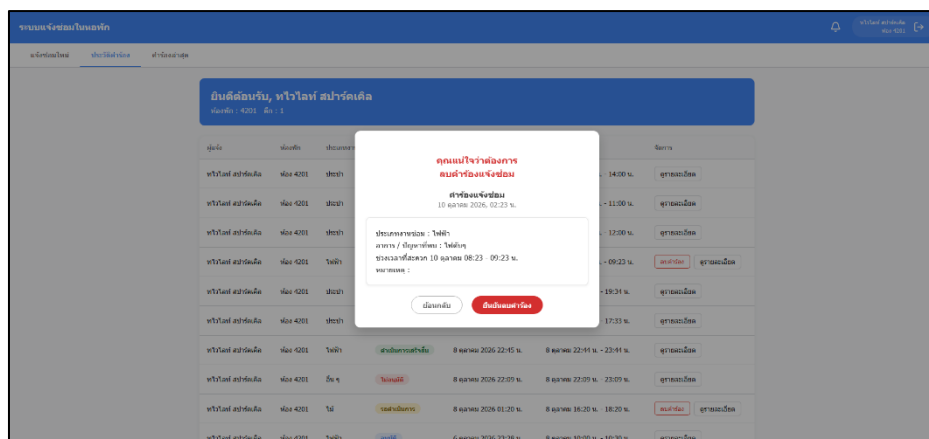
ขอแจ้งข้อมูลเบื้องต้น (ถ้ามี)

บันทึกข้อมูลเบื้องต้น พร้อมแจ้งข้อมูล

ประวัติการติดต่อ

ประวัติการติดต่อ

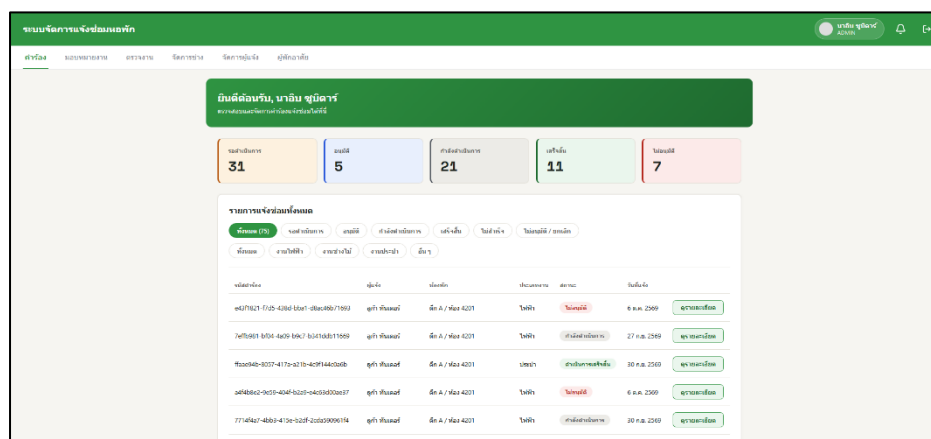
ภาพที่ 37 หน้าแจ้งซ่อมใหม่



ภาพที่ 40 หน้าลบคำร้องแจ้งซ่อม

4.1.3 ผู้ดูแลระบบ (Admin)

ผู้ดูแลระบบดูคำร้องทั้งหมดของระบบ พร้อมสถานะของแต่ละคำร้อง



ภาพที่ 41 หน้ารายการคำร้องทั้งหมด

ผู้ดูแลระบบตรวจสอบรายละเอียดคำร้อง และอนุมัติหรือปฏิเสธคำร้อง โดยการปฏิเสธสามารถระบุเหตุผลได้

ภาพที่ 42 หน้ารายละเอียดคำร้องและการพิจารณา

ผู้ดูแลระบบเลือกช่างและมอบหมายงานสำหรับคำร้องที่ได้รับอนุมัติแล้ว

ภาพที่ 43 หน้ามอบหมายงานให้ช่าง

เมื่อช่างดำเนินงานเสร็จสิ้น ผู้ดูแลระบบตรวจสอบงาน ประสานงานกับผู้แจ้ง และยืนยันผลการซ่อม

The screenshot shows a web form titled 'ตรวจซ่อมงาน' (Service Completion) for a 'PLUMBING' service. The form is divided into several sections:

- รายละเอียดงาน** (Job Details): A text area for 'ลักษณะงาน' (Job Description) with the value 'ถังอินทรี'.
- สถานะ** (Status): Two buttons, 'COMPLETED' (green) and 'IN PROGRESS' (orange).
- หมายเลขช่าง** (Technician Number): A text area for 'ช่างไม่ได้รับอนุญาต'.
- หมายเลขเคสเดิม** (Original Case Number): A text area for 'ช่างอีกทีมอื่น...'.
- ผู้แจ้ง** (Reporter): A section with 'ชื่อ' (Name) 'สุวิทย์ นามสกุล' and 'เบอร์' (Phone) '4201'. Below it, a 'โทรศัพท์มือถือ' (Mobile Phone) field with the number '087-380-2741' and a 'โทรออก' (Call) button.
- ช่างผู้รับผิดชอบ** (Responsible Technician): A section with a 'ช่าง' (Technician) field showing 'วิวัฒน์ นามสกุล' and a 'ช่างซ่อม' (Repairman) button.
- Buttons**: A green 'ยืนยันผลการซ่อม' (Confirm Service Completion) button and an orange 'มีปัญหามากกว่า 1 ครั้ง' (More than 1 problem) button.

ภาพที่ 44 หน้าตรวจสอบและยืนยันผลการซ่อม

ผู้ดูแลระบบดูรายการช่าง ค้นหา กรองตามความเชี่ยวชาญ สร้างบัญชีช่างใหม่ แก้ไขข้อมูล และดูประวัติการซ่อมของช่าง

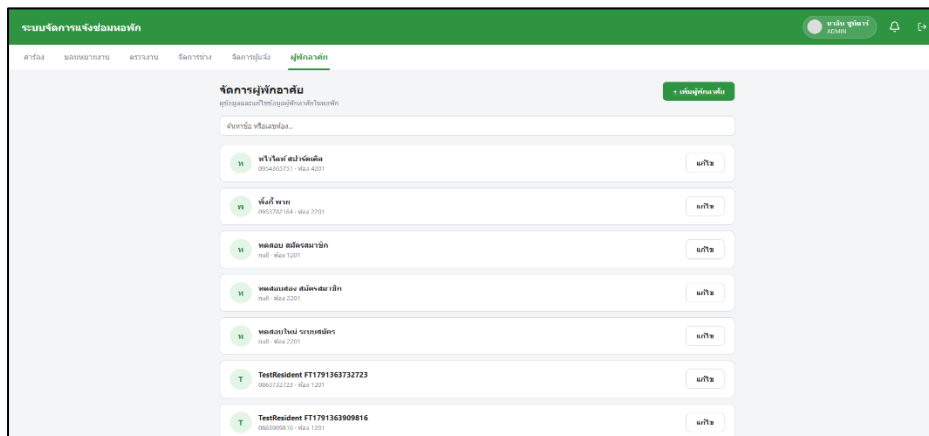
The screenshot shows a web interface titled 'ระบบจัดการช่างซ่อมหมวก' (Helmet Repair Technician Management System). The main section is 'จัดการช่างซ่อม' (Manage Technicians). It includes a search bar for 'ค้นหาชื่อ นามสกุล หรือเบอร์มือถือช่าง...' and a 'กดค้นหาช่าง' (Search Technician) button. Below the search bar is a list of technicians with the following details:

- ช่างซ่อม 1**: ชื่อ 'สมชาย ใจดี', เบอร์ '082-123-4567', สถานะ 'ว่าง' (Available).
- ช่างซ่อม 2**: ชื่อ 'สมชาย ใจดี', เบอร์ '082-123-4567', สถานะ 'ว่าง' (Available).
- ช่างซ่อม 3**: ชื่อ 'สมชาย ใจดี', เบอร์ '082-123-4567', สถานะ 'ว่าง' (Available).
- ช่างซ่อม 4**: ชื่อ 'สมชาย ใจดี', เบอร์ '082-123-4567', สถานะ 'ว่าง' (Available).
- ช่างซ่อม 5**: ชื่อ 'สมชาย ใจดี', เบอร์ '082-123-4567', สถานะ 'ว่าง' (Available).

Each technician entry has a 'ดูประวัติ' (View History) button and a 'แก้ไข' (Edit) button.

ภาพที่ 45 หน้าจัดการข้อมูลช่าง

ผู้ดูแลระบบรายการ เพิ่ม และแก้ไขข้อมูลผู้พักอาศัยและห้องพัก ซึ่งใช้ตรวจสอบกับข้อมูลของ
ผู้แจ้งต่อนสมัครสมาชิก



ภาพที่ 46 หน้าจัดการข้อมูลผู้พักอาศัย

4.1.4 ช่าง (Technician)

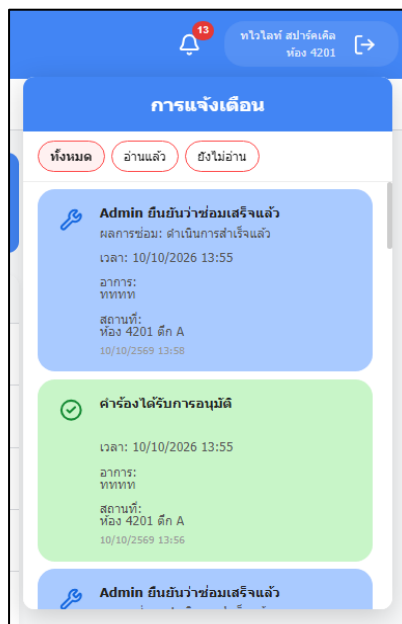
ช่างดูรายการงานซ่อมประจำวัน สรุปจำนวนงานตามสถานะ กรองงาน อัปเดตสถานะงาน และบันทึกหมายเหตุการดำเนินงาน



ภาพที่ 47 หน้างานซ่อมประจำวัน

4.1.5 การแจ้งเตือน

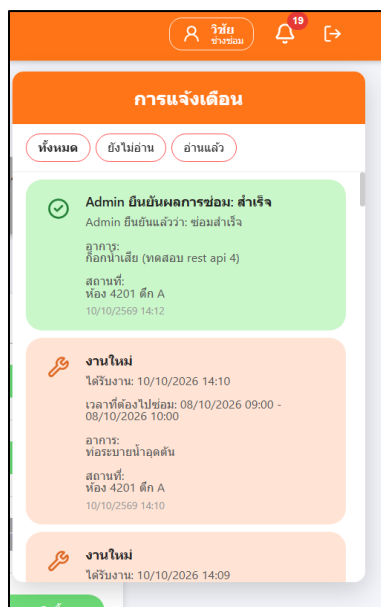
ระบบแจ้งเตือนผู้ใช้เมื่อเกิดเหตุการณ์สำคัญ เช่น มีคำร้องใหม่ ได้รับมอบหมายงาน หรืองานซ่อมเสร็จสิ้น โดยแต่ละบทบาทมีหน้าการแจ้งเตือนของตนเอง



ภาพที่ 48 หน้าการแจ้งเตือนของผู้แจ้ง



ภาพที่ 49 หน้าการแจ้งเตือนของผู้ดูแลระบบ



ภาพที่ 50 หน้าการแจ้งเตือนของช่าง

4.2 การทดสอบระบบแจ้งซ่อมในหอพัก

การทดสอบระบบดำเนินการเพื่อค้นหาข้อผิดพลาดและตรวจสอบว่าระบบทำงานตรงตามความต้องการที่กำหนดไว้ โดยครอบคลุมการทำงานของผู้ใช้งานทั้ง 3 บทบาท ได้แก่ ผู้แจ้ง (Reporter) ผู้ดูแลระบบ (Admin) และช่าง (Technician) ทั้งกรณีการทำงานปกติ (Normal Case) และกรณีที่ไม่เป็นไปตามเงื่อนไขของระบบ (Abnormal Case) เช่น การกรอกข้อมูลไม่ครบถ้วน การเข้าใช้งานโดยไม่ผ่านการเข้าสู่ระบบ หรือการเปลี่ยนสถานะงานที่ระบบไม่อนุญาต

การทดสอบแบ่งเป็น 33 ชุดทดสอบ (Test Suite: TS-01 ถึง TS-33) แต่ละชุดทดสอบประกอบด้วยกรณีทดสอบ (Test Case) ที่กำหนดรหัสในรูปแบบ TC-[ระดับ]-[หมายเลขชุด]-[ลำดับ] เช่น TC-UT-11-01 หมายถึง Unit Test ของชุด TS-11 กรณีที่ 1 โดยระดับการทดสอบใช้อักษรย่อ UT (Unit Test) IT (Integration Test) FT (Functional Test) และ UAT (User Acceptance Test) สำหรับ Unit Test ของ Service ในชุด TS-02 และ TS-07 ใช้รหัส TC-JU

การทดสอบแบ่งเป็น 4 ระดับ เพื่อตรวจสอบตั้งแต่การทำงานของส่วนย่อยของโปรแกรมไปจนถึงการใช้งานจริงของผู้ใช้ ดังตารางที่ 15

ตารางที่ 15 สรุประดับการทดสอบ เครื่องมือ และจำนวนกรณีทดสอบ

ระดับการทดสอบ	เครื่องมือ	จำนวนไฟล์
Unit Test	JUnit 5, Mockito	32
Integration Test	JUnit 5, Spring Boot Test, Mockito	33
Functional Test	JUnit 5, Selenium WebDriver (ChromeDriver)	22
User Acceptance Test (UAT)	JUnit 5, Selenium WebDriver (ChromeDriver)	16
รวม		103

ตารางที่ 16 ชุดทดสอบจำแนกตามบทบาทผู้ใช้งาน

บทบาท	ชุดทดสอบ	ฟังก์ชันที่ทดสอบ
ผู้แจ้ง (Reporter)	TS-01 ถึง TS-07	สมัครสมาชิกโดยตรวจสอบข้อมูลกับ Resident, เข้าสู่ระบบ, ส่งคำร้องแจ้งซ่อม, ดูรายการและรายละเอียดคำร้อง, ลบคำร้อง, ดูสถานะและประวัติสถานะของคำร้อง
ผู้ดูแลระบบ (Admin)	TS-08 ถึง TS-27	เข้าสู่ระบบ, ดูรายการและรายละเอียดคำร้อง, อนุมัติและปฏิเสธคำร้อง, มอบหมายงานให้ช่าง, ตรวจสอบและยืนยันผลการซ่อม, ประสานงานกับผู้แจ้ง, ดูประวัติการซ่อม, จัดการข้อมูลช่าง (TS-17 ถึง TS-20), จัดการข้อมูลผู้แจ้ง (TS-21 ถึง TS-23), จัดการข้อมูลผู้พักอาศัยและห้องพัก (TS-24 ถึง TS-27)
ช่าง (Technician)	TS-28 ถึง TS-33	ดูรายการงานซ่อมประจำวัน, กรองงานตามสถานะ, สรุปจำนวนงานประจำวัน, อัปเดตสถานะงาน, บันทึกหมายเหตุการดำเนินงาน, ดูข้อมูลส่วนตัว

ผลการทดสอบ

(1) การสมัครสมาชิกของ Reporter (TS-01)

กรณีปกติคือสมัครด้วยข้อมูลที่ถูกต้อง (TC-FT-01-02, TC-UAT-01-01) ส่วนกรณีผิดปกติคือระบบผู้พักอาศัยที่ไม่มีในระบบ ซึ่งระบบปฏิเสธการสมัคร (TC-FT-01-07) รวมถึงรหัสผ่านไม่ตรงกัน ไม่ยอมรับเงื่อนไข และไม่กรอกช่องที่จำเป็น นอกจากนี้ Unit Test ของ AuthServiceRegisterTest อีก 10 กรณีตรวจสอบความถูกต้องของข้อมูลที่กรอก

(2) การสร้างบัญชี Technician โดย Admin (TS-18)

Unit Test ครอบคลุมการสร้างสำเร็จ (TC-UT-18-01) ชื่อผู้ใช้งาน (TC-UT-18-02) และกรณีที่ Supabase ไม่ส่ง User ID หรือตอบข้อผิดพลาด (TC-UT-18-03, 18-04) Integration Test ตรวจสอบว่า API ตอบ HTTP 201 พร้อม JSON ที่ถูกต้อง (TC-IT-18-01 ถึง 18-03) และ Functional Test ตรวจสอบแบบฟอร์ม เช่น ส่งแบบฟอร์มว่าง และเบอร์โทรศัพท์ที่ไม่ถูกต้อง (TC-FT-18-03, 18-04)

(3) การเข้าสู่ระบบและการกำหนดสิทธิ์ตาม Role (TS-02, TS-08)

ระบบรับข้อมูลที่ถูกต้อง (TC-JU-02-01) และปฏิเสธเมื่อ request เป็น null อีเมลหรือรหัสผ่านว่าง รูปแบบอีเมลผิด หรือ Supabase ตอบข้อผิดพลาด (TC-JU-02-02 ถึง 02-09) เมื่อรหัสผ่านผิด ระบบกลับไปหน้าเข้าสู่ระบบ (TC-IT-02-02 ถึง 02-04) และเมื่อสำเร็จจะสร้าง Authentication และเก็บข้อมูลผู้ใช้ใน Session (TC-IT-02-05, 02-06) จากนั้นนำผู้ใช้ไปหน้าตามบทบาท คือ Admin ไป /admin/requests, Technician ไป /technician/dailywork และ Reporter ไป /reporter/requests โดยทดสอบเส้นทางของ Admin ใน TC-UT-08-05 และ TC-IT-08-05

(4) การส่งคำร้องแจ้งซ่อม (TS-03)

ระบบบันทึกคำร้องและเปลี่ยนเส้นทางเมื่อข้อมูลถูกต้อง (TC-UT-03-02, TC-IT-03-02) แต่จะกลับหน้าแบบฟอร์มเมื่อข้อมูลไม่ผ่านการตรวจสอบ หรือเมื่อ Service ส่ง IllegalArgumentException หรือ IllegalStateException (TC-UT-03-03 ถึง 03-05) และเปลี่ยนเส้นทางไปหน้าเข้าสู่ระบบเมื่อไม่มี userId (TC-IT-03-06)

(5) การลบคำร้องเฉพาะสถานะ PENDING และยังไม่มี RepairAssignment (TS-06)

เงื่อนไขนี้ถูกบังคับใน Service ด้วย BusinessException ส่วนการทดสอบระดับ Controller ตรวจสอบว่าลบคำร้อง PENDING สำเร็จ (TC-IT-06-01) เรียก Service ด้วย userId และ requestId ที่ถูกต้อง (TC-IT-06-02) และเมื่อจำลองให้ Service ปฏิเสธเพราะสถานะไม่ใช่ PENDING หรือถูกมอบหมายแล้ว (TC-IT-06-03, 06-05) ระบบต้องไม่ลบคำร้อง

(6) การอนุมัติและปฏิเสธคำร้อง (TS-11) ทดสอบการอนุมัติ (TC-UT-11-01)

การส่ง adminId ไปยัง Service (TC-UT-11-02) และการปฏิเสธทั้งแบบมีและไม่มีเหตุผล (TC-UT-11-03, 11-04) ส่วนกรณีที่ Admin ไม่ได้เข้าสู่ระบบ ระบบปฏิเสธการทำงานหรือเปลี่ยนเส้นทางไปหน้าเข้าสู่ระบบ (TC-UT-11-05, TC-IT-11-05) โดย TS-09 และ TS-10 ทดสอบการดูรายการและรายละเอียดคำร้องก่อนพิจารณา

7. การมอบหมายงานให้ Technician (TS-12)

ทดสอบการเปิดหน้ามอบหมายงาน การดึงข้อมูลคำร้องและรายชื่อช่าง การมอบหมายสำเร็จ และการปฏิเสธเมื่อไม่มี Admin ID ใน Session ทั้งระดับ Unit และ Integration ส่วน UAT (TC-UAT-12-01 ถึง 12-05) ทดสอบตั้งแต่การเข้าสู่ระบบของ Admin จนถึงกรอกแบบฟอร์มมอบหมายงาน

(8) การอัปเดตสถานะของ RepairAssignment (TS-31, TS-32)

TS-31 ตรวจสอบการเปลี่ยนสถานะสำเร็จ (TC-IT-31-01) กรณีไม่พบรายการมอบหมายหรือไม่พบช่าง (TC-IT-31-02, 31-03) และการปฏิเสธการเปลี่ยนสถานะที่ไม่ถูกต้อง (TC-IT-31-04) ส่วน TS-32 ตรวจสอบการบันทึกหมายเหตุพร้อมเปลี่ยนสถานะ ทั้งหมายเหตุปกติ ว่าง null และมีเฉพาะช่องว่าง และตรวจสอบว่าไม่บันทึกหมายเหตุเมื่อเปลี่ยนสถานะไม่ถูกต้อง (TC-UT-32-05, TC-IT-32-05) โดย TS-28 ถึง TS-30 ทดสอบการดึง กรอง และสรุปงานประจำวันซึ่งเป็นข้อมูลตั้งต้น

9. การบันทึก RepairRequestStatusHistory และ RepairAssignmentStatusHistory

ประวัติของงานที่มอบหมายถูกตรวจสอบว่าบันทึกเมื่อเปลี่ยนสถานะสำเร็จ (TC-IT-31-05) ส่วนประวัติของคำร้องทดสอบด้านการแสดงผล คือประวัติเรียงตามลำดับ (TC-JU-07-02) กรณีไม่มีประวัติต้องคืนรายการว่างโดยไม่เกิดข้อผิดพลาด (TC-JU-07-04) และหน้ารายละเอียดต้องมีทั้งคำร้องและประวัติ (TS-05)

10. การยืนยันผลการซ่อม (TS-13 ถึง TS-15)

TC-UT-13-01 ถึง 13-05 ตรวจสอบการเปิดหน้าตรวจสอบ การยืนยันงานเสร็จสิ้น และกรณีไม่มี Admin ID ส่วน ConfirmCompletionCommandTest ตรวจสอบการยืนยันทั้งแบบมีและไม่มีหมายเหตุ TS-14 ตรวจสอบการจัดลำดับงานโดยงานที่ช่างทำเสร็จแล้วต้องแสดงก่อน และงานที่ไม่มีข้อมูลการมอบหมายต้องไม่ถูกระบุว่าพร้อมตรวจสอบ ส่วน TS-15 ทดสอบการประสานงานกับผู้แจ้งก่อนยืนยันงาน

11. การแจ้งเตือนเมื่อเกิดเหตุการณ์สำคัญ

ระบบใช้ Observer Pattern จึงทดสอบว่าเมื่อเปลี่ยนสถานะงานสำเร็จ Service เรียก repairStatusSubject หนึ่งครั้ง (TC-UT-32-01) และไม่เรียกเมื่อเปลี่ยนสถานะไม่ถูกต้อง (TC-UT-32-05)

12. การเข้าถึงที่ไม่ตรงกับสิทธิ์

เมื่อไม่มี userId หรือ adminId ใน Session ระบบเปลี่ยนเส้นทางไปหน้าเข้าสู่ระบบ เช่น TC-IT-03-06, TC-UT-04-04, TC-UT-05-04, TC-UT-06-04, TC-UT-07-04, TC-IT-11-05 และ TC-UT-33-03 และระบบปฏิเสธเมื่อผู้แจ้งพยายามดูคำร้องของผู้อื่น (TC-JU-07-03)

บทที่ 5

สรุปผลการดำเนินงาน

5.1 สรุปผลการดำเนินงาน

การดำเนินโครงการระบบแจ้งซ่อมในหอพัก (Dormitory Repair Request System) มีวัตถุประสงค์เพื่อพัฒนาระบบสำหรับจัดการกระบวนการแจ้งซ่อมในหอพัก ตั้งแต่การส่งคำร้องแจ้งซ่อม การตรวจสอบและพิจารณา คำร้อง การมอบหมายงานให้ช่าง การดำเนินงานซ่อม การติดตามสถานะงาน และการยืนยันผลการซ่อม โดยระบบแบ่งการใช้งานตามบทบาทของผู้ใช้งาน ได้แก่ ผู้แจ้ง (Reporter) ผู้ดูแลระบบ (Admin) และช่าง (Technician) เพื่อให้แต่ละบทบาทสามารถดำเนินงานตามหน้าที่และสิทธิ์ที่กำหนดได้อย่างเหมาะสม

จากการดำเนินงานสามารถพัฒนาระบบให้รองรับกระบวนการทำงานหลักได้ โดยผู้แจ้งซ่อมสามารถสมัครสมาชิก เข้าสู่ระบบ ส่งคำร้องแจ้งซ่อม ตรวจสอบรายละเอียดและประวัติคำร้อง รวมถึงลบคำร้องได้ภายใต้เงื่อนไขที่กำหนด ส่วนผู้ดูแลระบบสามารถตรวจสอบคำร้อง พิจารณานุมัติหรือไม่อนุมัติคำร้อง มอบหมายงานให้ช่าง ตรวจสอบสถานะการดำเนินงาน และบันทึกผลการยืนยันการซ่อมได้ ขณะที่ช่างสามารถตรวจสอบงานที่ได้รับมอบหมายและปรับปรุงสถานะการดำเนินงานของงานซ่อมได้

ระบบมีการออกแบบโครงสร้างซอฟต์แวร์โดยแบ่งหน้าที่ออกเป็นส่วนต่าง ๆ ได้แก่ Controller, Service และ Repository เพื่อแยกความรับผิดชอบของแต่ละส่วนอย่างชัดเจน และประยุกต์ใช้หลักการออกแบบซอฟต์แวร์ เช่น SOLID, Dependency Injection และ Repository Pattern เพื่อช่วยให้ระบบมีโครงสร้างที่เป็นระเบียบ สามารถบำรุงรักษาและพัฒนาต่อได้ง่าย นอกจากนี้ยังประยุกต์ใช้ Design Pattern ของ GoF ได้แก่ State Pattern สำหรับจัดการพฤติกรรมของคำร้องตามสถานะ, Command Pattern สำหรับจัดการคำสั่งในการดำเนินงาน เช่น การมอบหมายงานและการยืนยันผล และ Observer Pattern สำหรับจัดการการแจ้งเตือนเมื่อเกิดเหตุการณ์ที่เกี่ยวข้องกับการเปลี่ยนแปลงสถานะหรือการมอบหมายงาน

ในด้านการออกแบบฐานข้อมูล ระบบใช้ฐานข้อมูล PostgreSQL บน Supabase โดยมีการออกแบบตารางให้รองรับข้อมูลผู้ใช้งาน ข้อมูลผู้พักอาศัย ห้องพัก คำร้องแจ้งซ่อม งานที่มอบหมายให้ช่าง และประวัติการเปลี่ยนแปลงสถานะของงาน มีการกำหนด Primary Key และ Foreign Key เพื่อรักษาความสัมพันธ์และความถูกต้องของข้อมูล รวมถึงกำหนดความสัมพันธ์ระหว่างเอนทิตีตามลักษณะของระบบ เช่น หนึ่งต่อหนึ่ง (One-to-One) และ หนึ่งต่อหลาย (One-to-Many)

ระบบยังมีการแบ่งส่วนการทำงานระหว่าง Web Application และ REST API เพื่อรองรับการใช้งานผ่านเว็บเบราว์เซอร์และการเรียกใช้บริการผ่าน API โดยมีการกำหนดสิทธิ์การเข้าถึงตามบทบาทของผู้ใช้งาน และใช้

Supabase Authentication สำหรับการยืนยันตัวตน ทั้งนี้ระบบได้รับการนำไปติดตั้งและใช้งานบน Railway Cloud เพื่อให้สามารถเข้าถึงระบบผ่านเครือข่ายอินเทอร์เน็ตได้

จากผลการดำเนินงาน ระบบสามารถสนับสนุนกระบวนการจัดการงานแจ้งซ่อมภายในหอพักได้อย่างเป็นระบบ ตั้งแต่การแจ้งปัญหาไปจนถึงการดำเนินการซ่อมและการยืนยันผล ช่วยลดการจัดการข้อมูลแบบกระจัดกระจาย เพิ่มความสะดวกในการติดตามสถานะงาน และทำให้ผู้เกี่ยวข้องสามารถตรวจสอบข้อมูลที่เกี่ยวข้องกับงานซ่อมได้อย่างชัดเจน

5.2 ปัญหาที่พบในการทำงาน

- (1) Service ทำหน้าที่หลายอย่าง เช่น map ข้อมูล และรับผิดชอบ business logic แก้ไขโดย Service ทำหน้าที่หลายอย่าง เช่น map ข้อมูล และรับผิดชอบ business logic
- (2) การตั้งชื่อ template ที่ไม่สอดคล้องกับ path ที่เขียนใน controller ทำให้เกิดปัญหาเวลา deploy แก้ไขโดยตรวจสอบชื่อ template และ path ให้สอดคล้องกัน

5.3 แนวทางในการพัฒนาต่อ

- (1) พัฒนาให้ระบบเชื่อมต่อกับฐานข้อมูลสัญญาเช่าอัตโนมัติ เพื่อเพิ่มประสิทธิภาพในการจัดการข้อมูลและลดขั้นตอนการนำเข้าข้อมูลผู้พักด้วยตนเอง
- (2) พัฒนาระบบการแจ้งเตือนให้มีความหลากหลายมากขึ้น สามารถพัฒนาการแจ้งเตือนให้รองรับ Email, Notification ภายในระบบ หรือช่องทางอื่น เพื่อให้ผู้แจ้งซ่อมและผู้ดูแลระบบได้รับข้อมูลเกี่ยวกับการเปลี่ยนแปลงสถานะของคำร้องอย่างรวดเร็ว
- (3) พัฒนาระบบรายงานและสถิติ เพิ่ม Dashboard และรายงานสถิติ เช่น จำนวนคำร้องในแต่ละช่วงเวลา จำนวนงานที่ดำเนินการเสร็จ จำนวนงานที่อยู่ระหว่างดำเนินการ ประเภทปัญหาที่พบมากที่สุด และระยะเวลาเฉลี่ยในการดำเนินงาน เพื่อช่วยให้ผู้ดูแลระบบสามารถนำข้อมูลไปวิเคราะห์และวางแผนการจัดการงานซ่อมได้
- (4) เพิ่มระบบแนบรูปภาพหรือหลักฐานประกอบคำร้อง ควรเพิ่มความสามารถในการแนบรูปภาพปัญหาหรือบริเวณที่ต้องการซ่อม เพื่อช่วยให้ผู้ดูแลระบบและช่างสามารถเข้าใจลักษณะของปัญหาได้ชัดเจนก่อนดำเนินการซ่อม
- (5) เพิ่มระบบประเมินความพึงพอใจหลังการซ่อม หลังจากงานซ่อมเสร็จสิ้น ควรเปิดโอกาสให้ผู้แจ้งประเมินความพึงพอใจและแสดงความคิดเห็นเกี่ยวกับการให้บริการ เพื่อนำข้อมูลไปใช้ในการปรับปรุงคุณภาพการให้บริการของช่างและระบบโดยรวม

- (6) พัฒนาระบบให้รองรับการใช้งานบนอุปกรณ์ที่หลากหลาย ควรปรับปรุงส่วนติดต่อผู้ใช้งานให้เป็น Responsive Web Design เพื่อให้สามารถใช้งานได้อย่างเหมาะสมทั้งบนคอมพิวเตอร์ แท็บเล็ต และ โทรศัพท์มือถือ